weft 1.0
A Modern Literate Programming Tool

Cuauhtemoc Pacheco
Derived from nuweb 1.64 by Preston Briggs[1]
with contributions by Keith Harwood, Simon Wright,
Marc W. Mengel, John D. Ramsdell, and Gregor Goldbach

March 3, 2026

[1]Original nuweb work supported by ARPA, through ONR grant N00014-91-J-1989.

Contents

1 Introduction **1**
- 1.1 weft . . . 1
- 1.2 Writing weft . . . 3
  - 1.2.1 Source File Extension: `.weft` . . . 3
  - 1.2.2 The Major Commands . . . 4
  - 1.2.3 The Minor Commands . . . 7
- 1.3 Sectioning commands . . . 8
- 1.4 Running weft . . . 9
- 1.5 Markdown Output . . . 10
- 1.6 Section Markers and Language Auto-Detection . . . 11
  - 1.6.1 Overriding the Language with `@l` . . . 11
  - 1.6.2 The `@L` Command: Defining New Languages . . . 12
  - 1.6.3 The Language Table . . . 13
  - 1.6.4 Syntax Highlighting in Markdown Output . . . 13
  - 1.6.5 Output Format of Section Markers . . . 13
  - 1.6.6 Debugging Tangled Code . . . 13
- 1.7 Restrictions . . . 17
- 1.8 Project Introspection . . . 18
  - 1.8.1 Map (`-m`) . . . 18
  - 1.8.2 Extract (`-e`) . . . 18
- 1.9 Acknowledgements . . . 18

2 The Overall Structure **20**
- 2.1 Files . . . 20
  - 2.1.1 The Main Files . . . 22
  - 2.1.2 Support Files . . . 24
- 2.2 The Main Routine . . . 24
  - 2.2.1 Command-Line Arguments . . . 25
  - 2.2.2 File Names . . . 32
- 2.3 Pass One . . . 38
  - 2.3.1 Accumulating Definitions . . . 42
  - 2.3.2 Block Comments . . . 44
  - 2.3.3 Fixing the Cross References . . . 50
  - 2.3.4 Dealing with fragment parameters . . . 50
- 2.4 Writing the Latex File . . . 54
  - 2.4.1 Formatting Definitions . . . 58
  - 2.4.2 Generating the Indices . . . 77
- 2.5 Writing the Markdown File . . . 84
  - 2.5.1 Formatting Definitions in Markdown . . . 88
  - 2.5.2 Generating the Indices in Markdown . . . 101
  - 2.5.3 Identifier Cross-References in Markdown . . . 106
- 2.6 Map and Extract Output . . . 109

2.6.1 JSON Map Output . . . 109
2.6.2 Fragment Extraction . . . 112
2.7 Writing the Output Files . . . 114

3 The Support Routines **118**
3.1 Source Files . . . 118
3.1.1 Global Declarations . . . 118
3.1.2 Local Declarations . . . 119
3.1.3 Reading a File . . . 119
3.1.4 Opening a File . . . 123
3.2 Scraps . . . 124
3.2.1 Collecting Page Numbers . . . 149
3.3 Names . . . 151
3.3.1 Language Auto-Detection . . . 159
3.3.2 User-Defined Languages . . . 161
3.3.3 Weave Format Declaration . . . 163
3.3.4 Collecting File and Fragment Names . . . 164
3.3.5 Comment Styles . . . 166
3.4 Searching for Index Entries . . . 176
3.4.1 Retrieving scrap uses . . . 177
3.4.2 Building the Automata . . . 180
3.4.3 Searching the Scraps . . . 184
3.5 Labels . . . 190
3.6 Memory Management . . . 193
3.6.1 Allocating Memory . . . 194
3.6.2 Freeing Memory . . . 195

4 Man page **196**

5 Indices **201**
5.1 Files . . . 201
5.2 Fragments . . . 201
5.3 Identifiers . . . 205

Chapter 1

Introduction

In 1984, Knuth introduced the idea of *literate programming* and described a pair of tools to support the practise [3]. His approach was to combine Pascal code with TEX documentation to produce a new language, `WEB`, that offered programmers a superior approach to programming. He wrote several programs in `WEB`, including `weave` and `tangle`, the programs used to support literate programming. The idea was that a programmer wrote one document, the web file, that combined documentation (written in TEX [4]) with code (written in Pascal).

Running `tangle` on the web file would produce a complete Pascal program, ready for compilation by an ordinary Pascal compiler. The primary function of `tangle` is to allow the programmer to present elements of the program in any desired order, regardless of the restrictions imposed by the programming language. Thus, the programmer is free to present his program in a top-down fashion, bottom-up fashion, or whatever seems best in terms of promoting understanding and maintenance.

Running `weave` on the web file would produce a TEX file, ready to be processed by TEX. The resulting document included a variety of automatically generated indices and cross-references that made it much easier to navigate the code. Additionally, all of the code sections were automatically prettyprinted, resulting in a quite impressive document.

Knuth also wrote the programs for TEX and METAFONT entirely in `WEB`, eventually publishing them in book form [5, 6]. These are probably the largest programs ever published in a readable form.

Inspired by Knuth's example, many people have experimented with `WEB`. Some people have even built web-like tools for their own favorite combinations of programming language and typesetting language. For example, `CWEB`, Knuth's current system of choice, works with a combination of C (or C++) and TEX [8]. Another system, FunnelWeb, is independent of any programming language and only mildly dependent on TEX [10]. Inspired by the versatility of FunnelWeb and by the daunting size of its documentation, Preston Briggs decided to write his own, very simple, tool for literate programming.[1]

weft is a fork of nuweb. The original nuweb code, written by Preston Briggs and maintained over the years by several contributors, provided an excellent foundation: a simple, language-independent literate programming tool. weft builds on that foundation with a modernized codebase, a modular source structure organized by concept, the removal of the unmaintained HTML output path, and the addition of a Markdown weave output format. The goal is to continue evolving the tool while preserving the simplicity and philosophy that made nuweb valuable in the first place.

1.1 weft

weft works with any programming language and produces documentation in LATEX [7] or Markdown format. LATEX was the original choice because it supports a multi-level sectioning scheme and has facilities for drawing figures. Markdown support was added to allow literate programs to produce lightweight documentation

[1] There is another system similar to nuweb, written by Norman Ramsey, called *noweb* [9]. It perhaps suffers from being overly Unix-dependent and requiring several programs to use. On the other hand, its command syntax is very nice. In any case, nuweb certainly owes its name and a number of features to his inspiration.

that renders well on platforms like GitHub, GitLab, and any Markdown viewer, without requiring a TEX installation.

Support for arbitrary programming languages was a design goal because programmers work in many languages (and sometimes combinations of several languages), *e.g.,* C, Fortran, C++, yacc, lex, Scheme, assembly, Postscript, and so forth. The need to support arbitrary programming languages has many consequences:

No prettyprinting Both `WEB` and `CWEB` are able to prettyprint the code sections of their documents because they understand the language well enough to parse it. Since weft aims to support *any* language, this feature must be abandoned. However, weft does allow particular individual formulas or fragments of LATEX code to be formatted and still be parts of output files. Also, keywords in scraps can be surrounded by `@_` to have them be bold in the output.

No index of identifiers Because `WEB` knows about Pascal, it is able to construct an index of all the identifiers occurring in the code sections (filtering out keywords and the standard type identifiers). Unfortunately, this isn't as easy for a language-independent tool. weft doesn't know what an identifier looks like in each language and certainly doesn't know all the keywords. (On the other hand, see the end of Section 1.2.3)

Of course, there must be some compensation for these losses or the whole idea would be a waste. Here are the advantages:

Simplicity The majority of the commands in `WEB` are concerned with control of the automatic prettyprinting. Since weft doesn't prettyprint, many commands are eliminated. A further set of commands is subsumed by LATEX and may also be eliminated. As a result, the set of commands is reduced to only four members (explained in the next section). This simplicity is also reflected in the size of this tool, which is quite a bit smaller than the tools used with other approaches.

No prettyprinting Everyone disagrees about how their code should look, so automatic formatting annoys many people. One approach is to provide ways to control the formatting. weft's approach is simpler—it performs no automatic formatting and therefore allows the programmer complete control of code layout. weft does allow individual scraps to be presented in either verbatim, math, or paragraph mode in the TEX output.

Control weft also offers the programmer complete control of the layout of his output files (the files generated during tangling). Of course, this is essential for languages that are sensitive to layout; but it is also important in many practical situations, *e.g.,* debugging.

Speed Since weft doesn't do too much, the weft tool runs quickly. By default it performs only tangling; weaving is activated with `-w`. When weaving is requested, both operations execute in a single pass.

Page numbers Inspired by the example of noweb, weft refers to all scraps by page number to simplify navigation. If there are multiple scraps on a page (say, page 17), they are distinguished by lower-case letters (*e.g.,* 17a, 17b, and so forth).

Multiple file output The programmer may specify more than one output file in a single weft file. This is required when constructing programs in a combination of languages (say, Fortran and C). It's also an advantage when constructing very large programs that would require a lot of compile time.

This last point is very important. By allowing the creation of multiple output files, the need for monolithic programs is avoided. This supports the creation of very large programs by groups of people.

A further reduction in compilation time is achieved by first writing each output file to a temporary location, then comparing the temporary file with the old version of the file. If there is no difference, the temporary file can be deleted. If the files differ, the old version is deleted and the temporary file renamed. This approach works well in combination with `make` (or similar tools), since `make` will avoid recompiling untouched output files.

1.2 Writing weft

The bulk of a weft file will be ordinary LaTeX (or Markdown, when using the `-w md` flag). In fact, any LaTeX file can serve as input to weft and will be simply copied through, unchanged, to the documentation file—unless a weft command is discovered. Similarly, any Markdown file can be processed with `-w md` and will be copied through to the `.md` output. All weft commands begin with an "at-sign" (`@`). Therefore, a file without at-signs will be copied unchanged. weft commands are used to specify *output files,* define *fragments,* and delimit *scraps*. These are the basic features of interest to the weft tool—all else is simply text to be copied to the documentation file.

1.2.1 Source File Extension: `.weft`

weft source files use the `.weft` extension. This single, format-agnostic extension replaces the earlier `.w` and `.wmd` conventions. The prose surrounding scraps may be written in LaTeX or Markdown; the at-sign commands (`@o`, `@d`, `@<`, etc.) are the same in both cases.

The weave output format is specified in one of two ways:

In the source file The `@W` directive declares the intended format: `@W tex` for LaTeX, `@W md` for Markdown. This should appear in the preamble, before any `@o` or `@d` directives.

On the command line `-w md` or `-w tex` overrides the `@W` directive. Using `-w` alone (without a format token) reads the format from `@W`.

If no extension is given on the command line, weft appends `.weft` automatically.

Default Behavior: Tangle Only

By default, `weft myfile.weft` performs only tangling (producing the output files). No documentation is generated unless `-w` is given:

```
weft programa.weft            tangle only (default)
weft -w programa.weft         weave per @W (error if missing)
weft -w md programa.weft      weave Markdown (override)
weft -w tex programa.weft     weave LaTeX (override)
```

The `-t` flag suppresses weaving even if `-w` is given. The `-o` flag suppresses tangling.

Cosmetic Fences

When writing `.weft` files with Markdown prose, it is often convenient to surround scrap definitions with Markdown fenced code blocks (triple backticks) so that the source file renders nicely in Markdown previewers such as GitHub or VS Code. For example:

```
Some prose in Markdown.

‘‘‘c
@d My fragment
@{code here
@}
‘‘‘

More prose.
```

weft silently strips these fence lines (lines that consist of three or more backticks, optionally followed by a language identifier) during Markdown weaving. They do not appear in the generated `.md` file and do not affect tangling. This is purely a cosmetic convenience for making the source file readable in any Markdown viewer.

Note that fences are only stripped in Markdown mode (`-w md`). In LaTeX mode, backtick lines are treated as ordinary text.

1.2.2 The Major Commands

Files and fragments are defined with the following commands:

`@o` ***file-name flags scrap*** Output a file. The file name is terminated by whitespace.

`@d` ***fragment-name scrap*** Define a fragment. The fragment name is terminated by a return or the beginning of a scrap.

`@q` ***fragment-name scrap*** Define a fragment. The fragment name is terminated by a return or the beginning of a scrap. This a quoted fragment.

A specific file may be specified several times, with each definition being written out, one after the other, in the order they appear. The definitions of fragments may be similarly specified piecemeal.

Fragment Names with Embedded Parameters

Fragment names can carry *parameters*—placeholder values embedded directly in the name using the `@'value@'` syntax. Inside the fragment body, `@1`, `@2`, ...`@9` refer to the first, second, ...ninth parameter in order of appearance.

For example, a reusable validation fragment:

```
@d Validate field @'name@' type @'string@'
@{if (!data.@1) throw new Error("Missing @1");
if (typeof data.@1 !== '@2')
  throw new TypeError("@1 must be @2");
@}
```

Each invocation supplies its own arguments:

```
@<Validate field @'email@' type @'string@'@>
@<Validate field @'age@'   type @'number@'@>
```

The first invocation expands to:

```
if (!data.email) throw new Error("Missing email");
if (typeof data.email !== 'string')
  throw new TypeError("email must be string");
```

Note three key points:

- The values in the *definition* (`@'name@'`, `@'string@'`) serve as default arguments and as documentation—they make the fragment name readable.
- At each *invocation*, different values can be substituted (`@'email@'` replaces `@1`, `@'string@'` replaces `@2`).
- If a fragment name is abbreviated with `...`, any missing arguments fall back to the defaults given in the definition.

The difference between a quoted fragment (`@q`) and an ordinary one (`@d`) is that inside a quoted fragment fragments are not expanded on output. Rather, they are formatted as uses of fragments so that the output file can itself be `weft` source. This allows you to create files containing fragments which can undergo further processing before the fragments are expanded, while also describing and documenting them in one place.

You can have both quoted and unquoted fragments with the same name. They are written out in order as usual, with those introduced by `@q` being quoted and those with `@d` expanded as normal.

In quoted fragments, the `@f` filename is quoted as well, so that when it is expanded it refers to the finally produced file, not any of the intermediate ones.

Scraps

Scraps have specific begin markers and end markers to allow precise control over the contents and layout. Note that any amount of whitespace (including carriage returns) may appear between a name and the beginning of a scrap. Scraps may also appear in the running text where they are formatted (almost) identically to their use in definitions, but don't appear in any code output.

`@{`*`anything`*`@}` where the scrap body includes every character in *anything*—all the blanks, all the tabs, all the carriage returns. This scrap will be typeset in verbatim mode. Using the `-l` option will cause the program to typeset the scrap with the help of LaTeX's `listings` package.

`@[`*`anything`*`@ ]` where the scrap body includes every character in *anything*—all the blanks, all the tabs, all the carriage returns. This scrap will be typeset in paragraph mode, allowing sections of TeX documents to be scraps, but still be prettyprinted in the document.

`@(`*`anything`*`@)` where the scrap body includes every character in *anything*—all the blanks, all the tabs, all the carriage returns. This scrap will be typeset in math mode. This allows this scrap to contain a formula which will be typeset nicely.

Inside a scrap, several special sequences are recognized.

Fragment invocation.

`@<`*`fragment-name`*`@>` Expands the named fragment inline. It is an error to specify recursive invocations. If the fragment name contains embedded parameters, the invocation supplies arguments at the same positions (see above).

Argument substitution.

`@1, @2, ..., @9` Inside a fragment body, substitutes the nth argument. If the argument was not supplied, the empty string is used.

At each argument position in an invocation, four forms are accepted:

`@'string@'` A literal string, copied verbatim (only `@@` is interpreted, producing a single `@`). This is the most common form.

`@<`*`fragment-name`*`@>` The fragment is expanded and its output becomes the argument value.

`@{text@}` The text is expanded as an anonymous fragment (inheriting the caller's arguments) and the result is passed as the argument. Useful for combining text and arguments into a single value.

`@1, @2, ..., @9` Forwards an argument from the calling fragment to the called one.

Other in-scrap commands.

`@x`*`label`*`@x` Marks a place inside a scrap and associates it to the label (which can be any text not containing a @). Expands to the reference number of the scrap followed by a numeric value. Outside scraps it expands to the same value. It is used so that text outside scraps can refer to particular places within scraps.

`@f` Inside a scrap this is replaced by the name of the current output file.

`@t` Inside a scrap this is replaced by the title of the fragment as it is at the point it is used, with all parameters replaced by actual arguments.

`@#` At the beginning of a line in a scrap this will suppress the normal indentation for that line. Use this, for example, when you have a `#ifdef` inside a nested scrap. Writing `@##ifdef` will cause it to be lined up on the left rather than indented with the rest of its code.

`@s` Inside a scrap supresses indentation for the following fragment expansion.

Note that fragment names may be abbreviated, either during invocation or definition. For example, it would be very tedious to have to type, repeatedly, the fragment name

```
@d Check for terminating at-sequence and return name if found
```

Therefore, weft provides a mechanism (borrowed from Knuth) of indicating abbreviated names.

```
@d Check for terminating...
```

Basically, the programmer need only type enough characters to identify the fragment name uniquely, followed by three periods. An abbreviation may even occur before the full version; weft simply preserves the longest version of a fragment name. Note also that blanks and tabs are insignificant within a fragment name; each string of them is replaced by a single blank.

Sometimes, for instance during program testing, it is convenient to comment out a few lines of code. In C or Fortran placing `/* ... */` around the relevant code is not a robust solution, as the code itself may contain comments. weft provides the command

```
@%
```

only to be used inside scraps. It behaves exactly the same as `%` in the normal LaTeX text body.

When scraps are written to a program file or a documentation file, tabs are expanded into spaces by default. Currently, weft assumes tab stops are set every eight characters. Furthermore, when a fragment is expanded in a scrap, the body of the fragment is indented to match the indentation of the fragment invocation. Therefore, care must be taken with languages (*e.g.,* Fortran) that are sensitive to indentation. These default behaviors may be changed for each output file (see below).

Flags

When defining an output file, the programmer has the option of using flags to control output of a particular file. The flags are intended to make life a little easier for programmers using certain languages. They introduce little language dependences; however, they do so only for a particular file. Thus it is still easy to mix languages within a single document. There are five "per-file" flags:

`-d` Forces the creation of `#line` directives in the output file. These are useful with C (and sometimes C++ and Fortran) on many Unix systems since they cause the compiler's error messages to refer to the web file rather than to the output file. Similarly, they allow source debugging in terms of the web file.

`-i` Suppresses the indentation of fragments. That is, when a fragment is expanded within a scrap, it will *not* be indented to match the indentation of the fragment invocation. This flag would seem most useful for Fortran programmers.

`-t` Suppresses expansion of tabs in the output file. This feature seems important when generating `make` files.

`-c`x Puts comments in generated code documenting the fragment that generated that code. The x selects the comment style for the language in use: `c` for C (`/* */`), `+` for C++ (`//`), and `p` for shell/Perl (`#`). If the global `-x` cross-reference flag is set the comment includes the page reference for the first scrap that generated the code.

`-s` Suppresses section markers and `#line` directives in the tangled output for this file. The language is still detected (from the extension or a preceding `@l`), so the Markdown weaver can emit syntax-highlighting classes, but the tangled code is emitted without any tangle annotations. This is useful for bootstrap source files or generated output that should remain clean.

These flags are still available for explicit control, but in most cases the automatic language detection described in Section 1.6 is more convenient.

1.2.3 The Minor Commands

There are some very low-level utility commands that may appear anywhere in the web file.

`@@` Causes a single "at sign" to be copied into the output.

`@_` Causes the text between it and the next `@_` to be made bold (for keywords, etc.)

`@i` *`file-name`* Includes a file. Includes may be nested, though there is currently a limit of 10 levels. The file name should be complete (no extension will be appended) and should be terminated by a carriage return. Normally the current directory is searched for the file to be included, but this can be varied using the `-I` flag on the command line. Each such flag adds one directory to the search path and they are searched in the order given.

`@r`x Changes the escape character '@' to 'x'. This must appear before any scrap definitions.

`@v` Always replaced by the string established by the `-V` flag, or a default string if the flag isn't given. This is intended to mark versions of the generated files.

`@l` ***tag*** Sets the language for the next `@o` or `@d` directive. The tag is looked up in the language table (Section 1.6) to determine the comment style for section markers and the `class` attribute for Markdown syntax highlighting. Placed on the line before the directive it applies to.

`@L` ***name style*** `[+d]` Defines a custom language mapping for the section marker system (see Section 1.6). This must appear in the document preamble, before any `@o` or `@d` directives that depend on it.

`@W` ***format*** Declares the weave output format for this source file. Valid formats are `md` (Markdown) and `tex` (LaTeX). Used by `-w` when no explicit format is given on the command line. A CLI format (`-w md` or `-w tex`) overrides this directive. Should appear in the document preamble.

In the text of the document, that is outside scraps, you may include scrap-like material.

`@{`*`Anything`*`@}` The included material, the *Anything*, is typeset as if it appeared inside a scrap. This is useful for referring to fragments in the text and for describing the literate programming process itself.

`@<`*`Fragment name`*`@>` The fragment named is expanded in place in the text. The expansion is presented verbatim, it is not interpreted for typesetting, so any special environment must be set up before and after this is used.

`@c` Outside of a scrap this is used to define a block comment using all the text until the next command. Inside a scrap the last block comment is expanded using the current indentation and breaking lines between words if 60 characters are exceeded.

Finally, there are three commands used to create indices to the fragment names, file definitions, and user-specified identifiers.

`@f` Create an index of file names.

`@m` Create an index of fragment names.

`@u` Create an index of user-specified identifiers.

These are usually placed in their own section in the LaTeX document; for example, see Chapter 5.

Identifiers must be explicitly specified for inclusion in the `@u` index. By convention, each identifier is marked at the point of its definition; all references to each identifier (inside scraps) will be discovered automatically. To "mark" an identifier for inclusion in the index, it must be mentioned at the end of a scrap. For example,

```
@d a scrap @{
Suppose the variables FOO and BAR are being declared
inside this scrap.
@| FOO BAR @}
```

Alphabetic identifiers have been used in this example, but any string of characters (not including whitespace or `@` characters) will do. Therefore, it's possible to add index entries for things like `<<=` if desired. An identifier may be declared in more than one scrap.

In the generated index, each identifier appears with a list of all the scraps using and defining it, where the defining scraps are distinguished by underlining. Note that the identifier doesn't actually have to appear in the defining scrap; it just has to be in the list of definitions at the end of a scrap.

1.3 Sectioning commands

For larger documents the indexes and usage lists get rather unwieldy and problems arise in naming things so that different things in different parts of the document don't get confused. weft has a sectioning command which keeps the fragment names and user identifiers separate. Thus, you can give a fragment in one section the same name as a fragment in another and the two won't be confused or connected in any way. Nor will user identifiers defined in one section be referenced in another. Except for the fact that scraps in successive sections can go into the same output file, this is the same as if the sections came from separate input files.

However, occasionally you may really want fragments from one section to be used in another. More often, you will want to identify a user identifier in one section with the same identifier in another (as, for example, a header file defined in one section is included in code in another). Extra commands allow a fragment defined in one section to be accessible from all other sections. Similarly, you can have scraps which define user identifiers and export them so that they can be used in other sections.

`@s` Start a new section.

`@S` Close the current section and don't start another.

`@d+` **fragment-name scrap** Define a fragment which is accessible in all sections, a global fragment.

`@D+` Likewise

`@q+` Likewise

`@Q+` Likewise

`@m+` Create an index of all such fragments.

`@u+` Create an index of globally accessible user identifiers.

There are two kinds of section, the base section which is where normally everything goes, and local sections which are introduced with the `@s` command. A local section comprises everything from the command which starts it to the one which ends it. A `@s` command will start a new local section. A `@S` command closes the current local section, but doesn't open another, so what follows goes into the base section. Note that fragments defined in the base section aren't global; they are accessible only in the base section, but they are accessible regardless of any local sections between their definition and their use.

Within a scrap:

`@<+`*fragment-name*`@>` Expand the globally accessible fragment with that name, rather than any local fragment.

`@+` Like `@|` except that the identifiers defined are exported to the global realm and are not directly referenced in any scrap in any section (not even the one where they are defined).

`@-` Like `@|` except that the identifiers are imported to the local realm. The cross-references show where the global variables are defined and defines the same names as locally accesible. Uses of the names within the section will point to this scrap.

Note that the `+` signs above are part of the commands. They are not part of the fragment names. If you want a fragment whose name begins with a plus sign, leave a space between the command and the name.

1.4 Running weft

weft is invoked using the following command:

> `weft` *flags file-name...*

One or more files may be processed at a time. If a file name has no extension, `.weft` will be appended. While a file name may specify a file in another directory, the resulting documentation file will always be created in the current directory. For example,

```
weft /foo/bar/quux
```

will take as input the file `/foo/bar/quux.weft` and will produce the tangled output files only. To also generate documentation, use `-w` (see below).

By default, weft performs only tangling—it generates the output files defined by `@o` directives and nothing else. Documentation (weaving) is produced only when the `-w` flag is given. When `-w` is present, both tangling and weaving execute in a single pass. Three flags control what weft produces:

`-t` Suppress weaving (useful to override `-w` in scripts).

`-o` Suppress generation of the output files (weave only).

`-c` Avoid testing output files for change before updating them.

Thus, the command

```
weft -to /foo/bar/quux
```

would simply scan the input and produce no output at all.

There are several additional command-line flags:

`-v` For "verbose", causes weft to write information about its progress to `stderr`.

`-n` Forces scraps to be numbered sequentially from 1 (instead of using page numbers). This form is perhaps more desirable for small webs.

`-s` Doesn't print list of scraps making up each file following each scrap.

`-d` Print "dangling" identifiers – user identifiers which are never referenced, in indices, etc.

`-p` *path* Prepend *path* to the filenames for all the output files.

`-w` *[format]* Activate weaving (documentation generation). Without `-w`, no documentation file is produced. The optional *format* argument selects the output format:

- `-w md` — force Markdown output (`.md` file).
- `-w tex` — force LaTeX output (`.tex` file).
- `-w` alone — read the format from the `@W` directive in the source file. If `@W` is absent, weft reports an error.

A format given on the command line overrides any `@W` directive. When Markdown output is selected, sequential numbering (`-n`) is forced automatically, since Markdown has no concept of page numbers. The Markdown output uses HTML anchors (`<a id="...">`) for cross-references and `<pre><code>` blocks for scraps, allowing clickable navigation between definitions and references. Cosmetic fence lines (triple backticks) in the source are silently stripped.

`-x` Include cross-reference numbers in scrap comments.

`-h` *options* Provide options for the hyperref package.

`-r` Turn on hyperlinks. You must include the —usepackage— options in the text.

`-l` Use the `listings` package for formatting scraps. Use this if you want to have a pretty-printer for your scraps. In order to e.g. have pretty Perl scraps, include the following LaTeX commands in your document:

```
\usepackage{listings}
...
\lstset{extendedchars=true,keepspaces=true,language=perl}
```

See the `listings` documentation for a list of formatting options. Be sure to include a `\usepackage{listings}` in your document.

`-V` *string* Provide *string* as the replacement for the @v operation.

`-m` Emit a JSON map of the project to stdout and exit. The map describes every output file and every fragment, their definition locations, usage relationships, and transitive dependencies. Neither weaving nor tangling is performed. See Section 2.6 for details.

`-e` *name* Extract a single fragment by name and emit its code and dependency information as Markdown to stdout. The match uses the same prefix logic as `@<name...@>`. Neither weaving nor tangling is performed. See Section 2.6 for details.

1.5 Markdown Output

When invoked with `-w md`, weft generates a Markdown file (`.md`) instead of a LaTeX file (`.tex`). This allows literate programs to be read on any platform that renders Markdown, including GitHub, GitLab, and local editors like VS Code.

The Markdown output preserves all the essential features of the LaTeX output—scrap definitions, cross-references, and indices—but uses a different set of formatting conventions:

Scrap headers File definitions appear as `**"filename"** **N** =` and fragment definitions as `**<*Name* N>** =`, where N is the sequential scrap number.

Scrap bodies Code is wrapped in `<pre><code>` HTML blocks (not fenced code blocks). This allows fragment references within scraps to be rendered as clickable HTML links with italic and bold formatting, which would not be possible inside Markdown fenced blocks.

HTML escaping Characters `<`, `>`, and `&` inside scraps are escaped as `<`, `>`, and `&` respectively.

Cross-references Each scrap has an HTML anchor (`<a id="weftN">`) and cross-reference links use `<a href="#weftN">`. Footer notes ("defined by", "referenced in") appear as Markdown blockquotes.

Indices The `@f`, `@m`, and `@u` commands produce Markdown lists with links. In the user identifier index, defining scraps appear in bold.

Sequential numbering The `-n` flag is forced automatically, since Markdown has no page numbers. Scraps are numbered 1, 2, 3, ... without letter suffixes.

Prose Text outside scraps passes through unmodified. The author writes Markdown prose directly in the `.weft` source file; weft does not interpret or transform it.

Syntax highlighting When the language of a scrap can be determined—from a preceding `@l` tag, the `@L` command, or the file extension of an `@o` name—weft emits `<code class="language-X">` on the opening tag. This allows client-side highlighters (highlight.js, Prism) and platforms like GitHub to apply syntax colouring automatically.

Tangling (output file generation) works identically regardless of the weave format. When `-w` is given, a single invocation produces both the documentation and the tangled output files in one pass.

1.6 Section Markers and Language Auto-Detection

When debugging tangled code, it is useful to correlate lines in the generated file back to the original web source. weft can auto-detect the programming language and generate *section markers*—comments that delimit the opening and closing of each scrap in the tangled output, including the source file name and line number. For C and C++, it also generates `#line` directives automatically.

The key idea is simple:

file extension ⟶ language ⟶ comment style ⟶ section markers in tangled output

For example, `@o server.js` has the extension `.js`. weft looks up `js` in its language table, finds that JavaScript uses `//` comments, and wraps each scrap in the tangled `server.js` with `//` markers:

```
// {1: web.weft:12}
console.log("hello");
// {:1}
```

Similarly, `@o main.c` matches `c`, which uses `/* */` comments and also enables `#line` directives:

```
#line 5 "core.weft"
/* {1: core.weft:5} */
#include <stdio.h>
/* {:1} */
```

And `@o config.py` matches `py`, using `#` comments:

```
# {3: settings.weft:40}
DATABASE_URL = "postgres://localhost/app"
# {:3}
```

This happens automatically—no flags are needed. The extension alone is enough.

1.6.1 Overriding the Language with `@l`

Sometimes the file extension is not enough, or you want a different behaviour. The `@l` command (lowercase) placed on the line before an `@o` or `@d` directive overrides the extension:

```
@l sql
@o queries.txt
@{SELECT * FROM users;
@}
```

Here the file is `queries.txt`, whose extension `.txt` has no comment style. But `@l sql` tells weft to use `--` comments, so the tangled output gets:

```
-- {1: db.weft:5}
SELECT * FROM users;
-- {:1}
```

`@l` also works on `@d` (fragment) definitions. This is useful for fragments that will be included in files of a specific language:

```
@l python
@d Validate email
@{import re
def validate(email):
    return bool(re.match(r'^[^@]+@[^@]+$', email))
@}
```

The language tag is stored on the fragment and used both for section markers in the tangled output and for syntax highlighting in the Markdown weave output.

`@l` is format-agnostic: it works identically in LaTeX and Markdown source files. Unlike the earlier approach of encoding the language in Markdown fences (````python`) or LaTeX markers (`%%%python`), `@l` uses the same at-sign syntax as every other weft command.

Note that cosmetic fences (```) may still be used around `@o`/`@d` definitions for readability in Markdown previewers—they are silently stripped during Markdown weaving (see Section 1.2.1). But fences no longer set the language; use `@l` for that.

Priority rules. When multiple sources of language information are available, weft resolves them in this order:

1. **Explicit per-file flags** (`-d`, `-s`, `-c`*x*, etc.) on the `@o` line always win for tangle behaviour.
2. `@l` **tag** immediately before the directive—overrides the extension.
3. **File extension** of the `@o` name—the default when no `@l` is present.

1.6.2 The `@L` Command: Defining New Languages

The built-in table covers over 250 entries, but it cannot anticipate every language or every project's preferences. The `@L` command defines new language mappings—or overrides existing ones—directly in the document preamble.

`@L` *name style* [`+d`]

- *name* is the tag that will be recognised in `@l` commands and file extensions (`@o file.name`).
- *style* is the comment syntax: `//`, `#`, `--`, `/*`, or `<!--`.
- `+d` (optional) enables `#line` directive generation.

Examples:

```
@L wgsl //          % WebGPU shading language: // comments
@L cedar //         % Cedar policy language
@L c //             % override built-in: use // instead of /* */
```

How @L interacts with the rest of the system. User-defined entries are searched *before* the built-in table, so `@L c //` effectively changes the comment style for all subsequent `@o main.c` directives (and for any `@l c` tag) from `/* */` to `//`. Up to 32 user-defined languages may be registered.

`@L` should appear in the document preamble, before any `@o` or `@d` that depends on it. It is processed during pass 1 and silently skipped during weaving.

1.6.3 The Language Table

weft includes a built-in table of over 250 entries (languages, file extensions, and common aliases), organised by comment style. Each entry maps a tag to a comment style and optionally enables `#line` directives. The tag is matched against `@l` tags and file extensions alike—there is no distinction between them.

Comment style	Tags (representative)
`/* */` + `#line`	c, h, cpp, cc, cxx, hpp, hxx, objc, objcpp, m, mm, ino, pde, cu, cuh, cilk
`/* */`	css, scss, sass, less, stylus, cl (OpenCL)
`//`	js, ts, jsx, tsx, mjs, cjs, mts, svelte, vue, java, scala, kotlin, kt, groovy, gradle, clj, cs, fs, fsx, vb, go, rust, rs, zig, swift, dart, odin, mojo, carbon, php, graphql, prisma, glsl, hlsl, wgsl, metal, vert, frag, proto, thrift, fbs, capnp, sol, reason, res, ml, sml, fut, apex, bicep, jsonnet, qml, p4
`#`	python, py, pyx, pyi, sh, bash, zsh, fish, ksh, nu, perl, pl, ruby, rb, r, tcl, awk, sed, coffee, makefile, mk, cmake, just, meson, bazel, bzl, julia, jl, nim, crystal, cr, elixir, ex, yaml, yml, toml, dockerfile, terraform, tf, hcl, nix, conf, ini, env, editorconfig, gitignore, powershell, ps1, xdc, sdc
`--`	lua, luau, sql, psql, plsql, plpgsql, mysql, sqlite, hive, haskell, hs, elm, purescript, purs, ada, adb, ads, vhdl, vhd, lean, lean4, agda, idris, idr, erlang
`<!-- -->`	html, htm, xml, svg, xhtml, xsl, xsd, dtd, rss, xaml, csproj, jsp, aspx, cshtml, erb, ejs, hbs, njk, twig, blade, pug, liquid, mustache, md, mdx, rst, tex, latex
(none)	json, csv, tsv, txt, diff, patch

For the full list, see the `lang_table` array in Section 3.3.1. If a tag is not found in either the user-defined (`@L`) or built-in table, weft defaults to `//` comments with no `#line` directives. Tags mapped to style 0 (e.g. `json`, `txt`) produce no markers at all.

1.6.4 Syntax Highlighting in Markdown Output

When weaving to Markdown (`-w md`), weft emits the language tag on the `<code>` element:

```
<pre><code class="language-python">def hello():
    print("hello")
</code></pre>
```

This allows client-side highlighters (highlight.js, Prism) and platforms like GitHub to apply syntax colouring automatically. The language tag comes from the same chain described above: `@l` tag or file extension.

For `@d` fragments, the tag comes from a preceding `@l` command (there is no file extension to infer from). If no language is known, weft emits a plain `<pre><code>` without a class.

1.6.5 Output Format of Section Markers

Each scrap in the tangled file is wrapped with an opening marker at the top and a closing marker at the bottom, using the comment syntax of the detected language. The opening marker includes the scrap number and the source location (file name and line number). When fragments are nested, each expansion gets its own pair of markers, producing a clear hierarchy in the output.

1.6.6 Debugging Tangled Code

When you work with literate programming, the code you execute is *tangled output*—generated files assembled from scraps scattered across `.w` files. An error at `server.js:47` does not correspond to line 47 of any file you wrote; it is line 47 of a file that weft assembled from fragments defined in `server.w`, `routes.w`, and

`database.w`. Without markers, debugging means mentally reconstructing which scrap produced which line. With markers, the tangled file tells you directly.

Reading the markers. Each marker encodes three pieces of information:

Part	Meaning
`{` or `{:`	Opening or closing delimiter
Number	Scrap ID—unique across the entire document
`file:line`	Source `.w` file and line number (opening only)

Examples by language. **JavaScript** (`//` comments):

```
// {3: routes.w:42}
app.get("/api/users", async (req, res) => {
  // {7: database.w:18}
  const users = await db.query("SELECT * FROM users");
  // {:7}
  res.json(users);
});
// {:3}
```

Python (`#` comments):

```
# {5: models.w:31}
class User:
    def __init__(self, name, email):
        self.name = name
        self.email = email
# {:5}
```

C (`/* */` comments with `#line` directives):

```
#line 12 "memory.w"
/* {2: memory.w:12} */
void *safe_malloc(size_t size) {
    void *ptr = malloc(size);
    if (!ptr) {
        /* {4: errors.w:8} */
        fprintf(stderr, "out of memory\n");
        exit(1);
        /* {:4} */
    }
    return ptr;
}
/* {:2} */
```

Lua (`--` comments):

```
-- {9: config.w:55}
local defaults = {
  port = 8080,
  host = "localhost",
}
-- {:9}
```

HTML (`<!-- -->` comments):

```
<!-- {6: layout.w:20} -->
<nav class="sidebar">
  <!-- {11: navigation.w:7} -->
  <ul>
    <li><a href="/">Home</a></li>
  </ul>
  <!-- {:11} -->
</nav>
<!-- {:6} -->
```

Debugging workflow. The following four-step procedure traces any error in tangled output back to the literate source.

Step 1: Locate the error in tangled output. You receive an error such as:

```
TypeError: Cannot read property 'id' of undefined
    at Object.<anonymous> (server.js:47)
```

Open `server.js` and go to line 47.

Step 2: Read the nearest opening marker. Scan *upward* from the error line until you find an opening marker:

```
// {3: routes.w:42}
```

This tells you: scrap 3 generated this code, defined in `routes.w` starting at line 42.

Step 3: Go to the literate source. Open `routes.w` at line 42. You will find the scrap with the bug, surrounded by prose that explains its intent:

```
We fetch the user by ID from the database. If the user
doesn't exist, we should return a 404, but currently we
assume the user always exists --- this is the bug.

@d Handle user fetch
@{const user = await getUser(req.params.id);
res.json(user.id);
@}
```

The prose makes the bug obvious: there is no null check. Fix it in the `.w` file, re-tangle, and the error is gone.

Step 4: Nested markers = nested fragments. When fragments invoke other fragments, markers nest:

```
// {3: routes.w:42}
app.get("/api/users/:id", async (req, res) => {
  // {7: database.w:18}
  const user = await db.query(
    "SELECT * FROM users WHERE id=$1",
    [req.params.id]);
  // {:7}
  if (!user) return res.status(404).json({error: "not found"});
  res.json(user);
});
// {:3}
```

If the bug is inside the `{7:...{:7}` block, go to `database.w:18`, not `routes.w:42`. The *innermost enclosing markers* point to the most specific source location. Always look at the tightest pair around the error line.

Interpreting nested hierarchies. Complex tangled output can have several nesting levels. Read them like parentheses—each opening matches its corresponding close:

```
/* {1: main.w:10} */          outermost: main file structure
#include <stdio.h>
/* {2: io.w:25} */            middle: I/O subsystem
void print_report() {
    /* {5: format.w:8} */     innermost: formatting detail
    printf("Report v%d\n", VERSION);
    /* {:5} */
}
/* {:2} */
int main() {
    /* {3: init.w:14} */
    init_subsystems();
    /* {:3} */
    print_report();
    return 0;
}
/* {:1} */
```

Scrap numbers are *document-global*—scrap 1 is the first `@{...@}` in the entire woven document, scrap 5 is the fifth. They are stable as long as you do not reorder scraps.

C/C++: `#line` directives. For C-family languages, weft also generates `#line` directives *above* the opening marker. These make the C compiler itself report errors against the `.w` source file:

```
#line 12 "memory.w"
/* {2: memory.w:12} */
void *safe_malloc(size_t size) { ... }
/* {:2} */
```

Now `gcc` will say:

```
memory.w:14: error: implicit declaration of function 'malloc'
```

instead of:

```
memory.c:47: error: implicit declaration of function 'malloc'
```

The `#line` directive and the section marker are complementary: `#line` makes the compiler report errors against the `.w` file directly; the `{N: file:line}` markers are human-readable when browsing the tangled output.

Common debugging scenarios. Runtime error with stack trace.

```
Error: ENOENT: no such file or directory, open '/data/config.json'
    at readConfig (server.js:123)
```

Open `server.js:123`, find the nearest marker `// {12: config.w:35}`, open `config.w:35`—the scrap that reads the config—and fix the path or add an existence check in the `.w` file.

Compiler error in C. With `#line` directives active, the compiler already reports:

```
parser.w:84: error: 'token_type' undeclared
```

Go directly to `parser.w:84`. The prose around the scrap explains what `token_type` should be and where it is defined.

Wrong output, no error. The program runs but produces wrong results. Open the tangled file and search for the relevant code. The markers tell you which `.w` file and scrap to examine:

```
# {15: transform.w:67}
result = [x * 2 for x in data]    # should be x * 3?
# {:15}
```

Go to `transform.w:67`. The prose says "triple each value" but the code says `* 2`. Bug found.

Identifying which fragment owns a line. Sometimes you know the line of code but not which `@d` definition produced it (multiple fragments with the same name accumulate). The scrap number is unique:

```
// {14: handlers.w:89}
validateInput(data);
// {:14}
// {22: handlers.w:112}
validatePermissions(user);
// {:22}
```

Scraps 14 and 22 are two different definitions (or continuations) of the same fragment. The line numbers point to each one independently.

Tips.

- *Never edit tangled files.* The markers remind you these are generated. Always trace back to the `.w` source and edit there.
- *Scrap numbers are stable* as long as you do not add or remove `@{...@}` blocks before them. If numbering shifts after edits, re-tangle and the markers update automatically.
- *Use* `grep` *on markers* to find all scraps from a specific file:
  `grep -n '{[0-9]*: database.w:' server.js`
- *Closing markers as verification.* If you see `{:7}` but no `{7:` above it in your viewport, scroll up—the scrap is larger than your screen. If you see `{7:` but no `{:7}`, the scrap extends below.
- *Markers in test output.* If your test framework shows code snippets on failure, the markers point straight to the literate source.
- *Disable markers selectively.* Use explicit flags on `@o` (e.g. `@o file -t`) with no comment flags to suppress markers for a specific output file when comments would interfere—for example, binary formats, data files, or files where comments are not valid syntax.

1.7 Restrictions

Because weft is intended to be a simple tool, a few restrictions have been established. Over time, some of these may be eliminated; others seem fundamental.

- The handling of errors is not completely ideal. In some cases, weft simply warns of a problem and continues; in other cases it halts immediately. This behavior should be regularized.
- weft warns about references to fragments that haven't been defined, but doesn't halt. The name of the fragment is included in the output file surrounded by `<>` signs. This seems most convenient for development, but may change in the future.
- File names and index entries should not contain any `@` signs.
- Fragment names may be (almost) any well-formed TEX string. It makes sense to change fonts or use math mode; however, care should be taken to ensure matching braces, brackets, and dollar signs.
- Anything is allowed in the body of a scrap; however, very long scraps (horizontally or vertically) may not typeset well.

- Temporary files (created for comparison to the eventual output files) are placed in the current directory. Since they may be renamed to an output file name, all the output files should be on the same file system as the current directory. Alternatively, you can use the `-p` flag to specify where the files go.
- Because page numbers cannot be determined until the document has been typeset, weft must be rerun after LaTeX to obtain a clean version of the document (very similar to the way it is sometimes necessary to rerun LaTeX to obtain an up-to-date table of contents after significant edits). weft will warn (in most cases) when this needs to be done; in the remaining cases, LaTeX will warn that labels may have changed. This restriction does not apply to Markdown output (`-w md`), which uses sequential numbering and requires no additional processing passes.

Very long scraps may be allowed to break across a page if declared with `@O` or `@D` (instead of `@o` and `@d`). This doesn't work very well as a default, since far too many short scraps will be broken across pages; however, as a user-controlled option, it seems very useful.

1.8 Project Introspection

The `-m` and `-e` flags support machine-readable introspection of a weft project. They run after pass 1 has built the name trees and do *not* invoke the weave or tangle phases.

1.8.1 Map (`-m`)

The command

```
weft -m project.weft
```

writes a JSON object to stdout with two arrays, `"files"` and `"fragments"`. For each entry the map records the name, language tag, definition scraps (with source file and line), and dependency relationships. File entries list the fragments they use; fragment entries list the scraps that invoke them and the fragments they reference in turn.

This gives an AI assistant or any external tool a complete dependency graph of the project in a single invocation, without having to parse the weft source files.

1.8.2 Extract (`-e`)

The command

```
weft -e "fragment name" project.weft
```

finds the named fragment (using prefix matching), then emits its scrap code and dependency information as Markdown to stdout. Referenced fragments are followed transitively, so the output contains the full closure of the requested fragment.

This allows an AI assistant to retrieve exactly the code it needs to understand or modify a specific fragment, together with all its dependencies, in one step.

1.9 Acknowledgements

Several people have contributed their time, ideas, and debugging skills. In particular, the contributions of the following people are gratefully acknowledged: Osman Buyukisik, Manuel Carriba, Adrian Clarke, Tim Harvey, Michael Lewis, Walter Ravenek, Rob Shillingsburg, Kayvan Sylvan, Dominique de Waleffe, and Scott Warren. Of course, most of these people would never have heard of nuweb (the original tool) without the efforts of George Greenwade.

Since maintenance has been taken over by Marc Mengel, Simon Wright and Keith Harwood online contributions have been made by:

- Walter Brown <wb@fnal.gov>
- Nicky van Foreest <n.d.vanforeest@math.utwente.nl>
- Javier Goizueta <jgoizueta@jazzfree.com>
- Alan Karp <karp@hp.com>

Chapter 2

The Overall Structure

Processing a web requires three major steps:

1. Read the source, accumulating file names, fragment names, scraps, and lists of cross-references.
2. Reread the source, copying out to the documentation file, with protection and cross-reference information for all the scraps.
3. Traverse the list of files names. For each file name:
   (a) Dump all the defining scraps into a temporary file.
   (b) If the file already exists and is unchanged, delete the temporary file; otherwise, rename the temporary file.

2.1 Files

I have divided the program into several files for quicker recompilation during development.

"global.h" 20≡
⟨ *Include files* 21 ⟩
⟨ *Type declarations* 22a, . . . ⟩
⟨ *Limits* 22b ⟩
⟨ *Global variable declarations* 27, . . . ⟩
⟨ *Function prototypes* 38a, . . . ⟩
⟨ *Operating System Dependencies* 25b ⟩◇

We'll need at least five of the standard system include files.

⟨ *Include files* 21 ⟩ ≡

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <signal.h>
#include <locale.h>
◇
```

Fragment referenced in 20.

Defines: `exit` 25a, 33, 37, 43c, 44a, 66b, 113b, 115b, 120, 121, 122a, 123a, 129, 130, 132, 138b, 140, 145b, 163, 167, 168a, 169bc, 171, 172, 177a, 184b, `fclose` 55b, 84b, 115ab, 116b, 122b, 149a, `FILE` 55ab, 61a, 65b, 67c, 68, 75, 76b, 78b, 82a, 84ab, 89a, 92, 93a, 98, 99, 100b, 102a, 104b, 106a, 107b, 109, 110, 112, 115a, 116b, 118ab, 124b, 125ab, 126b, 135, 139a, 147b, 149a, 186ab, 187cd, 190ab, 195, `fopen` 55b, 84b, 115b, 116b, 121, 123a, 149a, `fprintf` 30, 32c, 33, 37, 38b, 41, 43c, 44a, 48a, 54a, 55b, 56b, 59, 61b, 64c, 65a, 66b, 72ab, 73, 75, 76c, 82b, 84b, 88a, 89b, 90c, 91b, 96bc, 97, 99, 100c, 104c, 106c, 108, 109, 110, 112, 113b, 115ab, 116c, 120, 121, 122a, 123a, 125a, 126e, 129, 130, 132, 135, 138b, 140, 142bcd, 145ab, 147ab, 154b, 161, 162, 163, 164ab, 167, 168a, 169bc, 171, 172, 177a, 184b, 187a, 188b, 190cd, 191ab, `fputs` 48a, 49a, 50d, 54a, 56a, 58a, 59, 60a, 61ab, 62abcd, 63a, 64abcd, 65ab, 67c, 68, 70a, 71a, 72ab, 73, 74, 75, 76ac, 77ab, 79b, 80ab, 81c, 82b, 87a, 88ab, 89ab, 90c, 91ab, 92, 93ab, 95a, 96ab, 97, 98, 99, 100acd, 101a, 102b, 103ab, 104ac, 106bc, 107ac, 108, 109, 110, 111a, 112, 125a, 141, 145a, 146a, 186c, 187ab, 188ab, `getc` 116b, 118c, 120, 121, 122ab, 123a, `getenv` 32c, `isgraph` 122a, 163, 171, `islower` 156a, `isspace` 63a, 89c, 132, 163, 164a, 167, 169c, `isupper` Never used, `malloc` 78b, 102a, 177a, 194a, `putc` 49ab, 56c, 57a, 68, 69c, 73, 75, 76c, 77a, 79b, 80ab, 82b, 85ab, 86a, 93ab, 94, 95a, 97, 99, 100c, 102b, 103a, 125a, 141, 143bd, 144a, 145b, 146abc, `remove` 116ab, `setlocale` 32c, `size_t` 192c, 193b, 194a, `stderr` 30, 32c, 33, 37, 38b, 41, 43c, 44a, 55b, 64c, 65a, 66b, 73, 75, 84b, 90c, 91b, 97, 99, 113b, 115ab, 116c, 120, 121, 122a, 123a, 126e, 129, 130, 132, 138b, 140, 145b, 154b, 161, 162, 163, 164ab, 167, 168a, 169bc, 171, 172, 177a, 184b, 190d, 191ab, `strlen` 49a, 149b, 153ab, 179a, 189b, `toupper` 156a.

I like to use TRUE and FALSE in my code. I'd use an enum here, except that some systems seem to provide definitions of TRUE and FALSE by default. The following code seems to work on all the local systems.

⟨ *Type declarations* 22a ⟩ ≡

```
#ifndef FALSE
#define FALSE 0
#endif
#ifndef TRUE
#define TRUE 1
#endif
◇
```

Fragment defined by 22a, 50c, 114c, 151ab, 168c, 170b, 173d, 185c, 192a.
Fragment referenced in 20.
Defines: FALSE 27, 28a, 30, 31abc, 32ab, 37, 56c, 57a, 58b, 62c, 63d, 65b, 75, 80a, 81b, 82b, 85ab, 86a, 87b, 90a, 92, 99, 103a, 104c, 106c, 108, 113a, 120, 123a, 145b, 149a, 155a, 158, 159, 161, 164a, 165a, 185a, 187a, 188b, 189ab, TRUE 27, 28a, 30, 32a, 37, 54a, 57a, 58b, 62ad, 65b, 75, 80a, 81b, 82b, 85ab, 86a, 87b, 89bc, 92, 99, 103a, 104c, 106c, 108, 112, 120, 125b, 126e, 145b, 155a, 158, 159, 161, 164a, 185a, 187a, 188b, 189ab.

Here we define the maximum length for names (of fragments etc).

⟨ *Limits* 22b ⟩ ≡

```
#ifndef MAX_NAME_LEN
#define MAX_NAME_LEN 1024
#endif
◇
```

Fragment referenced in 20.
Defines: MAX_NAME_LEN 132, 135, 138b, 163, 167, 171, 173a, 184b, 189c.

2.1.1 The Main Files

The code is divided into four main files (introduced here) and five support files (introduced in the next section). The file main.c will contain the driver for the whole program (see Section 2.2).

"main.c" 22c≡

```
#include "global.h"
◇
```

File defined by 22c, 25a.

The first pass over the source file is contained in pass1.c. It handles collection of all the file names, fragments names, and scraps (see Section 2.3).

"pass1.c" 22d≡

```
#include "global.h"
◇
```

File defined by 22d, 38b.

The .tex file is created during a second pass over the source file. The file latex.c contains the code controlling the construction of the .tex file (see Section 2.4).

"latex.c" 22e≡

```
#include "global.h"
static int scraps = 1;
◇
```

File defined by 22e, 55ab, 61a, 65b, 66ab, 67bc, 68, 69a, 75, 76b, 78ab, 81b, 82a.
Uses: scraps 124a.

The Markdown output is created by the code in `markdown.c` (see Section 2.5).

"`markdown.c`" 23a≡

```
#include "global.h"
static int scraps = 1;
◇
```

File defined by 23a, 84ab, 89a, 92, 93a, 98, 99, 100b, 101b, 102a, 104b, 106a, 107b.
Uses: `scraps` 124a.

The code controlling the creation of the output files is in `output.c` (see Section 2.7).

"`output.c`" 23b≡

```
#include "global.h"
◇
```

File defined by 23b, 109, 110, 111ab, 112, 113ab, 114b.

2.1.2 Support Files

The support files contain a variety of support routines used to define and manipulate the major data abstractions. The file `input.c` holds all the routines used for referring to source files (see Section 3.1).

"`input.c`" 24a≡

```
#include "global.h"
◇
```

File defined by 24a, 118abc, 119b, 123a.

Creation and lookup of scraps is handled by routines in `scraps.c` (see Section 3.2).

"`scraps.c`" 24b≡

```
#include "global.h"
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.

The handling of file names and fragment names is detailed in `names.c` (see Section 3.3).

"`names.c`" 24c≡

```
#include "global.h"
◇
```

File defined by 24c, 153ab, 154a, 155a, 156a, 157, 159, 161, 162, 163, 167, 170ac, 171, 174cd.

Memory allocation and deallocation is handled by routines in `arena.c` (see Section 3.6).

"`arena.c`" 24d≡

```
#include "global.h"
◇
```

File defined by 24d, 192d, 193ab, 194b.

Finally, for best portability, I seem to need a file containing (useless!) definitions of all the global variables.

"`global.c`" 24e≡

```
#include "global.h"
⟨ Operating System Dependencies 25b ⟩
⟨ Global variable definitions 28a, ... ⟩
◇
```

File defined by 24e, 191d.

2.2 The Main Routine

The main routine is quite simple in structure. It wades through the optional command-line arguments, then handles any files listed on the command line.

"main.c" 25a≡

```
#include <stdlib.h>
int main(int argc, char **argv)
{
  int arg = 1;
  ⟨ Interpret command-line arguments 29c, ... ⟩
  ⟨ Set locale information 32c ⟩
  initialise_delimit_scrap_array();
  ⟨ Process the remaining arguments (file names) 33 ⟩
  exit(0);
}
◇
```

File defined by 22c, 25a.
Defines: `main` 195.
Uses: `exit` 21.

We only have two major operating system dependencies; the separators for file names, and how to set environment variables. For now we assume the latter can be accomplished via `putenv()` in `stdlib.h`.

⟨ *Operating System Dependencies* 25b ⟩ ≡

```
#if defined(VMS)
#define PATH_SEP(c) (c==']'||c==':')
#define PATH_SEP_CHAR ""
#define DEFAULT_PATH ""
#elif defined(MSDOS)
#define PATH_SEP(c) (c=='\\')
#define PATH_SEP_CHAR "\\"
#define DEFAULT_PATH "."
#else
#define PATH_SEP(c) (c=='/')
#define PATH_SEP_CHAR "/"
#define DEFAULT_PATH "."
#endif
◇
```

Fragment referenced in 20, 24e.

2.2.1 Command-Line Arguments

There are numerous possible command-line arguments:

`-t` Suppresses generation of the `.tex` file.

`-o` Suppresses generation of the output files.

`-d` list dangling identifier references in indexes.

`-c` Forces output files to overwrite old files of the same name without comparing for equality first.

`-v` The verbose flag. Forces output of progress reports.

`-n` Forces sequential numbering of scraps (instead of page numbers).

`-s` Doesn't print list of scraps making up file at end of each scrap.

`-x` Include cross-reference numbers in scrap comments.

`-p` *path* Prepend *path* to the filenames for all the output files.

`-h` *options* Provide options for the hyperref package.

`-r` Turn on hyperlinks. You must include the —usepackage— options in the text.

`-l` Use the `listings` package for formatting scraps.

There are two ways to get hyper-links into your documentation. With one, the `-r` flag simply arranges for fragment cross-references to be hyperlinks. You have to provide all the rest of the `hyperref` material yourself. The `-h` flag does more work, but requires you provide the `hyperref` options on the command line. You would use the first if you know you will use the same tools all the time. You would use the second if you have to distribute the weft file and use the build system to provide the `hyperref` options.

The `-h` option is used as follows. If you want hyperlinks in your document give a string to this option which is a valid option for the `hyperref` package. For example, `"dvips,colorlinks=true"` sets it to use the `dvips` driver and mark the links in colour. For more information about these options, see the `hyperref` documentation.

Then in your document you put the command `\NWuseHyperlinks` immediately after your `usepackage` commands. If you do both of these you will get hyperlinks. If you miss out either then nothing interesting happens.
Global flags are declared for each of the arguments.

⟨ *Global variable declarations* 27 ⟩ ≡

```
extern int tex_flag;      /* if FALSE, don't emit the documentation file */
extern int output_flag;   /* if FALSE, don't emit the output files */
extern int compare_flag;  /* if FALSE, overwrite without comparison */
extern int verbose_flag;  /* if TRUE, write progress information */
extern int number_flag;   /* if TRUE, use a sequential numbering scheme */
extern int scrap_flag;    /* if FALSE, don't print list of scraps */
extern int dangling_flag;    /* if FALSE, don't print dangling identifiers */
extern int xref_flag; /* If TRUE, print cross-references in scrap comments */
extern int prepend_flag;  /* If TRUE, prepend a path to the output file names */
extern char * dirpath;    /* The prepended directory path */
extern char * path_sep;   /* How to join path to filename */
extern int listings_flag;   /* if TRUE, use listings package for scrap formatting */
extern int version_info_flag; /* If TRUE, set up version string */
extern char *  version_string; /* What to print for @v */
extern int hyperref_flag; /* Are we preparing for hyperref
                             package. */
extern int hyperopt_flag; /* Are we preparing for hyperref options */
extern char * hyperoptions; /* The options to pass to the
                               hyperref package */
extern int includepath_flag; /* Do we have an include path? */
extern struct incl{char * name; struct incl * next;} * include_list;
                       /* The list of include paths */
extern int markdown_flag; /* if TRUE, generate Markdown instead of LaTeX */
extern int weave_flag;          /* if TRUE, weave was requested via -w */
extern int weave_format;        /* 0=none, 1=tex, 2=md */
extern int weave_format_from_cli; /* if TRUE, CLI override of @W */
extern int map_flag;       /* if TRUE, emit JSON map to stdout */
extern int extract_flag;   /* if TRUE, next arg is extract name */
extern char *extract_name; /* fragment name to extract */
◇
```

Fragment defined by 27, 28b, 29a, 42c, 47c, 63b, 117b, 127a, 152a, 192b.
Fragment referenced in 20.
Defines: `compare_flag` 28a, 30, 116a, `dangling_flag` 28a, 30, 82b, 104c, `extract_flag` 28a, 30, 32b, `extract_name` 28a, 32b, 37, `hyperoptions` 28a, 32a, 56ab, `hyperopt_flag` 28a, 30, 32a, `hyperref_flag` 28a, 30, 32a, 56a, `incl` 28a, 31b, 36, 121, `includepath_flag` 28a, 30, 31b, `map_flag` 28a, 30, 37, `markdown_flag` 28a, 30, 37, `number_flag` 28a, 30, 37, 149a, `output_flag` 28a, 30, 37, `scrap_flag` 28a, 30, 59, 90b, `tex_flag` 28a, 30, 37, 38b, 145b, `verbose_flag` 28a, 30, 38b, 55b, 84b, 115a, `version_info_flag` 28a, 30, 31c, `version_string` 28a, 31c, 58a, 71a, 87a, `weave_flag` 28a, 30, 37, `weave_format` 28a, 30, 37, 162, `weave_format_from_cli` 28a, 30, 37, 162, `xref_flag` 28a, 30, 146ac.
Uses: `FALSE` 22a, `next` 123b, `scraps` 124a, `TRUE` 22a.

The flags are all initialized for correct default behavior.

⟨ *Global variable definitions* 28a ⟩ ≡

```
int tex_flag = TRUE;
int output_flag = TRUE;
int compare_flag = TRUE;
int verbose_flag = FALSE;
int number_flag = FALSE;
int scrap_flag = TRUE;
int dangling_flag = FALSE;
int xref_flag = FALSE;
int prepend_flag = FALSE;
char * dirpath = DEFAULT_PATH; /* Default directory path */
char * path_sep = PATH_SEP_CHAR;
int listings_flag = FALSE;
int version_info_flag = FALSE;
char default_version_string[] = "no version";
char *  version_string = default_version_string;
int hyperref_flag = FALSE;
int hyperopt_flag = FALSE;
char * hyperoptions = "";
int includepath_flag = FALSE; /* Do we have an include path? */
struct incl * include_list = NULL;
                        /* The list of include paths */
int markdown_flag = FALSE;
int weave_flag = FALSE;
int weave_format = 0;
int weave_format_from_cli = FALSE;
int map_flag = FALSE;
int extract_flag = FALSE;
char *extract_name = NULL;
◇
```

Fragment defined by 28ac, 29b, 42d, 47d, 63c, 117c, 127b, 152b.
Fragment referenced in 24e.
Uses: `compare_flag` 27, `dangling_flag` 27, `extract_flag` 27, `extract_name` 27, `FALSE` 22a, `hyperoptions` 27, `hyperopt_flag` 27, `hyperref_flag` 27, `incl` 27, `includepath_flag` 27, `map_flag` 27, `markdown_flag` 27, `number_flag` 27, `output_flag` 27, `scrap_flag` 27, `tex_flag` 27, `TRUE` 22a, `verbose_flag` 27, `version_info_flag` 27, `version_string` 27, `weave_flag` 27, `weave_format` 27, `weave_format_from_cli` 27, `xref_flag` 27.

A global variable `nw_char` will be used for the weft meta-character, which by default will be @.

⟨ *Global variable declarations* 28b ⟩ ≡

```
extern int nw_char;
◇
```

Fragment defined by 27, 28b, 29a, 42c, 47c, 63b, 117b, 127a, 152a, 192b.
Fragment referenced in 20.
Defines: `nw_char` 28c, 37, 38c, 41, 43c, 44a, 45b, 47b, 48b, 49d, 52, 53, 54a, 56c, 57a, 59, 66a, 67c, 68, 69a, 70a, 71d, 72a, 76c, 82b, 85a, 86a, 93a, 95a, 96b, 118c, 120, 123a, 129, 130, 131a, 132, 133cd, 135, 138b, 141, 143c, 144a, 145ab, 147ab, 148a, 154b, 161, 162, 163, 167, 168ab, 169c, 171, 172, 173a, 184ab, 187a, 188b, 189ac.

⟨ *Global variable definitions* 28c ⟩ ≡

```
int nw_char='@';
◇
```

Fragment defined by 28ac, 29b, 42d, 47d, 63c, 117c, 127b, 152b.
Fragment referenced in 24e.
Defines: `nw_char` 28b, 37, 38c, 41, 43c, 44a, 45b, 47b, 48b, 49d, 52, 53, 54a, 56c, 57a, 59, 66a, 67c, 68, 69a, 70a, 71d, 72a, 76c, 82b, 85a, 86a, 93a, 95a, 96b, 118c, 120, 123a, 129, 130, 131a, 132, 133cd, 135, 138b, 141, 143c, 144a, 145ab, 147ab, 148a, 154b, 161, 162, 163, 167, 168ab, 169c, 171, 172, 173a, 184ab, 187a, 188b, 189ac.

We save the invocation name of the command in a global variable `command_name` for use in error messages.

⟨ *Global variable declarations* 29a ⟩ ≡

```
extern char *command_name;
◇
```

Fragment defined by 27, 28b, 29a, 42c, 47c, 63b, 117b, 127a, 152a, 192b.
Fragment referenced in 20.
Defines: `command_name` 29bc, 30, 33, 37, 41, 43c, 44a, 55b, 65a, 73, 75, 84b, 91b, 97, 99, 113b, 115b, 116c, 120, 121, 122a, 123a, 126e, 129, 130, 132, 138b, 140, 145b, 154b, 161, 162, 163, 164ab, 167, 168a, 169bc, 171, 172, 184b.

⟨ *Global variable definitions* 29b ⟩ ≡

```
char *command_name = NULL;
◇
```

Fragment defined by 28ac, 29b, 42d, 47d, 63c, 117c, 127b, 152b.
Fragment referenced in 24e.
Uses: `command_name` 29a.

The invocation name is conventionally passed in `argv[0]`.

⟨ *Interpret command-line arguments* 29c ⟩ ≡

```
command_name = argv[0];
◇
```

Fragment defined by 29cd.
Fragment referenced in 25a.
Uses: `command_name` 29a.

We need to examine the remaining entries in `argv`, looking for command-line arguments.

⟨ *Interpret command-line arguments* 29d ⟩ ≡

```
while (arg < argc) {
  char *s = argv[arg];
  if (*s++ == '-') {
    ⟨ Interpret the argument string s 30 ⟩
    arg++;
    ⟨ Perhaps get the prepend path 31a ⟩
    ⟨ Perhaps get the version info string 31c ⟩
    ⟨ Perhaps get the hyperref options 32a ⟩
    ⟨ Perhaps add an include path 31b ⟩
    ⟨ Perhaps get the extract name 32b ⟩
  }
  else break;
}◇
```

Fragment defined by 29cd.
Fragment referenced in 25a.

Several flags can be stacked behind a single minus sign; therefore, we've got to loop through the string, handling them all. If this flag requires an argument we skip to getting its argument straight away. This allows arguments to butt up to their flags and also avoids ambiguity about which value goes with which flag.

⟨ *Interpret the argument string* s 30 ⟩ ≡

```
{
  char c = *s++;
  while (c) {
    switch (c) {
      case 'c': compare_flag = FALSE;
                break;
      case 'd': dangling_flag = TRUE;
                break;
      case 'h': hyperopt_flag = TRUE;
                goto HasValue;
      case 'I': includepath_flag = TRUE;
                goto HasValue;
      case 'l': listings_flag = TRUE;
                break;
      case 'n': number_flag = TRUE;
                break;
      case 'o': output_flag = FALSE;
                break;
      case 'p': prepend_flag = TRUE;
                goto HasValue;
      case 'r': hyperref_flag = TRUE;
                break;
      case 's': scrap_flag = FALSE;
                break;
      case 't': tex_flag = FALSE;
                break;
      case 'e': extract_flag = TRUE;
                goto HasValue;
      case 'm': map_flag = TRUE;
                break;
      case 'v': verbose_flag = TRUE;
                break;
      case 'V': version_info_flag = TRUE;
                goto HasValue;
      case 'w': goto HasWeaveFormat;
      case 'x': xref_flag = TRUE;
                break;
      default:  fprintf(stderr, "%s: unexpected argument ignored.  ",
                        command_name);
                fprintf(stderr, "Usage is: %s [-cdmnostvx] "
                      "[-e name] [-I path] [-V version] "
                      "[-w [format]] [-h options] [-p path] file...\n",
                        command_name);
                break;
    }
    c = *s++;
  }
  goto HasValue;
HasWeaveFormat:
  {
    char *fmt = NULL;
    weave_flag = TRUE;
    if (*s != '\0')
      fmt = s;
    else if (arg + 1 < argc && argv[arg + 1][0] != '-') {
      /* Peek: only consume next arg if it looks like a format token */
      char *candidate = argv[arg + 1];
      if (strcmp(candidate, "md") == 0 || strcmp(candidate, "tex") == 0) {
        fmt = candidate;
        ++arg;
      }
    }
    if (fmt != NULL) {
      if (strcmp(fmt, "md") == 0) {
        weave_format = 2;
        markdown_flag = TRUE;
        weave_format_from_cli = TRUE;
      } else if (strcmp(fmt, "tex") == 0) {
        weave_format = 1;
        weave_format_from_cli = TRUE;
```

⟨ *Perhaps get the prepend path* 31a ⟩ ≡

```
if (prepend_flag)
{
  if (*s == '\0')
    s = argv[arg++];
  dirpath = s;
  prepend_flag = FALSE;
}
◇
```

Fragment referenced in 29d.
Uses: `FALSE` 22a.

⟨ *Perhaps add an include path* 31b ⟩ ≡

```
if (includepath_flag)
{
   struct incl * le
      = (struct incl *)arena_getmem(sizeof(struct incl));
   struct incl ** p = &include_list;

   if (*s == '\0')
     s = argv[arg++];
   le->name = save_string(s);
   le->next = NULL;
   while (*p != NULL)
      p = &((*p)->next);
   *p = le;
   includepath_flag = FALSE;
}
◇
```

Fragment referenced in 29d.
Uses: `arena_getmem` 193b, `FALSE` 22a, `incl` 27, `includepath_flag` 27, `next` 123b, `save_string` 153b.

⟨ *Perhaps get the version info string* 31c ⟩ ≡

```
if (version_info_flag)
{
   if (*s == '\0')
     s = argv[arg++];
   version_string = s;
   version_info_flag = FALSE;
}
◇
```

Fragment referenced in 29d.
Uses: `FALSE` 22a, `version_info_flag` 27, `version_string` 27.

⟨ *Perhaps get the hyperref options* 32a ⟩ ≡

```
if (hyperopt_flag)
{
  if (*s == '\0')
    s = argv[arg++];
  hyperoptions = s;
  hyperopt_flag = FALSE;
  hyperref_flag = TRUE;
}
◇
```

Fragment referenced in 29d.
Uses: `FALSE` 22a, `hyperoptions` 27, `hyperopt_flag` 27, `hyperref_flag` 27, `TRUE` 22a.

⟨ *Perhaps get the extract name* 32b ⟩ ≡

```
if (extract_flag) {
  extract_name = (*s != '\0') ? s : argv[arg++];
  extract_flag = FALSE;
}◇
```

Fragment referenced in 29d.
Uses: `extract_flag` 27, `extract_name` 27, `FALSE` 22a.

In order to be able to process files in foreign languages we set the locale information. `isgraph()` and friends need this (see Section 3.3). Try to read `LC_CTYPE` from the environment. Use `LC_ALL` if that fails. Don't set the locale if reading `LC_ALL` fails, too. Print a warning if setting the program's locale fails.

⟨ *Set locale information* 32c ⟩ ≡

```
{
  /* try to get locale information */
  char *s=getenv("LC_CTYPE");
  if (s==NULL) s=getenv("LC_ALL");

  /* set it */
  if (s!=NULL)
    if(setlocale(LC_CTYPE, s)==NULL)
      fprintf(stderr, "Setting locale failed\n");
}
◇
```

Fragment referenced in 25a.
Uses: `fprintf` 21, `getenv` 21, `setlocale` 21, `stderr` 21.

2.2.2 File Names

We expect at least one file name. While a missing file name might be ignored without causing any problems, we take the opportunity to report the usage convention.

⟨ *Process the remaining arguments (file names)* 33 ⟩ ≡

```
{
  if (arg >= argc) {
    fprintf(stderr, "%s: expected a file name.  ", command_name);
    fprintf(stderr, "Usage is: %s [-cdmnostvx] [-e name] [-w [format]] [-p path] file-name...\n", command_
    exit(-1);
  }
  do {
    ⟨ Handle the file name in argv[arg] 34 ⟩
    arg++;
  } while (arg < argc);
}◇
```

Fragment referenced in 25a.
Uses: `command_name` 29a, `exit` 21, `fprintf` 21, `stderr` 21.

The code to handle a particular file name is rather more tedious than the actual processing of the file. A file name may be an arbitrarily complicated path name, with an optional extension. If no extension is present, we add `.w` as a default. The extended path name will be kept in a local variable `source_name`. The resulting documentation file will be written in the current directory; its name will be kept in the variable `tex_name`.

⟨ *Handle the file name in* `argv[arg]` 34 ⟩ ≡

```
{
  char source_name[FILENAME_MAX];
  char tex_name[FILENAME_MAX];
  char aux_name[FILENAME_MAX];
  char *trim;
  char *dot;
  ⟨ Build source_name and tex_name 35 ⟩
  ⟨ Process a file 37 ⟩
}◇
```

Fragment referenced in 33.
Uses: `source_name` 117b.

I bump the pointer `p` through all the characters in `argv[arg]`, copying all the characters into `source_name` (via the pointer `q`).

At each slash, I update `trim` to point just past the slash in `source_name`. The effect is that `trim` will point at the file name without any leading directory specifications.

The pointer `dot` is made to point at the file name extension, if present. If there is no extension, we add `.weft` to the source name. The `tex_name` (weave output name) is computed later in "Process a file" once the weave format is known.

⟨ *Build* `source_name` *and* `tex_name` 35 ⟩ ≡

```
{
  char *p = argv[arg];
  char *q = source_name;
  char c;
  trim = q;
  dot = NULL;
  c = *p++;
  while (c) {
    *q++ = c;
    if (PATH_SEP(c)) {
      trim = q;
      dot = NULL;
    }
    else if (c == '.')
      dot = q - 1;
    c = *p++;
  }
  ⟨ Add the source path to the include path list 36 ⟩
  *q = '\0';
  if (!dot) {
    *q++ = '.';
    *q++ = 'w';
    *q++ = 'e';
    *q++ = 'f';
    *q++ = 't';
    *q = '\0';
  }
  /* Find trim again for tex_name computation later */
  {
    char *t = source_name;
    trim = t;
    dot = NULL;
    while (*t) {
      if (PATH_SEP(*t)) { trim = t + 1; dot = NULL; }
      else if (*t == '.') dot = t;
      t++;
    }
  }
}◇
```

Fragment referenced in 34.
Uses: `source_name` 117b.

If the source file has a directory part we add that directory to the end of the search path so that the path of last resort is the same as the source path, not, as one person put it, something irrelevant like the directory weft was invoked from.

⟨ *Add the source path to the include path list* 36 ⟩ ≡

```
if (trim != source_name) {
   struct incl * le
      = (struct incl *)arena_getmem(sizeof(struct incl));
   struct incl ** p = &include_list;
   char sv = *trim;

   *trim = '\0';
   le->name = save_string(source_name);
   le->next = NULL;
   while (*p != NULL)
      p = &((*p)->next);
   *p = le;
   *trim = sv;
}
◇
```

Fragment referenced in 35.
Uses: `arena_getmem` 193b, `incl` 27, `next` 123b, `save_string` 153b, `source_name` 117b.

Now that we're finally ready to process a file, it's not really too complex. We bundle most of the work into three routines `pass1` (see Section 2.3), `write_tex` (see Section 2.4), and `write_files` (see Section 2.7). After we're finished with a particular file, we must remember to release its storage (see Section 3.6).

The default behavior is to tangle only. Weaving happens only when `-w` is given. The weave format is determined by (in priority order): a CLI argument (`-w md` or `-w tex`), or the `@W` directive in the source file. If `-w` is given without a format and no `@W` is present, weft reports an error and exits. The `-t` flag suppresses weaving even if `-w` was given.

⟨ *Process a file* 37 ⟩ ≡

```
{
  int do_weave = FALSE;
  int effective_format = 0;

  /* Reset weave_format for this file unless CLI locked it */
  if (!weave_format_from_cli)
    weave_format = 0;

  pass1(source_name);
  current_sector = 1;
  prev_sector = 1;

  if (map_flag) {
    write_map(file_names, macro_names);
    /* skip weave and tangle */
  } else if (extract_name) {
    write_extract(extract_name, file_names, macro_names);
    /* skip weave and tangle */
  } else {
    /* Determine whether to weave and with which format */
    if (weave_flag && !tex_flag) {
      /* -t suppresses weave: tex_flag==FALSE means user passed -t */
    } else if (weave_flag) {
      effective_format = weave_format;
      if (effective_format == 0) {
        fprintf(stderr, "%s: -w requires a weave format.\n"
                "  Use -w md, -w tex, or add %cW md/%cW tex to the source file.\n",
                command_name, nw_char, nw_char);
        exit(-1);
      }
      do_weave = TRUE;
    }

    if (do_weave) {
      /* Compute weave output name now that format is known */
      const char *weave_ext = (effective_format == 2) ? ".md" : ".tex";
      if (dot) {
        *dot = '\0';
        sprintf(tex_name, "%s%s%s%s", dirpath, path_sep, trim, weave_ext);
        sprintf(aux_name, "%s%s%s.aux", dirpath, path_sep, trim);
        *dot = '.';
      } else {
        sprintf(tex_name, "%s%s%s%s", dirpath, path_sep, trim, weave_ext);
        sprintf(aux_name, "%s%s%s.aux", dirpath, path_sep, trim);
      }

      if (effective_format == 2) {
        markdown_flag = TRUE;
        number_flag = TRUE;
        collect_numbers(aux_name);
        write_md(source_name, tex_name, 0);
      } else {
        collect_numbers(aux_name);
        write_tex(source_name, tex_name, 0);
      }
    }

    if (output_flag)
      write_files(file_names);
  }
  arena_free();
}◇
```


Fragment referenced in 34.

Uses: `arena_free` 194b, `collect_numbers` 149a, `command_name` 29a, `current_sector` 42c, `exit` 21, `extract_name` 27, `FALSE` 22a, `file_names` 152a, `fprintf` 21, `macro_names` 152a, `map_flag` 27, `markdown_flag` 27, `number_flag` 27, `nw_char` 28bc, `output_flag` 27, `pass1` 38b, `prev_sector` 42c, `source_name` 117b, `stderr` 21, `tex_flag` 27, `TRUE` 22a, `weave_flag` 27, `weave_format` 27, `weave_format_from_cli` 27, `write_extract` 113b, `write_files` 114b, `write_map` 111a, `write_md` 84b, `write_tex` 55b.

2.3 Pass One

During the first pass, we scan the file, recording the definitions of each fragment and file and accumulating all the scraps.

⟨ *Function prototypes* 38a ⟩ ≡

```
extern void pass1(char *file_name);
◇
```

Fragment defined by 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c.
Fragment referenced in 20.
Uses: `file_name` 123c, `pass1` 38b.

The routine `pass1` takes a single argument, the name of the source file. It opens the file, then initializes the scrap structures (see Section 3.2) and the roots of the file-name tree, the fragment-name tree, and the tree of user-specified index entries (see Section 3.3). After completing all the necessary preparation, we make a pass over the file, filling in all our data structures. Next, we seach all the scraps for references to the user-specified index entries. Finally, we must reverse all the cross-reference lists accumulated while scanning the scraps.

`"pass1.c"` 38b≡

```
void pass1(char *file_name)
{
  if (verbose_flag)
    fprintf(stderr, "reading %s\n", file_name);
  source_open(file_name);
  init_scraps();
  macro_names = NULL;
  file_names = NULL;
  user_names = NULL;
  ⟨ Scan the source file, looking for at-sequences 38c ⟩
  if (tex_flag)
    search();
  ⟨ Reverse cross-reference lists 50b ⟩
}
◇
```

File defined by 22d, 38b.
Defines: `pass1` 37, 38a, 70a, 95a.
Uses: `file_name` 123c, `file_names` 152a, `fprintf` 21, `init_scraps` 124c, `macro_names` 152a, `search` 180a, `source_open` 123a, `stderr` 21, `tex_flag` 27, `user_names` 152a, `verbose_flag` 27.

The only thing we look for in the first pass are the command sequences. All ordinary text is skipped entirely.

⟨ *Scan the source file, looking for at-sequences* 38c ⟩ ≡

```
{
  int c = source_get();
  while (c != EOF) {
    if (c == nw_char)
      ⟨ Scan at-sequence 41 ⟩
    c = source_get();
  }
}◇
```

Fragment referenced in 38b.
Uses: `nw_char` 28bc, `source_get` 118c.

The `@l` command (lowercase) sets the language for the next `@o` or `@d` directive by storing the tag in `pending_lang`. It replaces the earlier mechanism of detecting Markdown fences (```lang) and LaTeX markers (`%%%lang`) during pass1. A typical usage is:

```
@l python
@d Validate input
@{import re
...
@}
```

⟨ *Collect language apply in pass1* 39a ⟩ ≡

```
{
  char lang[64];
  char *lp = lang;
  int lc = source_get();
  /* skip optional whitespace */
  while (lc == ' ' || lc == '\t')
    lc = source_get();
  /* read language tag */
  while (lc != EOF && lc != '\n' && lc != ' ' && lc != '\t'
         && (lp - lang) < 62) {
    *lp++ = lc;
    lc = source_get();
  }
  *lp = '\0';
  if (lang[0] != '\0') {
    strncpy(pending_lang, lang, 63);
    pending_lang[63] = '\0';
  }
  /* consume rest of line */
  while (lc != EOF && lc != '\n')
    lc = source_get();
}◇
```

Fragment referenced in 41.
Uses: `pending_lang` 152a, `source_get` 118c.

The `@L` command (Section 3.3.2) lets users register custom language mappings. During pass1, the actual parsing and registration is handled by `collect_lang_def()` in `names.c`. During the weave passes (LaTeX and Markdown), we simply consume the rest of the line so that the arguments do not leak into the typeset output.

⟨ *Collect user language definition in pass1* 39b ⟩ ≡

```
collect_lang_def();◇
```

Fragment referenced in 41.
Uses: `collect_lang_def` 161.

⟨ *Collect weave format declaration in pass1* 39c ⟩ ≡

```
collect_weave_format();◇
```

Fragment referenced in 41.
Uses: `collect_weave_format` 162.

⟨ *Skip at-L language definition line* 40 ⟩ ≡

```
{ int lc = source_get();
  while (lc != '\n' && lc != EOF)
    lc = source_get();
}
c = source_get();◇
```

Fragment referenced in 57a, 86a.
Uses: `source_get` 118c.

Six at-sequences are interesting during the first pass: `@o` and `@O` (file definitions), `@d`, `@D`, and `@q` (fragment definitions), `@c` (block comments), `@s`/`@S` (sectors), `@l` (language apply), and `@L` (language definitions). We skip past all others immediately, warning if unexpected sequences are discovered.

⟨ *Scan at-sequence* 41 ⟩ ≡

```
{
  char quoted = 0;

  c = source_get();
  switch (c) {
    case 'r':
          c = source_get();
          nw_char = c;
          update_delimit_scrap();
          break;
    case 'O':
    case 'o': ⟨ Build output file definition 42e ⟩
              break;
    case 'Q':
    case 'q': quoted = 1;
    case 'D':
    case 'd': ⟨ Build fragment definition 43a ⟩
              break;
    case 's':
              ⟨ Step to next sector 42a ⟩
              break;
    case 'S':
              ⟨ Close the current sector 42b ⟩
              break;
    case '<':
    case '(':
    case '[':
    case '{': ⟨ Skip over an in-text scrap 43c ⟩
              break;
    case 'c': ⟨ Collect a block comment 44b ⟩
              break;
    case 'l': ⟨ Collect language apply in pass1 39a ⟩
              break;
    case 'L': ⟨ Collect user language definition in pass1 39b ⟩
              break;
    case 'W': ⟨ Collect weave format declaration in pass1 39c ⟩
              break;
    case 'x':
    case 'v':
    case 'u':
    case 'm':
    case 'f': /* ignore during this pass */
              break;
    default:  if (c==nw_char) /* ignore during this pass */
                break;
              fprintf(stderr,
                      "%s: unexpected %c sequence ignored (%s, line %d)\n",
                      command_name, nw_char, source_name, source_line);
              break;
  }
}◇
```

Fragment referenced in 38c.
Uses: `command_name` 29a, `fprintf` 21, `nw_char` 28bc, `source_get` 118c, `source_line` 117b, `source_name` 117b, `stderr` 21, `update_delimit_scrap` 69a.

⟨ *Step to next sector* 42a ⟩ ≡

```
prev_sector += 1;
current_sector = prev_sector;
c = source_get();
◇
```

Fragment referenced in 41, 57a, 86a.
Uses: `current_sector` 42c, `prev_sector` 42c, `source_get` 118c.

⟨ *Close the current sector* 42b ⟩ ≡

```
current_sector = 1;
c = source_get();
◇
```

Fragment referenced in 41, 57a, 86a.
Uses: `current_sector` 42c, `source_get` 118c.

⟨ *Global variable declarations* 42c ⟩ ≡

```
extern unsigned char current_sector;
extern unsigned char prev_sector;
◇
```

Fragment defined by 27, 28b, 29a, 42c, 47c, 63b, 117b, 127a, 152a, 192b.
Fragment referenced in 20.
Defines: `current_sector` 37, 42abd, 77b, 81c, 101a, 104a, 128c, 132, 167, 171, `prev_sector` 37, 42ad.

⟨ *Global variable definitions* 42d ⟩ ≡

```
unsigned char current_sector = 1;
unsigned char prev_sector = 1;
◇
```

Fragment defined by 28ac, 29b, 42d, 47d, 63c, 117c, 127b, 152b.
Fragment referenced in 24e.
Uses: `current_sector` 42c, `prev_sector` 42c.

2.3.1 Accumulating Definitions

There are three steps required to handle a definition:

1. Build an entry for the name so we can look it up later.
2. Collect the scrap and save it in the table of scraps.
3. Attach the scrap to the name.

We go through the same steps for both file names and fragment names.

⟨ *Build output file definition* 42e ⟩ ≡

```
{
  Name *name = collect_file_name(); /* returns a pointer to the name entry */
  int scrap = collect_scrap();      /* returns an index to the scrap */
  ⟨ Add scrap to name's definition list 43b ⟩
}◇
```

Fragment referenced in 41.
Uses: `collect_file_name` 163, `collect_scrap` 128b, `Name` 151b.

⟨ *Build fragment definition* 43a ⟩ ≡

```
{
  Name *name = collect_macro_name();
  int scrap = collect_scrap();
  ⟨ Add scrap to name's definition list 43b ⟩
}◇
```

Fragment referenced in 41.
Uses: `collect_macro_name` 167, `collect_scrap` 128b, `Name` 151b.

Since a file or fragment may be defined by many scraps, we maintain them in a simple linked list. The list is actually built in reverse order, with each new definition being added to the head of the list.

⟨ *Add* `scrap` *to* `name`*'s definition list* 43b ⟩ ≡

```
{
  Scrap_Node *def = (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
  def->scrap = scrap;
  def->quoted = quoted;
  def->next = name->defs;
  name->defs = def;
}◇
```

Fragment referenced in 42e, 43a.
Uses: `arena_getmem` 193b, `defs` 123c, `next` 123b, `Scrap_Node` 151a.

⟨ *Skip over an in-text scrap* 43c ⟩ ≡

```
{
   int c;
   int depth = 1;
   while ((c = source_get()) != EOF) {
      if (c == nw_char)
         ⟨ Skip over at-sign or go to skipped 44a ⟩
   }
   fprintf(stderr, "%s: unexpected EOF in text at (%s, %d)\n",
                   command_name, source_name, source_line);
   exit(-1);

skipped:  ;
}
◇
```

Fragment referenced in 41.
Uses: `command_name` 29a, `exit` 21, `fprintf` 21, `nw_char` 28bc, `source_get` 118c, `source_line` 117b, `source_name` 117b, `stderr` 21.

⟨ *Skip over at-sign or go to skipped* 44a ⟩ ≡

```
{
   c = source_get();
   switch (c) {
     case '{': case '[': case '(': case '<':
        depth += 1;
        break;
     case '}': case ']': case ')': case '>':
        if (--depth == 0)
           goto skipped;
     case 'x': case '|': case ',':
     case '%': case '1': case '2':
     case '3': case '4': case '5': case '6':
     case '7': case '8': case '9': case '_':
     case 'f': case '#': case '+': case '-':
     case 'v': case '*': case 'c': case '\'':
     case 's':
        break;
     default:
        if (c != nw_char) {
           fprintf(stderr, "%s: unexpected %c%c in text at (%s, %d)\n",
                           command_name, nw_char, c, source_name, source_line);
           exit(-1);
        }
        break;
   }
}
◇
```

Fragment referenced in 43c.
Uses: `command_name` 29a, `exit` 21, `fprintf` 21, `nw_char` 28bc, `source_get` 118c, `source_line` 117b, `source_name` 117b, `stderr` 21.

2.3.2 Block Comments

⟨ *Collect a block comment* 44b ⟩ ≡

```
{
   char * p = blockBuff;
   char * e = blockBuff + (sizeof(blockBuff)/sizeof(blockBuff[0])) - 1;

   ⟨ Skip whitespace 45a ⟩
   while (p < e)
   {
      ⟨ Add one char to the block buffer 45b ⟩
   }
   if (p == e)
   {
      ⟨ Skip to the next nw-char 47b ⟩
   }
   *p = '\000';
}
◇
```

Fragment referenced in 41.
Uses: `blockBuff` 47c.

⟨ *Skip whitespace* 45a ⟩ ≡

```
while (source_peek == ' '
       || source_peek == '\t'
       || source_peek == '\n')
   (void)source_get();
◇
```

Fragment referenced in 44b.
Uses: `source_get` 118c, `source_peek` 118c.

⟨ *Add one char to the block buffer* 45b ⟩ ≡

```
int c = source_get();

if (c == nw_char)
{
   ⟨ Add an at character to the block or break 46a ⟩
}
else if (c == EOF)
{
   source_ungetc(&c);
   break;
}
else
{
   ⟨ Add any other character to the block 46b ⟩
}
◇
```

Fragment referenced in 44b.
Uses: `nw_char` 28bc, `source_get` 118c, `source_ungetc` 119b.

⟨ *Add an at character to the block or break* 46a ⟩ ≡

```
int cc = source_peek;

if (cc == 'c')
{
   do
      c = source_get();
   while (c <= ' ');

   break;
}
else if (cc == 'd'
         || cc == 'D'
         || cc == 'q'
         || cc == 'Q'
         || cc == 'o'
         || cc == 'O'
         || cc == EOF)
{
   source_ungetc(&c);
   break;
}
else
{
   *p++ = c;
   *p++ = source_get();
}
◇
```

Fragment referenced in 45b.
Uses: `source_get` 118c, `source_peek` 118c, `source_ungetc` 119b.

⟨ *Add any other character to the block* 46b ⟩ ≡

```
   ⟨ Perhaps skip white-space 47a ⟩
   *p++ = c;
◇
```

Fragment referenced in 45b.

⟨ *Perhaps skip white-space* 47a ⟩ ≡

```
if (c == ' ')
{
   while (source_peek == ' ')
      c = source_get();
}
if (c == '\n')
{
   if (source_peek == '\n')
   {
      do
         c = source_get();
      while (source_peek == '\n');
   }
   else
      c = ' ';
}
◇
```

Fragment referenced in 46b.
Uses: `source_get` 118c, `source_peek` 118c.

⟨ *Skip to the next nw-char* 47b ⟩ ≡

```
int c;

while ((c = source_get()), c != nw_char && c != EOF)/* Skip */
source_ungetc(&c);◇
```

Fragment referenced in 44b.
Uses: `nw_char` 28bc, `source_get` 118c, `source_ungetc` 119b.

⟨ *Global variable declarations* 47c ⟩ ≡

```
extern char blockBuff[6400];
◇
```

Fragment defined by 27, 28b, 29a, 42c, 47c, 63b, 117b, 127a, 152a, 192b.
Fragment referenced in 20.
Defines: `blockBuff` 44b, 47d, 48b.

⟨ *Global variable definitions* 47d ⟩ ≡

```
char blockBuff[6400];
◇
```

Fragment defined by 28ac, 29b, 42d, 47d, 63c, 117c, 127b, 152b.
Fragment referenced in 24e.
Uses: `blockBuff` 47c.

We don't show block comments inside scraps in the TEX file, but we do show where they go.

⟨ *Show presence of a block comment* 48a ⟩ ≡

```
{
  fputs(delimit_scrap[scrap_type][1],file);
  fprintf(file, "\\hbox{\\sffamily\\slshape (Comment)}");
  fputs(delimit_scrap[scrap_type][0], file);
}
◇
```

Fragment referenced in 70a.
Uses: `fprintf` 21, `fputs` 21, `scrap_type` 67b, 68.

The block comment is presently in the block buffer. Here we copy it into the scrap whence we shall copy it into the output file.

⟨ *Include block comment in a scrap* 48b ⟩ ≡

```
{
   char * p = blockBuff;

   push(nw_char, &writer);
   do
   {
      push(c, &writer);
      c = *p++;
   } while (c != '\0');
}◇
```

Fragment referenced in 130.
Uses: `blockBuff` 47c, `nw_char` 28bc, `push` 127d.

⟨ *Copy block comment from scrap* 49a ⟩ ≡

```
{
   int bgn = indent + global_indent;
   int posn = bgn + strlen(comment_begin[comment_flag]);
   int i;

   ⟨ Perhaps put a delayed indent 146b ⟩
   c = pop(&reader);
   fputs(comment_begin[comment_flag], file);
   while (c != '\0')
   {
      ⟨ Move a word to the file 49b ⟩
      ⟨ If we break the line at this word 49c ⟩
      {
         putc('\n', file);
         for (i = 0; i < bgn ; i++)
            putc(' ', file);
         c = pop(&reader);
         if (c != '\0')
         {
            posn = bgn + strlen(comment_mid[comment_flag]);
            fputs(comment_mid[comment_flag], file);
         }
      }
   }
   fputs(comment_end[comment_flag], file);
}
◇
```

Fragment referenced in 144a.
Uses: `comment_begin` 166, `comment_end` 166, `comment_mid` 166, `fputs` 21, `pop` 134c, `putc` 21, `strlen` 21.

⟨ *Move a word to the file* 49b ⟩ ≡

```
do
{
   putc(c, file);
   posn += 1;
   c = pop(&reader);
} while (c > ' ');
◇
```

Fragment referenced in 49a.
Uses: `pop` 134c, `putc` 21.

⟨ *If we break the line at this word* 49c ⟩ ≡

```
if (c == '\n' || (c == ' ' && posn > 60))◇
```

Fragment referenced in 49a.

⟨ *Skip over a block comment* 49d ⟩ ≡

```
if (last == nw_char && c == 'c')
   while ((c = pop(reader.m)) != '\0')
      /* Skip */;
◇
```

Fragment referenced in 183.
Uses: `nw_char` 28bc, `pop` 134c.

⟨ *Begin or end a block comment* 50a ⟩ ≡

```
if (inBlock)
{
   ⟨ End block 62c ⟩
}
else
{
   ⟨ Start block 62a ⟩
}◇
```

Fragment referenced in 57a.

2.3.3 Fixing the Cross References

Since the definition and reference lists for each name are accumulated in reverse order, we take the time at the end of `pass1` to reverse them all so they'll be simpler to print out prettily. The code for `reverse_lists` appears in Section 3.3.

⟨ *Reverse cross-reference lists* 50b ⟩ ≡

```
{
  reverse_lists(file_names);
  reverse_lists(macro_names);
  reverse_lists(user_names);
}◇
```

Fragment referenced in 38b.
Uses: `file_names` 152a, `macro_names` 152a, `reverse_lists` 174c, `user_names` 152a.

2.3.4 Dealing with fragment parameters

Fragment parameters were added on later in nuweb's development. There still is not, for example, an index of fragment parameters. We need a data type to keep track of fragment parameters.

⟨ *Type declarations* 50c ⟩ ≡

```
typedef int *Parameters;
◇
```

Fragment defined by 22a, 50c, 114c, 151ab, 168c, 170b, 173d, 185c, 192a.
Fragment referenced in 20.
Defines: `Parameters` 53, 58b, 87b, 124b, 138b, 139a, 141.

When we are copying a scrap to the output, we check for an embedded parameter. If we find it we copy it. Otherwise it's an old-style parameter and we can then pull the nth string from the `Parameters` list when we see an `@1` `@2`, etc.

⟨ *Handle macro parameter substitution* 50d ⟩ ≡

```
case '1': case '2': case '3':
case '4': case '5': case '6':
case '7': case '8': case '9':
  {
    Arglist * args;
    Name * name;

    lookup(c - '1', inArgs, inParams, &name, &args);
```

```
    if (name == (Name *)1) {
      Embed_Node * q = (Embed_Node *)args;
      indent = write_scraps(file, spelling, q->defs,
                            global_indent + indent,
                            indent_chars, debug_flag,
                            tab_flag, indent_flag,
                            0, 0,
                            q->args, inParams,
                            local_parameters, "");
    }
    else if (name != NULL) {
       int i, narg;
       char * p = name->spelling;
       Arglist *q = args;

       ⟨ Perhaps comment this macro 146a ⟩
       indent = write_scraps(file, spelling, name->defs,
                             global_indent + indent,
                             indent_chars, debug_flag,
                             tab_flag, indent_flag,
                             comment_flag, location_flag,
                             args, name->arg,
                             local_parameters, p);
    }
    else if (args != NULL) {
       if (delayed_indent) {
         ⟨ Put out the indent 143b ⟩
       }
       fputs((char *)args, file);
    }
    else if ( parameters && parameters[c - '1'] ) {
      Scrap_Node param_defs;
      param_defs.scrap = parameters[c - '1'];
      param_defs.next = 0;
      write_scraps(file, spelling, &param_defs,
                   global_indent + indent,
                   indent_chars, debug_flag,
                   tab_flag, indent_flag,
                   comment_flag, location_flag,
                   NULL, NULL, 0, "");
    } else if (delayed_indent) {
      ⟨ Put out the indent 143b ⟩
    }
  }
◇
```

Fragment referenced in 144a.
Uses: `Arglist` 170b, `defs` 123c, `delayed_indent` 139b, `Embed_Node` 173d, `fputs` 21, `lookup` 136b, `Name` 151b, `next` 123b, `Scrap_Node` 151a, `write_scraps` 139a.

Now onto actually parsing fragment parameters from a call. We start off checking for fragment parameters, an `@(` sequence followed by parameters separated by `@,` sequences, and terminated by a `@)` sequence.

We collect separate scraps for each parameter, and write the scrap numbers down in the text. For example, if the file has:

```
@<foo @( param1 @, param2 @)@>
```

we actually make new scraps, say 10 and 11, for param1 and param2, and write in the collected scrap:

```
@<foo @(10@,11@)@>
```

⟨ *Save macro parameters* 52 ⟩ ≡

```
{
  int param_scrap;
  char param_buf[10];

  push(nw_char, &writer);
  push('(', &writer);
  do {
     param_scrap = collect_scrap();
     sprintf(param_buf, "%d", param_scrap);
     pushs(param_buf, &writer);
     push(nw_char, &writer);
     push(scrap_ended_with, &writer);
     add_to_use(name, current_scrap);
  } while( scrap_ended_with == ',' );
  do
    c = source_get();
  while( ' ' == c );
  if (c == nw_char) {
    c = source_get();
  }
  if (c != '>') {
    /* ZZZ print error */;
  }
}◇
```

Fragment referenced in 133c.
Uses: `add_to_use` 134a, `collect_scrap` 128b, `nw_char` 28bc, `push` 127d, `pushs` 128a, `source_get` 118c.

If we get inside, we have at least one parameter, which will be at the beginning of the parms buffer, and we prime the pump with the first character.

⟨ *Check for macro parameters* 53 ⟩ ≡

```
if (c == '(') {
  Parameters res = arena_getmem(10 * sizeof(int));
  int *p2 = res;
  int count = 0;
  int scrapnum;

  while( c && c != ')' ) {
    scrapnum = 0;
    c = pop(manager);
    while( '0' <= c && c <= '9' ) {
      scrapnum = scrapnum  * 10 + c - '0';
      c = pop(manager);
    }
    if ( c == nw_char ) {
      c = pop(manager);
    }
    *p2++ = scrapnum;
  }
  while (count < 10) {
    *p2++ = 0;
    count++;
  }
  while( c && c != nw_char ) {
      c = pop(manager);
  }
  if ( c == nw_char ) {
    c = pop(manager);
  }
  *parameters = res;
}
```

◇

Fragment referenced in 138c.
Uses: `arena_getmem` 193b, `nw_char` 28bc, `Parameters` 50c, `pop` 134c.

These are used in `write_tex` to output the argument list for a fragment.

⟨ *Format macro parameters* 54a ⟩ ≡

```
    char sep;

    sep = '(';
    do {
      fputc(sep,file);

      fputs("{\\footnotesize ", file);
      write_single_scrap_ref(file, scraps + 1);
      fprintf(file, "\\label{scrap%d}\n", scraps + 1);
      fputs(" }", file);

      source_last = '{';
      copy_scrap(file, TRUE, NULL);

      ++scraps;

      sep = ',';
    } while ( source_last != ')' && source_last != EOF );
    fputs(" ) ",file);
    do
      c = source_get();
    while(c != nw_char && c != EOF);
    if (c == nw_char) {
      c = source_get();
    }
◇
```

Fragment referenced in 73, 75.
Uses: `copy_scrap` 68, `fprintf` 21, `fputs` 21, `nw_char` 28bc, `scraps` 124a, `source_get` 118c, `source_last` 118c, `TRUE` 22a, `write_single_scrap_ref` 125b.

2.4 Writing the Latex File

The second pass (invoked via a call to `write_tex`) copies most of the text from the source file straight into a `.tex` file. Definitions are formatted slightly and cross-reference information is printed out.

Note that all the formatting is handled in this section. If you don't like the format of definitions or indices or whatever, it'll be in this section somewhere. Similarly, if someone wanted to modify weft to work with a different typesetting system, this would be the place to look.

⟨ *Function prototypes* 54b ⟩ ≡

```
extern void write_tex(char *file_name, char *tex_name, unsigned char sector);
◇
```

Fragment defined by 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c.
Fragment referenced in 20.
Uses: `file_name` 123c, `sector` 123c, `write_tex` 55b.

We need a few local function declarations before we get into the body of `write_tex`.

"latex.c" 55a≡

```
static void copy_scrap(FILE *file, int prefix, Name *name);            /* formats the body of a scrap */
static void print_scrap_numbers(FILE *tex_file, Scrap_Node *scraps);      /* formats a list of scrap numbe
static void format_entry(Name *name, FILE *tex_file, unsigned char sector);            /* formats an inde
static void format_file_entry(Name *name, FILE *tex_file);        /* formats a file index entry */
static void format_user_entry(Name *name, FILE *tex_file, unsigned char sector);
static void write_arg(FILE *tex_file, char *p);
static void write_literal(FILE *tex_file, char *p, int mode);
static void write_ArglistElement(FILE *file, Arglist *args, char **params);
◇
```

File defined by 22e, 55ab, 61a, 65b, 66ab, 67bc, 68, 69a, 75, 76b, 78ab, 81b, 82a.
Uses: `Arglist` 170b, `copy_scrap` 68, `FILE` 21, `format_entry` 78b, `format_file_entry` 76b, `format_user_entry` 82a, `Name` 151b, `print_scrap_numbers` 65b, `scraps` 124a, `Scrap_Node` 151a, `sector` 123c, `write_arg` 61a, `write_ArglistElement` 75, `write_literal` 67c.

The routine `write_tex` takes two file names as parameters: the name of the web source file and the name of the `.tex` output file.

"latex.c" 55b≡

```
void write_tex(char *file_name, char *tex_name, unsigned char sector)
{
  (void)sector;
  FILE *tex_file = fopen(tex_name, "w");
  if (tex_file) {
    if (verbose_flag)
      fprintf(stderr, "writing %s\n", tex_name);
    source_open(file_name);
    ⟨ Write LaTeX limbo definitions 56a ⟩
    ⟨ Copy source_file into tex_file 56c ⟩
    fclose(tex_file);
  }
  else
    fprintf(stderr, "%s: can't open %s\n", command_name, tex_name);
}
◇
```

File defined by 22e, 55ab, 61a, 65b, 66ab, 67bc, 68, 69a, 75, 76b, 78ab, 81b, 82a.
Defines: `write_tex` 37, 54b.
Uses: `command_name` 29a, `fclose` 21, `FILE` 21, `file_name` 123c, `fopen` 21, `fprintf` 21, `sector` 123c, `source_open` 123a, `stderr` 21, `verbose_flag` 27.

Now that the `\NW...` macros are used, it seems convenient to write default definitions for those macros so that source files need not define anything new. If a user wants to change any of the macros (to use hyperref or to write in some language other than english) he or she can redefine the commands.

⟨ *Write LaTeX limbo definitions* 56a ⟩ ≡

```
if (hyperref_flag) {
   fputs("\\newcommand{\\NWtarget}[2]{\\hypertarget{#1}{#2}}\n", tex_file);
   fputs("\\newcommand{\\NWlink}[2]{\\hyperlink{#1}{#2}}\n", tex_file);
} else {
   fputs("\\newcommand{\\NWtarget}[2]{#2}\n", tex_file);
   fputs("\\newcommand{\\NWlink}[2]{#2}\n", tex_file);
}
fputs("\\newcommand{\\NWtxtMacroDefBy}{Fragment defined by}\n", tex_file);
fputs("\\newcommand{\\NWtxtMacroRefIn}{Fragment referenced in}\n", tex_file);
fputs("\\newcommand{\\NWtxtMacroNoRef}{Fragment never referenced}\n", tex_file);
fputs("\\newcommand{\\NWtxtDefBy}{Defined by}\n", tex_file);
fputs("\\newcommand{\\NWtxtRefIn}{Referenced in}\n", tex_file);
fputs("\\newcommand{\\NWtxtNoRef}{Not referenced}\n", tex_file);
fputs("\\newcommand{\\NWtxtFileDefBy}{File defined by}\n", tex_file);
fputs("\\newcommand{\\NWtxtIdentsUsed}{Uses:}\n", tex_file);
fputs("\\newcommand{\\NWtxtIdentsNotUsed}{Never used}\n", tex_file);
fputs("\\newcommand{\\NWtxtIdentsDefed}{Defines:}\n", tex_file);
fputs("\\newcommand{\\NWsep}{${\\diamond}$}\n", tex_file);
fputs("\\newcommand{\\NWnotglobal}{(not defined globally)}\n", tex_file);
fputs("\\newcommand{\\NWuseHyperlinks}{", tex_file);
if (hyperoptions[0] != '\0')
{
   ⟨ Write the hyperlink usage macro 56b ⟩
}
fputs("}\n", tex_file);
◇
```

Fragment referenced in 55b.
Uses: `fputs` 21, `hyperoptions` 27, `hyperref_flag` 27, `Uses` 185c.

⟨ *Write the hyperlink usage macro* 56b ⟩ ≡

```
fprintf(tex_file, "\\usepackage[%s]{hyperref}", hyperoptions);◇
```

Fragment referenced in 56a.
Uses: `fprintf` 21, `hyperoptions` 27.

We make our second (and final) pass through the source web, this time copying characters straight into the `.tex` file. However, we keep an eye peeled for `@` characters, which signal a command sequence.

⟨ *Copy* `source_file` *into* `tex_file` 56c ⟩ ≡

```
{
  int inBlock = FALSE;
  int c = source_get();
  while (c != EOF) {
    if (c == nw_char)
      {
      ⟨ Interpret at-sequence 57a ⟩
      }
    else {
      putc(c, tex_file);
      c = source_get();
    }
  }
}◇
```

Fragment referenced in 55b.
Uses: `FALSE` 22a, `nw_char` 28bc, `putc` 21, `source_get` 118c.

⟨ *Interpret at-sequence* 57a ⟩ ≡

```
{
  int big_definition = FALSE;
  c = source_get();
  switch (c) {
    case 'r':
          c = source_get();
          nw_char = c;
          update_delimit_scrap();
          break;
    case 'O': big_definition = TRUE;
    case 'o': ⟨ Write output file definition 59 ⟩
              break;
    case 'Q':
    case 'D': big_definition = TRUE;
    case 'q':
    case 'd': ⟨ Write macro definition 60a ⟩
              break;
    case 's':
              ⟨ Step to next sector 42a ⟩
              break;
    case 'S':
              ⟨ Close the current sector 42b ⟩
              break;
    case '{':
    case '[':
    case '(': ⟨ Write in-text scrap 63d ⟩
              break;
    case '<': ⟨ Expand macro into tex_file 58b ⟩
              break;
    case 'x': ⟨ Copy label from source into (57b tex_file ) 71b ⟩
              c = source_get();
              break;
    case 'c': ⟨ Begin or end a block comment 50a ⟩
              c = source_get();
              break;
    case 'f': ⟨ Write index of file names 76a ⟩
              break;
    case 'm': ⟨ Write index of macro names 77b ⟩
              break;
    case 'u': ⟨ Write index of user-specified names 81c ⟩
              break;
    case 'v': ⟨ Copy version info into tex file 58a ⟩
              break;
    case 'l':
    case 'L': ⟨ Skip at-L language definition line 40 ⟩
              break;
    case 'W': ⟨ Skip at-L language definition line 40 ⟩
              break;
    default:
          if (c==nw_char)
            putc(c, tex_file);
          c = source_get();
              break;
  }
}◇
```

Fragment referenced in 56c.

⟨ *Copy version info into tex file* 58a ⟩ ≡

```
fputs(version_string, tex_file);
c = source_get();
◇
```

Fragment referenced in 57a.
Uses: `fputs` 21, `source_get` 118c, `version_string` 27.

⟨ *Expand macro into file* 58b ⟩ ≡

```
{
   Parameters local_parameters = 0;
   int changed;
   char indent_chars[MAX_INDENT];
   Arglist *a;
   Name *name;
   Arglist * args;
   a = collect_scrap_name(0);
   name = a->name;
   args = instance(a->args, NULL, NULL, &changed);
   name->mark = TRUE;
   write_scraps(file, tex_name, name->defs, 0, indent_chars, 0, 0, 1, 0, 0,
         args, name->arg, local_parameters, tex_name);
   name->mark = FALSE;
   c = source_get();
}
◇
```

Fragment referenced in 57a.
Uses: `Arglist` 170b, `collect_scrap_name` 171, `defs` 123c, `FALSE` 22a, `instance` 137a, `MAX_INDENT` 114c, `Name` 151b, `Parameters` 50c, `source_get` 118c, `TRUE` 22a, `write_scraps` 139a.

2.4.1 Formatting Definitions

We go through a fair amount of effort to format a file definition. I've derived most of the LaTeX commands experimentally; it's quite likely that an expert could do a better job. The LaTeX for the previous fragment definition should look like this (perhaps modulo the scrap references):

```
\begin{flushleft} \small
\begin{minipage}{\linewidth}\label{scrap70}\raggedright\small
$\langle$Interpret at-sequence {\footnotesize 18}$\rangle\equiv$
\vspace{-1ex}
\begin{list}{}{} \item
\mbox{}\verb@{@\\
\mbox{}\verb@  int big_definition = FALSE;@\\
\mbox{}\verb@  c = source_get();@\\
\mbox{}\verb@  switch (c) {@\\
\mbox{}\verb@    case 'O': big_definition = TRUE;@\\
\mbox{}\verb@    case 'o': @$\langle$Write output file definition {\footnotesize 19a}$\rangle$\verb@@\\

⋮

\mbox{}\verb@    case '@{\tt @}\verb@': putc(c, tex_file);@\\
\mbox{}\verb@    default:  c = source_get();@\\
\mbox{}\verb@              break;@\\
\mbox{}\verb@  }@\\
\mbox{}\verb@}@$\diamond$
\end{list}
```

```
\vspace{-1ex}
\footnotesize\addtolength{\baselineskip}{-1ex}
\begin{list}{}{\setlength{\itemsep}{-\parsep}\setlength{\itemindent}{-\leftmargin}}
\item Fragment referenced in scrap 17b.
\end{list}
\end{minipage}\vspace{4ex}
\end{flushleft}
```

The *flushleft* environment is used to avoid LaTeX warnings about underful lines. The *minipage* environment is used to avoid page breaks in the middle of scraps. The *verb* command allows arbitrary characters to be printed (however, note the special handling of the `@` case in the switch statement).

Fragment and file definitions are formatted nearly identically. I've factored the common parts out into separate scraps.

⟨ *Write output file definition* 59 ⟩ ≡

```
{
  Name *name = collect_file_name();
  ⟨ Begin the scrap environment 61b ⟩
  fputs("\\NWtarget{weft", tex_file);
  write_single_scrap_ref(tex_file, scraps);
  fputs("}{} ", tex_file);
  fprintf(tex_file, "\\verb%c\"%s\"%c\\nobreak\\ {\\footnotesize {", nw_char, name->spelling, nw_char);
  write_single_scrap_ref(tex_file, scraps);
  fputs("}}$\\equiv$\n", tex_file);
  ⟨ Fill in the middle of the scrap environment 62d ⟩
  ⟨ Begin the cross-reference environment 64a ⟩
  if ( scrap_flag ) {
    ⟨ Write file defs 64c ⟩
  }
  format_defs_refs(tex_file, scraps);
  format_uses_refs(tex_file, scraps++);
  ⟨ Finish the cross-reference environment 64b ⟩
  ⟨ Finish the scrap environment 63a ⟩
}◇
```

Fragment referenced in 57a.
Uses: `collect_file_name` 163, `format_defs_refs` 187d, `format_uses_refs` 186b, `fprintf` 21, `fputs` 21, `Name` 151b, `nw_char` 28bc, `scraps` 124a, `scrap_flag` 27, `write_single_scrap_ref` 125b.

I don't format a fragment name at all specially, figuring the programmer might want to use italics or bold face in the midst of the name.

⟨ *Write macro definition* 60a ⟩ ≡

```
{
  Name *name = collect_macro_name();

  ⟨ Begin the scrap environment 61b ⟩
  fputs("\\NWtarget{weft", tex_file);
  write_single_scrap_ref(tex_file, scraps);
  fputs("}{} $\\langle\\,${\\itshape ", tex_file);
  ⟨ Write the macro's name 60b ⟩
  fputs("}\\nobreak\\ {\\footnotesize {", tex_file);
  write_single_scrap_ref(tex_file, scraps);
  fputs("}}$\\,\\rangle\\equiv$\n", tex_file);
  ⟨ Fill in the middle of the scrap environment 62d ⟩
  ⟨ Begin the cross-reference environment 64a ⟩
  ⟨ Write macro defs 64d ⟩
  ⟨ Write macro refs 65a ⟩
  format_defs_refs(tex_file, scraps);
  format_uses_refs(tex_file, scraps++);
  ⟨ Finish the cross-reference environment 64b ⟩
  ⟨ Finish the scrap environment 63a ⟩
}◇
```

Fragment referenced in 57a.
Uses: `collect_macro_name` 167, `format_defs_refs` 187d, `format_uses_refs` 186b, `fputs` 21, `Name` 151b, `scraps` 124a, `write_single_scrap_ref` 125b.

⟨ *Write the macro's name* 60b ⟩ ≡

```
{
  char * p = name->spelling;
  int i = 0;

  while (*p != '\000') {
    if (*p == ARG_CHR) {
      write_arg(tex_file, name->arg[i++]);
      p++;
    }
    else
       fputc(*p++, tex_file);
  }
}◇
```

Fragment referenced in 60a, 79b.
Uses: `ARG_CHR` 168c, `write_arg` 61a.

"latex.c" 61a≡

```
static void write_arg(FILE * tex_file, char * p)
{
   fputs("\\hbox{\\slshape\\sffamily ", tex_file);
   while (*p)
   {
      switch (*p)
      {
      case '$':
      case '_':
      case '^':
      case '#':
         fputc('\\', tex_file);
         break;
      default:
         break;
      }
      fputc(*p, tex_file);
      p++;
   }

   fputs("\\/}", tex_file);
}
◇
```

File defined by 22e, 55ab, 61a, 65b, 66ab, 67bc, 68, 69a, 75, 76b, 78ab, 81b, 82a.
Defines: `write_arg` 55a, 60b, 70a, 75.
Uses: `FILE` 21, `fputs` 21.

⟨ *Begin the scrap environment* 61b ⟩ ≡

```
{
  if (big_definition)
  {
    if (inBlock)
    {
       ⟨ End block 62c ⟩
    }
    fputs("\\begin{flushleft} \\small", tex_file);
  }
  else
  {
    if (inBlock)
    {
       ⟨ Switch block 62b ⟩
    }
    else
    {
       ⟨ Start block 62a ⟩
    }
  }
  fprintf(tex_file, "\\label{scrap%d}\\raggedright\\small\n", scraps);
}◇
```

Fragment referenced in 59, 60a.
Uses: `fprintf` 21, `fputs` 21, `scraps` 124a.

⟨ *Start block* 62a ⟩ ≡

```
fputs("\\begin{flushleft} \\small\n\\begin{minipage}{\\linewidth}", tex_file);
inBlock = TRUE;◇
```

Fragment referenced in 50a, 61b.
Uses: `fputs` 21, `TRUE` 22a.

⟨ *Switch block* 62b ⟩ ≡

```
fputs("\\par\\vspace{\\baselineskip}\n",  tex_file);◇
```

Fragment referenced in 61b.
Uses: `fputs` 21.

⟨ *End block* 62c ⟩ ≡

```
fputs("\\end{minipage}\\vspace{4ex}\n",  tex_file);
fputs("\\end{flushleft}\n", tex_file);
inBlock = FALSE;◇
```

Fragment referenced in 50a, 61b, 63a.
Uses: `FALSE` 22a, `fputs` 21.

The interesting things here are the ◇ inserted at the end of each scrap and the various spacing commands. The diamond helps to clearly indicate the end of a scrap. The spacing commands were derived empirically; they may be adjusted to taste.

⟨ *Fill in the middle of the scrap environment* 62d ⟩ ≡

```
{
  fputs("\\vspace{-1ex}\n\\begin{list}{}{} \\item\n", tex_file);
  extra_scraps = 0;
  copy_scrap(tex_file, TRUE, name);
  fputs("{\\NWsep}\n\\end{list}\n", tex_file);
}◇
```

Fragment referenced in 59, 60a.
Uses: `copy_scrap` 68, `extra_scraps` 63b, `fputs` 21, `TRUE` 22a.

We've got one last spacing command, controlling the amount of white space after a scrap.

Note also the whitespace eater. I use it to remove any blank lines that appear after a scrap in the source file. This way, text following a scrap will not be indented. Again, this is a matter of personal taste.

⟨ *Finish the scrap environment* 63a ⟩ ≡

```
{
  scraps += extra_scraps;
  if (big_definition)
    fputs("\\vspace{4ex}\n\\end{flushleft}\n", tex_file);
  else
  {
     ⟨ End block 62c ⟩
  }
  do
    c = source_get();
  while (isspace(c));
}◇
```

Fragment referenced in 59, 60a.
Uses: `extra_scraps` 63b, `fputs` 21, `isspace` 21, `scraps` 124a, `source_get` 118c.

⟨ *Global variable declarations* 63b ⟩ ≡

```
extern int extra_scraps;
◇
```

Fragment defined by 27, 28b, 29a, 42c, 47c, 63b, 117b, 127a, 152a, 192b.
Fragment referenced in 20.
Defines: `extra_scraps` 62d, 63ac, 75, 89bc, 99.

⟨ *Global variable definitions* 63c ⟩ ≡

```
int extra_scraps = 0;
 ◇
```

Fragment defined by 28ac, 29b, 42d, 47d, 63c, 117c, 127b, 152b.
Fragment referenced in 24e.
Uses: `extra_scraps` 63b.

⟨ *Write in-text scrap* 63d ⟩ ≡

```
copy_scrap(tex_file, FALSE, NULL);
c = source_get();
◇
```

Fragment referenced in 57a.
Uses: `copy_scrap` 68, `FALSE` 22a, `source_get` 118c.

Formatting Cross References

⟨ *Begin the cross-reference environment* 64a ⟩ ≡

```
{
  fputs("\\vspace{-1.5ex}\n", tex_file);
  fputs("\\footnotesize\n", tex_file);
  fputs("\\begin{list}{}{\\setlength{\\itemsep}{-\\parsep}",
    tex_file);
  fputs("\\setlength{\\itemindent}{-\\leftmargin}}\n", tex_file);}
◇
```

Fragment referenced in 59, 60a.
Uses: `fputs` 21.

There may (unusually) be nothing to output in the cross-reference section, so output an empty list item anyway to avoid having an empty list.

⟨ *Finish the cross-reference environment* 64b ⟩ ≡

```
{
  fputs("\n\\item{}", tex_file);
  fputs("\n\\end{list}\n", tex_file);
}◇
```

Fragment referenced in 59, 60a.
Uses: `fputs` 21.

⟨ *Write file defs* 64c ⟩ ≡

```
{
  if (name->defs) {
    if (name->defs->next) {
      fputs("\\item \\NWtxtFileDefBy\\ ", tex_file);
      print_scrap_numbers(tex_file, name->defs);
    }
  } else {
    fprintf(stderr,
            "would have crashed in 'Write file defs' for '%s'\n",
             name->spelling);
  }
}◇
```

Fragment referenced in 59.
Uses: `defs` 123c, `fprintf` 21, `fputs` 21, `next` 123b, `print_scrap_numbers` 65b, `stderr` 21.

⟨ *Write macro defs* 64d ⟩ ≡

```
{
  if (name->defs->next) {
    fputs("\\item \\NWtxtMacroDefBy\\ ", tex_file);
    print_scrap_numbers(tex_file, name->defs);
  }
}◇
```

Fragment referenced in 60a.
Uses: `defs` 123c, `fputs` 21, `next` 123b, `print_scrap_numbers` 65b.

⟨ *Write macro refs* 65a ⟩ ≡

```
{
  if (name->uses) {
    if (name->uses->next) {
      fputs("\\item \\NWtxtMacroRefIn\\ ", tex_file);
      print_scrap_numbers(tex_file, name->uses);
    }
    else {
      fputs("\\item \\NWtxtMacroRefIn\\ ", tex_file);
      fputs("\\NWlink{weft", tex_file);
      write_single_scrap_ref(tex_file, name->uses->scrap);
      fputs("}{", tex_file);
      write_single_scrap_ref(tex_file, name->uses->scrap);
      fputs("}", tex_file);
      fputs(".\n", tex_file);
    }
  }
  else {
    fputs("\\item {\\NWtxtMacroNoRef}.\n", tex_file);
    fprintf(stderr, "%s: <%s> never referenced.\n",
            command_name, name->spelling);
  }
}◇
```

Fragment referenced in 60a.
Uses: `command_name` 29a, `fprintf` 21, `fputs` 21, `next` 123b, `print_scrap_numbers` 65b, `stderr` 21, `uses` 123c, `write_single_scrap_ref` 125b.

"latex.c" 65b≡

```
static void print_scrap_numbers(FILE *tex_file, Scrap_Node *scraps)
{
  int page;
  fputs("\\NWlink{weft", tex_file);
  write_scrap_ref(tex_file, scraps->scrap, -1, &page);
  fputs("}{", tex_file);
  write_scrap_ref(tex_file, scraps->scrap, TRUE, &page);
  fputs("}", tex_file);
  scraps = scraps->next;
  while (scraps) {
    fputs("\\NWlink{weft", tex_file);
    write_scrap_ref(tex_file, scraps->scrap, -1, &page);
    fputs("}{", tex_file);
    write_scrap_ref(tex_file, scraps->scrap, FALSE, &page);
    scraps = scraps->next;
    fputs("}", tex_file);
  }
  fputs(".\n", tex_file);
}
◇
```

File defined by 22e, 55ab, 61a, 65b, 66ab, 67bc, 68, 69a, 75, 76b, 78ab, 81b, 82a.
Defines: `print_scrap_numbers` 55a, 64cd, 65a, 77a, 80b.
Uses: `FALSE` 22a, `FILE` 21, `fputs` 21, `next` 123b, `page` 123c, `scraps` 124a, `Scrap_Node` 151a, `TRUE` 22a, `write_scrap_ref` 125a.

Formatting a Scrap

We add a `\mbox{}` at the beginning of each line to avoid problems with older versions of TEX. This is the only place we really care whether a scrap is delimited with `@{...@}`, `@[...@]`, or `@(...@)`, and we base our output sequences on that.

We have an array `delimit_scrap` where we store our strings that perform the formatting for one line of a scrap. Upon initialisation we copy our strings from the source `orig_delimit_scrap` to it to have the strings writeable. We might change the strings later when we encounter the command to change the 'weft character'.

"latex.c" 66a≡

```
static char *orig_delimit_scrap[3][5] = {
  /* {} mode: begin, end, insert nw_char, prefix, suffix */
  { "\\verb@", "@", "@{\\tt @}\\verb@", "\\mbox{}", "\\\\" },
  /* [] mode: begin, end, insert nw_char, prefix, suffix */
  { "", "", "@", "", "" },
  /* () mode: begin, end, insert nw_char, prefix, suffix */
  { "$", "$", "@", "", "" },
};

static char *delimit_scrap[3][5];
◇
```

File defined by 22e, 55ab, 61a, 65b, 66ab, 67bc, 68, 69a, 75, 76b, 78ab, 81b, 82a.
Uses: `nw_char` 28bc.

The function `initialise_delimit_scrap_array` does the copying. If we want to have the listings package do the formatting we have to replace only two of those strings: the `verb` command has to be replaced by the package's `lstinline` command.

"latex.c" 66b≡

```
void initialise_delimit_scrap_array(void) {
  int i,j;
  for(i = 0; i < 3; i++) {
    for(j = 0; j < 5; j++) {
      if((delimit_scrap[i][j] = strdup(orig_delimit_scrap[i][j])) == NULL) {
        fprintf(stderr, "Not enough memory for string allocation\n");
        exit(EXIT_FAILURE);
      }
    }
  }

  /* replace verb by lstinline */
  if (listings_flag) {
    free(delimit_scrap[0][0]);
    if((delimit_scrap[0][0]=strdup("\\lstinline@")) == NULL) {
      fprintf(stderr, "Not enough memory for string allocation\n");
      exit(EXIT_FAILURE);
    }
    free(delimit_scrap[0][2]);
    if((delimit_scrap[0][2]=strdup("@{\\tt @}\\lstinline@")) == NULL) {
      fprintf(stderr, "Not enough memory for string allocation\n");
      exit(EXIT_FAILURE);
    }
  }
}
◇
```

File defined by 22e, 55ab, 61a, 65b, 66ab, 67bc, 68, 69a, 75, 76b, 78ab, 81b, 82a.
Uses: `exit` 21, `fprintf` 21, `stderr` 21.

⟨ *Function prototypes* 67a ⟩ ≡

```
void initialise_delimit_scrap_array(void);
◇
```

Fragment defined by 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c.
Fragment referenced in 20.

"latex.c" 67b≡

```
int scrap_type = 0;
◇
```

File defined by 22e, 55ab, 61a, 65b, 66ab, 67bc, 68, 69a, 75, 76b, 78ab, 81b, 82a.
Defines: `scrap_type` 48a, 68, 70a, 72ab, 73, 75.

"latex.c" 67c≡

```
static void write_literal(FILE * tex_file, char * p, int mode)
{
   fputs(delimit_scrap[mode][0], tex_file);
   while (*p!= '\000') {
     if (*p == nw_char)
       fputs(delimit_scrap[mode][2], tex_file);
     else
       fputc(*p, tex_file);
     p++;
   }
   fputs(delimit_scrap[mode][1], tex_file);
}
◇
```

File defined by 22e, 55ab, 61a, 65b, 66ab, 67bc, 68, 69a, 75, 76b, 78ab, 81b, 82a.
Defines: `write_literal` 55a, 73, 75.
Uses: `FILE` 21, `fputs` 21, `nw_char` 28bc.

"latex.c" 68≡

```
static void copy_scrap(FILE *file, int prefix, Name *name)
{
  int indent = 0;
  int c;
  char ** params = name->arg;
  if (source_last == '{') scrap_type = 0;
  if (source_last == '[') scrap_type = 1;
  if (source_last == '(') scrap_type = 2;
  c = source_get();
  if (prefix) fputs(delimit_scrap[scrap_type][3], file);
  fputs(delimit_scrap[scrap_type][0], file);
  while (1) {
    switch (c) {
      case '\n': fputs(delimit_scrap[scrap_type][1], file);
                 if (prefix) fputs(delimit_scrap[scrap_type][4], file);
                 fputs("\n", file);
                 if (prefix) fputs(delimit_scrap[scrap_type][3], file);
                 fputs(delimit_scrap[scrap_type][0], file);
                 indent = 0;
                 break;
      case '\t': ⟨ Expand tab into spaces 69c ⟩
                 break;
      default:
         if (c==nw_char)
           {
             ⟨ Check at-sequence for end-of-scrap 70a ⟩
             break;
           }
         putc(c, file);
                 indent++;
                 break;
    }
    c = source_get();
  }
}
◇
```

File defined by 22e, 55ab, 61a, 65b, 66ab, 67bc, 68, 69a, 75, 76b, 78ab, 81b, 82a.
Defines: `copy_scrap` 54a, 55a, 62d, 63d, `scrap_type` 48a, 67b, 70a, 72ab, 73, 75.
Uses: `FILE` 21, `fputs` 21, `Name` 151b, `nw_char` 28bc, `putc` 21, `source_get` 118c, `source_last` 118c.

When we encounter the command to change the 'weft character' we call this function. It updates the scrap formatting directives accordingly.

"latex.c" 69a≡

```
void update_delimit_scrap(void)
{
  /* {}-mode begin */
  if (listings_flag) {
    delimit_scrap[0][0][10] = nw_char;
  } else {
    delimit_scrap[0][0][5] = nw_char;
  }
  /* {}-mode end */
  delimit_scrap[0][1][0] = nw_char;
  /* {}-mode insert nw_char */

  delimit_scrap[0][2][0] = nw_char;
  delimit_scrap[0][2][6] = nw_char;

  if (listings_flag) {
    delimit_scrap[0][2][18] = nw_char;
  } else {
    delimit_scrap[0][2][13] = nw_char;
  }

  /* []-mode insert nw_char */
  delimit_scrap[1][2][0] = nw_char;

  /* ()-mode insert nw_char */
  delimit_scrap[2][2][0] = nw_char;
}
◇
```

File defined by 22e, 55ab, 61a, 65b, 66ab, 67bc, 68, 69a, 75, 76b, 78ab, 81b, 82a.
Defines: `update_delimit_scrap` 41, 57a, 69b, 86a.
Uses: `nw_char` 28bc.

⟨ *Function prototypes* 69b ⟩ ≡

```
void update_delimit_scrap(void);
◇
```

Fragment defined by 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c.
Fragment referenced in 20.
Uses: `update_delimit_scrap` 69a.

⟨ *Expand tab into spaces* 69c ⟩ ≡

```
{
  int delta = 8 - (indent % 8);
  indent += delta;
  while (delta > 0) {
    putc(' ', file);
    delta--;
  }
}◇
```

Fragment referenced in 68, 143d.
Uses: `putc` 21.

⟨ *Check at-sequence for end-of-scrap* 70a ⟩ ≡

```
{
  c = source_get();
  switch (c) {
    case 'c': ⟨ Show presence of a block comment 48a ⟩
              break;
    case 'x': ⟨ Copy label from source into (70b file ) 71b ⟩
              break;
    case 'v': ⟨ Copy version info into file 71a ⟩
    case 's':
              break;
    case '+':
    case '-':
    case '*':
    case '|': ⟨ Skip over index entries 71d ⟩
    case ',':
    case ')':
    case ']':
    case '}': fputs(delimit_scrap[scrap_type][1], file);
              return;
    case '<': ⟨ Format macro name 73 ⟩
              break;
    case '%': ⟨ Skip commented-out code 71e ⟩
              break;
    case '_': ⟨ Bold Keyword 72a ⟩
              break;
    case 't': ⟨ Italic "fragment title" 72b ⟩
              break;
    case 'f': ⟨ Italic "file name" 72b ⟩
              break;
    case '1': case '2': case '3':
    case '4': case '5': case '6':
    case '7': case '8': case '9':
              if (name == NULL
                  || name->arg[c - '1'] == NULL) {
                fputs(delimit_scrap[scrap_type][2], file);
                fputc(c,   file);
              }
              else {
                fputs(delimit_scrap[scrap_type][1], file);
                write_arg(file, name->arg[c - '1']);
                fputs(delimit_scrap[scrap_type][0], file);
              }
              break;
    default:
          if (c==nw_char)
            {
              fputs(delimit_scrap[scrap_type][2], file);
              break;
            }
          /* ignore these since pass1 will have warned about them */
              break;
  }
}◇
```

Fragment referenced in 68.

⟨ *Copy version info into file* 71a ⟩ ≡

```
fputs(version_string, file);
◇
```

Fragment referenced in 70a, 95a, 144a.
Uses: `fputs` 21, `version_string` 27.

⟨ *Copy label from source into* 71b ⟩ ≡

```
{
   ⟨ Get label from (71c source_get() ) 189c ⟩
   write_label(label_name, @1);
}◇
```

Fragment referenced in 57a, 70a, 86a, 95a.
Uses: `source_get` 118c.

There's no need to check for errors here, since we will have already pointed out any during the first pass.

⟨ *Skip over index entries* 71d ⟩ ≡

```
{
  do {
    do
      c = source_get();
    while (c != nw_char);
    c = source_get();
  } while (c != '}' && c != ']' && c != ')' );
}◇
```

Fragment referenced in 70a, 95a.
Uses: `nw_char` 28bc, `source_get` 118c.

⟨ *Skip commented-out code* 71e ⟩ ≡

```
{
        do
                c = source_get();
        while (c != '\n');
}◇
```

Fragment referenced in 70a, 95a, 130.
Uses: `source_get` 118c.

This scrap helps deal with bold keywords:

⟨ *Bold Keyword* 72a ⟩ ≡

```
{
  fputs(delimit_scrap[scrap_type][1],file);
  fprintf(file, "\\hbox{\\sffamily\\bfseries ");
  c = source_get();
  do {
      fputc(c, file);
      c = source_get();
  } while (c != nw_char);
  c = source_get();
  fprintf(file, "}");
  fputs(delimit_scrap[scrap_type][0], file);
}◇
```

Fragment referenced in 70a.
Uses: `fprintf` 21, `fputs` 21, `nw_char` 28bc, `scrap_type` 67b, 68, `source_get` 118c.

⟨ *Italic "whatever"* 72b ⟩ ≡

```
{
  fputs(delimit_scrap[scrap_type][1],file);
  fprintf(file, "\\hbox{\\sffamily\\slshape whatever}");
  fputs(delimit_scrap[scrap_type][0], file);
}◇
```

Fragment referenced in 70a.
Uses: `fprintf` 21, `fputs` 21, `scrap_type` 67b, 68.

⟨ *Format macro name* 73 ⟩ ≡

```
{
  Arglist *args = collect_scrap_name(-1);
  Name *name = args->name;
  char * p = name->spelling;
  Arglist *q = args->args;
  int narg = 0;

  fputs(delimit_scrap[scrap_type][1],file);
  if (prefix)
    fputs("\\hbox{", file);
  fputs("$\\langle\\,${\\itshape ", file);
  while (*p != '\000') {
    if (*p == ARG_CHR) {
      if (q == NULL) {
         write_literal(file, name->arg[narg], scrap_type);
      }
      else {
        write_ArglistElement(file, q, params);
        q = q->next;
      }
      p++;
      narg++;
    }
    else
       fputc(*p++, file);
  }
  fputs("}\\nobreak\\ ", file);
  if (scrap_name_has_parameters) {
    ⟨ Format macro parameters 54a ⟩
  }
  fprintf(file, "{\\footnotesize ");
  if (name->defs)
    ⟨ Write abbreviated definition list 74 ⟩
  else {
    putc('?', file);
    fprintf(stderr, "%s: never defined <%s>\n",
            command_name, name->spelling);
  }
  fputs("}$\\,\\rangle$", file);
  if (prefix)
     fputs("}", file);
  fputs(delimit_scrap[scrap_type][0], file);
}◇
```

Fragment referenced in 70a.

Uses: `Arglist` 170b, `ARG_CHR` 168c, `collect_scrap_name` 171, `command_name` 29a, `defs` 123c, `fprintf` 21, `fputs` 21, `Name` 151b, `next` 123b, `putc` 21, `scrap_type` 67b, 68, `stderr` 21, `write_ArglistElement` 75, `write_literal` 67c.

⟨ *Write abbreviated definition list* 74 ⟩ ≡

```
{
  Scrap_Node *p = name->defs;
  fputs("\\NWlink{weft", file);
  write_single_scrap_ref(file, p->scrap);
  fputs("}{", file);
  write_single_scrap_ref(file, p->scrap);
  fputs("}", file);
  p = p->next;
  if (p)
    fputs(", \\ldots\\ ", file);
}◇
```

Fragment referenced in 73, 75.
Uses: `defs` 123c, `fputs` 21, `next` 123b, `Scrap_Node` 151a, `write_single_scrap_ref` 125b.

"latex.c" 75≡

```
static void
write_ArglistElement(FILE * file, Arglist * args, char ** params)
{
  Name *name = args->name;
  Arglist *q = args->args;

  if (name == NULL) {
    char * p = (char*)q;

    if (p[0] == ARG_CHR) {
       write_arg(file, params[p[1] - '1']);
    } else {
       write_literal(file, (char *)q, 0);
    }
  } else if (name == (Name *)1) {
    Scrap_Node * qq = (Scrap_Node *)q;
    qq->quoted = TRUE;
    fputs(delimit_scrap[scrap_type][0], file);
    write_scraps(file, "", qq,
                 -1, "", 0, 0, 0, 0, 0,
                 NULL, params, 0, "");
    fputs(delimit_scrap[scrap_type][1], file);
    extra_scraps++;
    qq->quoted = FALSE;
  } else {
    char * p = name->spelling;
    fputs("$\\langle\\,${\\itshape ", file);
      while (*p != '\000') {
      if (*p == ARG_CHR) {
        write_ArglistElement(file, q, params);
        q = q->next;
        p++;
      }
      else
         fputc(*p++, file);
    }
    fputs("}\\nobreak\\ ", file);
    if (scrap_name_has_parameters) {
      int c;

      ⟨ Format macro parameters 54a ⟩
    }
    fprintf(file, "{\\footnotesize ");
    if (name->defs)
      ⟨ Write abbreviated definition list 74 ⟩
    else {
      putc('?', file);
      fprintf(stderr, "%s: never defined <%s>\n",
              command_name, name->spelling);
    }
    fputs("}$\\,\\rangle$", file);
  }
}
◇
```

File defined by 22e, 55ab, 61a, 65b, 66ab, 67bc, 68, 69a, 75, 76b, 78ab, 81b, 82a.
Defines: `write_ArglistElement` 55a, 73.
Uses: `Arglist` 170b, `ARG_CHR` 168c, `command_name` 29a, `defs` 123c, `extra_scraps` 63b, `FALSE` 22a, `FILE` 21, `fprintf` 21, `fputs` 21, `Name` 151b, `next` 123b, `putc` 21, `Scrap_Node` 151a, `scrap_type` 67b, 68, `stderr` 21, `TRUE` 22a, `write_arg` 61a, `write_literal` 67c, `write_scraps` 139a.

2.4.2 Generating the Indices

⟨ *Write index of file names* 76a ⟩ ≡

```
{
  if (file_names) {
    fputs("\n{\\small\\begin{list}{}{\\setlength{\\itemsep}{-\\parsep}",
          tex_file);
    fputs("\\setlength{\\itemindent}{-\\leftmargin}}\n", tex_file);
    format_file_entry(file_names, tex_file);
    fputs("\\end{list}}", tex_file);
  }
  c = source_get();
}◇
```

Fragment referenced in 57a.
Uses: `file_names` 152a, `format_file_entry` 76b, `fputs` 21, `source_get` 118c.

"latex.c" 76b≡

```
static void format_file_entry(Name *name, FILE *tex_file)
{
  while (name) {
    format_file_entry(name->llink, tex_file);
    ⟨ Format a file index entry 76c ⟩
    name = name->rlink;
  }
}
◇
```

File defined by 22e, 55ab, 61a, 65b, 66ab, 67bc, 68, 69a, 75, 76b, 78ab, 81b, 82a.
Defines: `format_file_entry` 55a, 76a.
Uses: `FILE` 21, `Name` 151b.

⟨ *Format a file index entry* 76c ⟩ ≡

```
fputs("\\item ", tex_file);
fprintf(tex_file, "\\verb%c\"%s\"%c ", nw_char, name->spelling, nw_char);
⟨ Write file's defining scrap numbers 77a ⟩
putc('\n', tex_file);◇
```

Fragment referenced in 76b.
Uses: `fprintf` 21, `fputs` 21, `nw_char` 28bc, `putc` 21.

⟨ *Write file's defining scrap numbers* 77a ⟩ ≡

```
{
  Scrap_Node *p = name->defs;
  fputs("{\\footnotesize {\\NWtxtDefBy}", tex_file);
  if (p->next) {
    /* fputs("s ", tex_file); */
      putc(' ', tex_file);
    print_scrap_numbers(tex_file, p);
  }
  else {
    putc(' ', tex_file);
    fputs("\\NWlink{weft", tex_file);
    write_single_scrap_ref(tex_file, p->scrap);
    fputs("}{", tex_file);
    write_single_scrap_ref(tex_file, p->scrap);
    fputs("}", tex_file);
    putc('.', tex_file);
  }
  putc('}', tex_file);
}◇
```

Fragment referenced in 76c.
Uses: `defs` 123c, `fputs` 21, `next` 123b, `print_scrap_numbers` 65b, `putc` 21, `Scrap_Node` 151a, `write_single_scrap_ref` 125b.

⟨ *Write index of macro names* 77b ⟩ ≡

```
{
  unsigned char sector = current_sector;
  int c = source_get();
  if (c == '+')
     sector = 0;
  else
     source_ungetc(&c);
  if (has_sector(macro_names, sector)) {
    fputs("\n{\\small\\begin{list}{}{\\setlength{\\itemsep}{-\\parsep}",
          tex_file);
    fputs("\\setlength{\\itemindent}{-\\leftmargin}}\n", tex_file);
    format_entry(macro_names, tex_file, sector);
    fputs("\\end{list}}", tex_file);
  } else {
    fputs("None.\n", tex_file);
  }
}
c = source_get();
◇
```

Fragment referenced in 57a.
Uses: `current_sector` 42c, `format_entry` 78b, `fputs` 21, `has_sector` 81b, `macro_names` 152a, `sector` 123c, `source_get` 118c, `source_ungetc` 119b.

"latex.c" 78a≡

```
static int load_entry(Name * name, Name ** nms, int n)
{
   while (name) {
      n = load_entry(name->llink, nms, n);
      nms[n++] = name;
      name = name->rlink;
   }
   return n;
}
◇
```

File defined by 22e, 55ab, 61a, 65b, 66ab, 67bc, 68, 69a, 75, 76b, 78ab, 81b, 82a.
Defines: `load_entry` 78b.
Uses: `Name` 151b.

"latex.c" 78b≡

```
static void format_entry(Name *name, FILE *tex_file, unsigned char sector)
{
  Name ** nms = malloc(num_scraps()*sizeof(Name *));
  int n = load_entry(name, nms, 0);
  int i;

  ⟨ Sort nms of size n for ⟨ Rob's ordering 78c ⟩ 79a ⟩
  for (i = 0; i < n; i++)
  {
     Name * name = nms[i];

     ⟨ Format an index entry 79b ⟩
  }
}
◇
```

File defined by 22e, 55ab, 61a, 65b, 66ab, 67bc, 68, 69a, 75, 76b, 78ab, 81b, 82a.
Defines: `format_entry` 55a, 77b.
Uses: `FILE` 21, `load_entry` 78a, `malloc` 21, `Name` 151b, `num_scraps` 124a, `sector` 123c.

⟨ *Rob's ordering* 78c ⟩ ≡

```
robs_strcmp(ki->spelling, kj->spelling) < 0◇
```

Fragment referenced in 78b, 102a.
Uses: `robs_strcmp` 156a.

⟨ *Sort key of size n for ordering* 79a ⟩ ≡

```
int j;
for (j = 1; j < n; j++)
{
   int i = j - 1;
   Name * kj = key[j];

   do
   {
      Name * ki = key[i];

      if (ordering)
         break;
      key[i + 1] = ki;
      i -= 1;
   } while (i >= 0);
   key[i + 1] = kj;
}
◇
```

Fragment referenced in 78b, 102a.
Uses: `Name` 151b.

⟨ *Format an index entry* 79b ⟩ ≡

```
if (name->sector == sector){
  fputs("\\item ", tex_file);
  fputs("$\\langle\\,$", tex_file);
  ⟨ Write the macro's name 60b ⟩
  fputs("\\nobreak\\ {\\footnotesize ", tex_file);
  ⟨ Write defining scrap numbers 80a ⟩
  fputs("}$\\,\\rangle$ ", tex_file);
  ⟨ Write referencing scrap numbers 80b ⟩
  putc('\n', tex_file);
}◇
```

Fragment referenced in 78b.
Uses: `fputs` 21, `putc` 21, `sector` 123c.

⟨ *Write defining scrap numbers* 80a ⟩ ≡

```
{
  Scrap_Node *p = name->defs;
  if (p) {
    int page;
    fputs("\\NWlink{weft", tex_file);
    write_scrap_ref(tex_file, p->scrap, -1, &page);
    fputs("}{", tex_file);
    write_scrap_ref(tex_file, p->scrap, TRUE, &page);
    fputs("}", tex_file);
    p = p->next;
    while (p) {
      fputs("\\NWlink{weft", tex_file);
      write_scrap_ref(tex_file, p->scrap, -1, &page);
      fputs("}{", tex_file);
      write_scrap_ref(tex_file, p->scrap, FALSE, &page);
      fputs("}", tex_file);
      p = p->next;
    }
  }
  else
    putc('?', tex_file);
}◇
```

Fragment referenced in 79b.
Uses: `defs` 123c, `FALSE` 22a, `fputs` 21, `next` 123b, `page` 123c, `putc` 21, `Scrap_Node` 151a, `TRUE` 22a, `write_scrap_ref` 125a.

⟨ *Write referencing scrap numbers* 80b ⟩ ≡

```
{
  Scrap_Node *p = name->uses;
  fputs("{\\footnotesize ", tex_file);
  if (p) {
    fputs("{\\NWtxtRefIn}", tex_file);
    if (p->next) {
      /* fputs("s ", tex_file); */
      putc(' ', tex_file);
      print_scrap_numbers(tex_file, p);
    }
    else {
      putc(' ', tex_file);
      fputs("\\NWlink{weft", tex_file);
      write_single_scrap_ref(tex_file, p->scrap);
      fputs("}{", tex_file);
      write_single_scrap_ref(tex_file, p->scrap);
      fputs("}", tex_file);
      putc('.', tex_file);
    }
  }
  else
    fputs("{\\NWtxtNoRef}.", tex_file);
  putc('}', tex_file);
}◇
```

Fragment referenced in 79b.
Uses: `fputs` 21, `next` 123b, `print_scrap_numbers` 65b, `putc` 21, `Scrap_Node` 151a, `uses` 123c, `write_single_scrap_ref` 125b.

⟨ *Function prototypes* 81a ⟩ ≡

```
extern int has_sector(Name *, unsigned char);
◇
```

Fragment defined by 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c.
Fragment referenced in 20.
Uses: `has_sector` 81b, `Name` 151b.

"latex.c" 81b≡

```
int has_sector(Name * name, unsigned char sector)
{
  while(name) {
    if (name->sector == sector)
       return TRUE;
    if (has_sector(name->llink, sector))
       return TRUE;
     name = name->rlink;
   }
   return FALSE;
}
◇
```

File defined by 22e, 55ab, 61a, 65b, 66ab, 67bc, 68, 69a, 75, 76b, 78ab, 81b, 82a.
Defines: `has_sector` 77b, 81ac, 101a, 104a.
Uses: `FALSE` 22a, `Name` 151b, `sector` 123c, `TRUE` 22a.

⟨ *Write index of user-specified names* 81c ⟩ ≡

```
{
    unsigned char sector = current_sector;
    c = source_get();
    if (c == '+') {
       sector = 0;
       c = source_get();
    }
    if (has_sector(user_names, sector)) {
      fputs("\n{\\small\\begin{list}{}{\\setlength{\\itemsep}{-\\parsep}",
            tex_file);
      fputs("\\setlength{\\itemindent}{-\\leftmargin}}\n", tex_file);
      format_user_entry(user_names, tex_file, sector);
      fputs("\\end{list}}", tex_file);
    }
}◇
```

Fragment referenced in 57a.
Uses: `current_sector` 42c, `format_user_entry` 82a, `fputs` 21, `has_sector` 81b, `sector` 123c, `source_get` 118c, `user_names` 152a.

"latex.c" 82a≡

```
static void format_user_entry(Name *name, FILE *tex_file, unsigned char sector)
{
  while (name) {
    format_user_entry(name->llink, tex_file, sector);
    ⟨ Format a user index entry 82b ⟩
    name = name->rlink;
  }
}
◇
```

File defined by 22e, 55ab, 61a, 65b, 66ab, 67bc, 68, 69a, 75, 76b, 78ab, 81b, 82a.
Defines: `format_user_entry` 55a, 81c.
Uses: `FILE` 21, `Name` 151b, `sector` 123c.

⟨ *Format a user index entry* 82b ⟩ ≡

```
if (name->sector == sector){
  Scrap_Node *uses = name->uses;
  if ( uses || dangling_flag ) {
    int page;
    Scrap_Node *defs = name->defs;
    fprintf(tex_file, "\\item \\verb%c%s%c: ", nw_char,name->spelling,nw_char);
    if (!uses) {
        fputs("(\\underline{", tex_file);
        fputs("\\NWlink{weft", tex_file);
        write_single_scrap_ref(tex_file, defs->scrap);
        fputs("}{", tex_file);
        write_single_scrap_ref(tex_file, defs->scrap);
        fputs("})}", tex_file);
        page = -2;
        defs = defs->next;
    }
    else
      if (!defs || uses->scrap < defs->scrap) {
      fputs("\\NWlink{weft", tex_file);
      write_scrap_ref(tex_file, uses->scrap, -1, &page);
      fputs("}{", tex_file);
      write_scrap_ref(tex_file, uses->scrap, TRUE, &page);
      fputs("}", tex_file);
      uses = uses->next;
    }
    else {
      if (defs->scrap == uses->scrap)
        uses = uses->next;
      fputs("\\underline{", tex_file);

      fputs("\\NWlink{weft", tex_file);
      write_single_scrap_ref(tex_file, defs->scrap);
      fputs("}{", tex_file);
      write_single_scrap_ref(tex_file, defs->scrap);
      fputs("}}", tex_file);
      page = -2;
      defs = defs->next;
    }
    while (uses || defs) {
      if (uses && (!defs || uses->scrap < defs->scrap)) {
        fputs("\\NWlink{weft", tex_file);
        write_scrap_ref(tex_file, uses->scrap, -1, &page);
```

```
          fputs("}{", tex_file);
          write_scrap_ref(tex_file, uses->scrap, FALSE, &page);
          fputs("}", tex_file);
          uses = uses->next;
        }
        else {
          if (uses && defs->scrap == uses->scrap)
            uses = uses->next;
          fputs(", \\underline{", tex_file);

          fputs("\\NWlink{weft", tex_file);
          write_single_scrap_ref(tex_file, defs->scrap);
          fputs("}{", tex_file);
          write_single_scrap_ref(tex_file, defs->scrap);
          fputs("}", tex_file);

          putc('}', tex_file);
          page = -2;
          defs = defs->next;
        }
      }
      fputs(".\n", tex_file);
    }
  }◇
```

Fragment referenced in 82a.
Uses: `dangling_flag` 27, `defs` 123c, `FALSE` 22a, `fprintf` 21, `fputs` 21, `next` 123b, `nw_char` 28bc, `page` 123c, `putc` 21, `Scrap_Node` 151a, `sector` 123c, `TRUE` 22a, `uses` 123c, `write_scrap_ref` 125a, `write_single_scrap_ref` 125b.

2.5 Writing the Markdown File

This section implements the Markdown weave output, activated by the `-w md` command-line flag. It is structurally parallel to the LaTeX output in Section 2.4, but emits Markdown with embedded HTML for scraps (using `<pre><code>` blocks so that fragment references inside code can be clickable links).

⟨ *Function prototypes* 83 ⟩ ≡

```
extern void write_md(char *file_name, char *md_name, unsigned char sector);
◇
```

Fragment defined by 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c.
Fragment referenced in 20.
Uses: `file_name` 123c, `sector` 123c, `write_md` 84b.

We need a few local function declarations before we get into the body of `write_md`.

"markdown.c" 84a≡

```
static void md_copy_scrap(FILE *file, int prefix, Name *name);
static void md_print_scrap_numbers(FILE *md_file, Scrap_Node *scraps);
static void md_format_entry(Name *name, FILE *md_file, unsigned char sector);
static void md_format_file_entry(Name *name, FILE *md_file);
static void md_format_user_entry(Name *name, FILE *md_file, unsigned char sector);
static void md_write_arg(FILE *md_file, char *p);
static void md_write_literal(FILE *md_file, char *p);
static void md_write_ArglistElement(FILE *file, Arglist *args, char **params);
static void md_format_uses_refs(FILE *md_file, int scrap);
static void md_format_defs_refs(FILE *md_file, int scrap);
◇
```

File defined by 23a, 84ab, 89a, 92, 93a, 98, 99, 100b, 101b, 102a, 104b, 106a, 107b.
Uses: `Arglist` 170b, `FILE` 21, `md_copy_scrap` 93a, `md_format_defs_refs` 107b, `md_format_entry` 102a, `md_format_file_entry` 100b, `md_format_user_entry` 104b, `md_format_uses_refs` 106a, `md_print_scrap_numbers` 92, `md_write_arg` 89a, `md_write_ArglistElement` 99, `md_write_literal` 98, `Name` 151b, `scraps` 124a, `Scrap_Node` 151a, `sector` 123c.

The routine `write_md` takes two file names as parameters: the name of the web source file and the name of the `.md` output file.

"markdown.c" 84b≡

```
void write_md(char *file_name, char *md_name, unsigned char sector)
{
  (void)sector;
  FILE *md_file = fopen(md_name, "w");
  if (md_file) {
    if (verbose_flag)
      fprintf(stderr, "writing %s\n", md_name);
    source_open(file_name);
    ⟨ Copy source_file into md_file 85a ⟩
    fclose(md_file);
  }
  else
    fprintf(stderr, "%s: can't open %s\n", command_name, md_name);
}
◇
```

File defined by 23a, 84ab, 89a, 92, 93a, 98, 99, 100b, 101b, 102a, 104b, 106a, 107b.
Defines: `write_md` 37, 83.
Uses: `command_name` 29a, `fclose` 21, `FILE` 21, `file_name` 123c, `fopen` 21, `fprintf` 21, `sector` 123c, `source_open` 123a, `stderr` 21, `verbose_flag` 27.

We make our second pass through the source web, copying characters straight into the `.md` file. We watch for `@` characters which signal a command sequence. We also detect and discard cosmetic fence lines (lines that are exactly ‘‘‘ optionally followed by a language tag).

⟨ *Copy* `source_file` *into* `md_file` 85a ⟩ ≡

```
{
  int c = source_get();
  int at_line_start = TRUE;
  while (c != EOF) {
    if (c == nw_char)
      {
      ⟨ Interpret md at-sequence 86a ⟩
      }
    else if (c == '`' && at_line_start) {
      ⟨ Check for cosmetic fence line 85b ⟩
    }
    else {
      if (c == '\n') at_line_start = TRUE;
      else at_line_start = FALSE;
      putc(c, md_file);
      c = source_get();
    }
  }
}◇
```

Fragment referenced in 84b.
Uses: `FALSE` 22a, `nw_char` 28bc, `putc` 21, `source_get` 118c, `TRUE` 22a.

Cosmetic fence lines are lines that start with ```. We peek ahead to see if the line matches; if so, we skip to end of line. Otherwise we output what we already consumed and continue.

⟨ *Check for cosmetic fence line* 85b ⟩ ≡

```
{
  int c2 = source_get();
  int c3 = (c2 == '`') ? source_get() : 0;
  if (c2 == '`' && c3 == '`') {
    /* Skip rest of fence line */
    int fc = source_get();
    while (fc != '\n' && fc != EOF)
      fc = source_get();
    c = source_get();
    at_line_start = TRUE;
  } else {
    /* Not a fence, output what we consumed */
    putc('`', md_file);
    if (c2 == '`') {
      putc('`', md_file);
      c = c3;
    } else {
      c = c2;
    }
    at_line_start = FALSE;
  }
}◇
```

Fragment referenced in 85a.
Uses: `FALSE` 22a, `putc` 21, `source_get` 118c, `TRUE` 22a.

⟨ *Interpret md at-sequence* 86a ⟩ ≡

```
{
  int big_definition = FALSE;
  c = source_get();
  switch (c) {
    case 'r':
          c = source_get();
          nw_char = c;
          update_delimit_scrap();
          break;
    case 'O': big_definition = TRUE;
    case 'o': ⟨ Write md output file definition 88a ⟩
              break;
    case 'Q':
    case 'D': big_definition = TRUE;
    case 'q':
    case 'd': ⟨ Write md macro definition 88b ⟩
              break;
    case 's':
              ⟨ Step to next sector 42a ⟩
              break;
    case 'S':
              ⟨ Close the current sector 42b ⟩
              break;
    case '{':
    case '[':
    case '(': ⟨ Write md in-text scrap 90a ⟩
              break;
    case '<': ⟨ Expand md macro into md_file 87b ⟩
              break;
    case 'x': ⟨ Copy label from source into (86b md_file ) 71b ⟩
              c = source_get();
              break;
    case 'c': /* Block comments: no special handling in Markdown */
              c = source_get();
              break;
    case 'f': ⟨ Write md index of file names 100a ⟩
              break;
    case 'm': ⟨ Write md index of macro names 101a ⟩
              break;
    case 'u': ⟨ Write md index of user-specified names 104a ⟩
              break;
    case 'v': ⟨ Copy version info into md file 87a ⟩
              break;
    case 'l':
    case 'L': ⟨ Skip at-L language definition line 40 ⟩
              break;
    case 'W': ⟨ Skip at-L language definition line 40 ⟩
              break;
    default:
          if (c==nw_char)
            putc(c, md_file);
          c = source_get();
              break;
  }
}◇
```

Fragment referenced in 85a.

⟨ *Copy version info into md file* 87a ⟩ ≡

```
fputs(version_string, md_file);
c = source_get();
◇
```

Fragment referenced in 86a.
Uses: `fputs` 21, `source_get` 118c, `version_string` 27.

⟨ *Expand md macro into file* 87b ⟩ ≡

```
{
   Parameters local_parameters = 0;
   int changed;
   char indent_chars[MAX_INDENT];
   Arglist *a;
   Name *name;
   Arglist * args;
   a = collect_scrap_name(0);
   name = a->name;
   args = instance(a->args, NULL, NULL, &changed);
   name->mark = TRUE;
   write_scraps(file, md_name, name->defs, 0, indent_chars, 0, 0, 1, 0, 0,
         args, name->arg, local_parameters, md_name);
   name->mark = FALSE;
   c = source_get();
}
◇
```

Fragment referenced in 86a.
Uses: `Arglist` 170b, `collect_scrap_name` 171, `defs` 123c, `FALSE` 22a, `instance` 137a, `MAX_INDENT` 114c, `Name` 151b, `Parameters` 50c, `source_get` 118c, `TRUE` 22a, `write_scraps` 139a.

2.5.1 Formatting Definitions in Markdown

File and fragment definitions use HTML anchors and formatting. A file definition header looks like:

```
<a id="weft1"></a>
**"filename"** **1** =
<pre><code>...code...</code></pre>
```

A fragment definition header looks like:

```
<a id="weft1"></a>
**&lt;*Fragment Name* 1&gt;** =
<pre><code>...code...</code></pre>
```

⟨ *Write md output file definition* 88a ⟩ ≡

```
{
  Name *name = collect_file_name();
  (void)big_definition;
  fputs("\n<a id=\"weft", md_file);
  write_single_scrap_ref(md_file, scraps);
  fputs("\"></a>\n", md_file);
  fprintf(md_file, "**\"%s\"**", name->spelling);
  fputs(" **", md_file);
  write_single_scrap_ref(md_file, scraps);
  fputs("** =\n", md_file);
  ⟨ Fill in the md scrap 89b ⟩
  ⟨ Write md file cross-refs 90b ⟩
  ⟨ Finish md scrap 89c ⟩
}◇
```

Fragment referenced in 86a.
Uses: `collect_file_name` 163, `fprintf` 21, `fputs` 21, `Name` 151b, `scraps` 124a, `write_single_scrap_ref` 125b.

⟨ *Write md macro definition* 88b ⟩ ≡

```
{
  Name *name = collect_macro_name();
  (void)big_definition;
  fputs("\n<a id=\"weft", md_file);
  write_single_scrap_ref(md_file, scraps);
  fputs("\"></a>\n", md_file);
  fputs("**&lt;*", md_file);
  ⟨ Write the md macro's name 88c ⟩
  fputs("* ", md_file);
  write_single_scrap_ref(md_file, scraps);
  fputs("&gt;** =\n", md_file);
  ⟨ Fill in the md scrap 89b ⟩
  ⟨ Write md macro cross-refs 90d ⟩
  ⟨ Finish md scrap 89c ⟩
}◇
```

Fragment referenced in 86a.
Uses: `collect_macro_name` 167, `fputs` 21, `Name` 151b, `scraps` 124a, `write_single_scrap_ref` 125b.

⟨ *Write the md macro's name* 88c ⟩ ≡

```
{
  char * p = name->spelling;
  int i = 0;

  while (*p != '\000') {
    if (*p == ARG_CHR) {
      md_write_arg(md_file, name->arg[i++]);
      p++;
    }
    else
       fputc(*p++, md_file);
  }
}◇
```

Fragment referenced in 88b, 102b.
Uses: `ARG_CHR` 168c, `md_write_arg` 89a.

"markdown.c" 89a≡

```
static void md_write_arg(FILE * md_file, char * p)
{
   fputs("<i>", md_file);
   while (*p)
   {
      switch (*p)
      {
      case '<':
         fputs("&lt;", md_file);
         break;
      case '>':
         fputs("&gt;", md_file);
         break;
      case '&':
         fputs("&amp;", md_file);
         break;
      default:
         fputc(*p, md_file);
         break;
      }
      p++;
   }
   fputs("</i>", md_file);
}
◇
```

File defined by 23a, 84ab, 89a, 92, 93a, 98, 99, 100b, 101b, 102a, 104b, 106a, 107b.
Defines: `md_write_arg` 84a, 88c, 95a, 99.
Uses: `FILE` 21, `fputs` 21.

⟨ *Fill in the md scrap* 89b ⟩ ≡

```
{
  if (name->lang[0] != '\0')
    fprintf(md_file, "<pre><code class=\"language-%s\">", name->lang);
  else
    fputs("<pre><code>", md_file);
  extra_scraps = 0;
  md_copy_scrap(md_file, TRUE, name);
  fputs("</code></pre>\n", md_file);
}◇
```

Fragment referenced in 88ab.
Uses: `extra_scraps` 63b, `fprintf` 21, `fputs` 21, `md_copy_scrap` 93a, `TRUE` 22a.

⟨ *Finish md scrap* 89c ⟩ ≡

```
{
  scraps += extra_scraps;
  do
    c = source_get();
  while (isspace(c));
  at_line_start = TRUE;
}◇
```

Fragment referenced in 88ab.
Uses: `extra_scraps` 63b, `isspace` 21, `scraps` 124a, `source_get` 118c, `TRUE` 22a.

⟨ *Write md in-text scrap* 90a ⟩ ≡

```
md_copy_scrap(md_file, FALSE, NULL);
c = source_get();
◇
```

Fragment referenced in 86a.
Uses: `FALSE` 22a, `md_copy_scrap` 93a, `source_get` 118c.

Cross References in Markdown

Cross-references appear as blockquotes after each scrap.

⟨ *Write md file cross-refs* 90b ⟩ ≡

```
{
  if ( scrap_flag ) {
    ⟨ Write md file defs 90c ⟩
  }
  md_format_defs_refs(md_file, scraps);
  md_format_uses_refs(md_file, scraps++);
}◇
```

Fragment referenced in 88a.
Uses: `md_format_defs_refs` 107b, `md_format_uses_refs` 106a, `scraps` 124a, `scrap_flag` 27.

⟨ *Write md file defs* 90c ⟩ ≡

```
{
  if (name->defs) {
    if (name->defs->next) {
      fputs("> *File defined by* ", md_file);
      md_print_scrap_numbers(md_file, name->defs);
    }
  } else {
    fprintf(stderr,
            "would have crashed in 'Write md file defs' for '%s'\n",
             name->spelling);
  }
}◇
```

Fragment referenced in 90b.
Uses: `defs` 123c, `fprintf` 21, `fputs` 21, `md_print_scrap_numbers` 92, `next` 123b, `stderr` 21.

⟨ *Write md macro cross-refs* 90d ⟩ ≡

```
{
  ⟨ Write md macro defs 91a ⟩
  ⟨ Write md macro refs 91b ⟩
  md_format_defs_refs(md_file, scraps);
  md_format_uses_refs(md_file, scraps++);
}◇
```

Fragment referenced in 88b.
Uses: `md_format_defs_refs` 107b, `md_format_uses_refs` 106a, `scraps` 124a.

⟨ *Write md macro defs* 91a ⟩ ≡

```
{
  if (name->defs->next) {
    fputs("> *Fragment defined by* ", md_file);
    md_print_scrap_numbers(md_file, name->defs);
  }
}◇
```

Fragment referenced in 90d.
Uses: `defs` 123c, `fputs` 21, `md_print_scrap_numbers` 92, `next` 123b.

⟨ *Write md macro refs* 91b ⟩ ≡

```
{
  if (name->uses) {
    if (name->uses->next) {
      fputs("> *Fragment referenced in* ", md_file);
      md_print_scrap_numbers(md_file, name->uses);
    }
    else {
      fputs("> *Fragment referenced in* ", md_file);
      fputs("<a href=\"#weft", md_file);
      write_single_scrap_ref(md_file, name->uses->scrap);
      fputs("\">", md_file);
      write_single_scrap_ref(md_file, name->uses->scrap);
      fputs("</a>", md_file);
      fputs(".\n", md_file);
    }
  }
  else {
    fputs("> *Fragment never referenced*.\n", md_file);
    fprintf(stderr, "%s: <%s> never referenced.\n",
            command_name, name->spelling);
  }
}◇
```

Fragment referenced in 90d.
Uses: `command_name` 29a, `fprintf` 21, `fputs` 21, `md_print_scrap_numbers` 92, `next` 123b, `stderr` 21, `uses` 123c, `write_single_scrap_ref` 125b.

"markdown.c" 92≡

```
static void md_print_scrap_numbers(FILE *md_file, Scrap_Node *scraps)
{
  int page;
  fputs("<a href=\"#weft", md_file);
  write_scrap_ref(md_file, scraps->scrap, -1, &page);
  fputs("\">", md_file);
  write_scrap_ref(md_file, scraps->scrap, TRUE, &page);
  fputs("</a>", md_file);
  scraps = scraps->next;
  while (scraps) {
    fputs("<a href=\"#weft", md_file);
    write_scrap_ref(md_file, scraps->scrap, -1, &page);
    fputs("\">", md_file);
    write_scrap_ref(md_file, scraps->scrap, FALSE, &page);
    fputs("</a>", md_file);
    scraps = scraps->next;
  }
  fputs(".\n", md_file);
}
◇
```

File defined by 23a, 84ab, 89a, 92, 93a, 98, 99, 100b, 101b, 102a, 104b, 106a, 107b.
Defines: `md_print_scrap_numbers` 84a, 90c, 91ab, 100d, 103b.
Uses: `FALSE` 22a, `FILE` 21, `fputs` 21, `next` 123b, `page` 123c, `scraps` 124a, `Scrap_Node` 151a, `TRUE` 22a, `write_scrap_ref` 125a.

Formatting a Scrap in Markdown

Scraps are output inside `<pre><code>` blocks. Characters `<`, `>`, and `&` are HTML-escaped. At-sequences within scraps (macro references, keywords, parameters, etc.) emit HTML markup directly.

"markdown.c" 93a≡

```
static int md_scrap_type = 0;

static void md_copy_scrap(FILE *file, int prefix, Name *name)
{
  int indent = 0;
  int c;
  char ** params = name ? name->arg : NULL;
  if (source_last == '{') md_scrap_type = 0;
  if (source_last == '[') md_scrap_type = 1;
  if (source_last == '(') md_scrap_type = 2;
  c = source_get();
  while (1) {
    switch (c) {
      case '\n': fputs("\n", file);
                 indent = 0;
                 break;
      case '\t': ⟨ Expand md tab into spaces 94 ⟩
                 break;
      default:
         if (c==nw_char)
           {
             ⟨ Check md at-sequence for end-of-scrap 95a ⟩
             break;
           }
         if (md_scrap_type == 0 && prefix) {
           ⟨ HTML-escape character c 93b ⟩
         } else {
           putc(c, file);
         }
                 indent++;
                 break;
    }
    c = source_get();
  }
}
◇
```

File defined by 23a, 84ab, 89a, 92, 93a, 98, 99, 100b, 101b, 102a, 104b, 106a, 107b.
Defines: `md_copy_scrap` 84a, 89b, 90a, `md_scrap_type` 95a.
Uses: `FILE` 21, `fputs` 21, `Name` 151b, `nw_char` 28bc, `putc` 21, `source_get` 118c, `source_last` 118c.

⟨ *HTML-escape character c* 93b ⟩ ≡

```
switch (c) {
  case '<': fputs("&lt;", file); break;
  case '>': fputs("&gt;", file); break;
  case '&': fputs("&amp;", file); break;
  default:  putc(c, file); break;
}◇
```

Fragment referenced in 93a.
Uses: `fputs` 21, `putc` 21.

⟨ *Expand md tab into spaces* 94 ⟩ ≡

```
{
  int delta = 8 - (indent % 8);
  indent += delta;
  while (delta > 0) {
    putc(' ', file);
    delta--;
  }
}◇
```

Fragment referenced in 93a.
Uses: `putc` 21.

⟨ *Check md at-sequence for end-of-scrap* 95a ⟩ ≡

```
{
  c = source_get();
  switch (c) {
    case 'c': ⟨ Show md presence of a block comment 96a ⟩
              break;
    case 'x': ⟨ Copy label from source into (95b file ) 71b ⟩
              break;
    case 'v': ⟨ Copy version info into file 71a ⟩
    case 's':
              break;
    case '+':
    case '-':
    case '*':
    case '|': ⟨ Skip over index entries 71d ⟩
    case ',':
    case ')':
    case ']':
    case '}': return;
    case '<': ⟨ Format md macro name 97 ⟩
              break;
    case '%': ⟨ Skip commented-out code 71e ⟩
              break;
    case '_': ⟨ Bold md Keyword 96b ⟩
              break;
    case 't': ⟨ Italic md "fragment title" 96c ⟩
              break;
    case 'f': ⟨ Italic md "file name" 96c ⟩
              break;
    case '1': case '2': case '3':
    case '4': case '5': case '6':
    case '7': case '8': case '9':
              if (name == NULL
                  || name->arg[c - '1'] == NULL) {
                if (md_scrap_type == 0 && prefix)
                  fputs("&amp;", file);
                else
                  putc(nw_char, file);
                fputc(c,   file);
              }
              else {
                md_write_arg(file, name->arg[c - '1']);
              }
              break;
    default:
          if (c==nw_char)
            {
              if (md_scrap_type == 0 && prefix)
                fputs("&amp;", file);
              else
                putc(nw_char, file);
              break;
            }
          /* ignore these since pass1 will have warned about them */
              break;
  }
}◇
```

Fragment referenced in 93a.

⟨ *Show md presence of a block comment* 96a ⟩ ≡

```
{
  fputs("<i>(Comment)</i>", file);
}
◇
```

Fragment referenced in 95a.
Uses: `fputs` 21.

⟨ *Bold md Keyword* 96b ⟩ ≡

```
{
  fprintf(file, "<b>");
  c = source_get();
  do {
      if (c == '<') fputs("&lt;", file);
      else if (c == '>') fputs("&gt;", file);
      else if (c == '&') fputs("&amp;", file);
      else fputc(c, file);
      c = source_get();
  } while (c != nw_char);
  c = source_get();
  fprintf(file, "</b>");
}◇
```

Fragment referenced in 95a.
Uses: `fprintf` 21, `fputs` 21, `nw_char` 28bc, `source_get` 118c.

⟨ *Italic md "whatever"* 96c ⟩ ≡

```
{
  fprintf(file, "<i>whatever</i>");
}◇
```

Fragment referenced in 95a.
Uses: `fprintf` 21.

⟨ *Format md macro name* 97 ⟩ ≡

```
{
  Arglist *args = collect_scrap_name(-1);
  Name *name = args->name;
  char * p = name->spelling;
  Arglist *q = args->args;
  int narg = 0;

  fputs("&lt;<a href=\"#weft", file);
  if (name->defs) {
    write_single_scrap_ref(file, name->defs->scrap);
  } else {
    putc('?', file);
  }
  fputs("\"><i>", file);
  while (*p != '\000') {
    if (*p == ARG_CHR) {
      if (q == NULL) {
         md_write_literal(file, name->arg[narg]);
      }
      else {
        md_write_ArglistElement(file, q, params);
        q = q->next;
      }
      p++;
      narg++;
    }
    else {
      if (*p == '<') fputs("&lt;", file);
      else if (*p == '>') fputs("&gt;", file);
      else if (*p == '&') fputs("&amp;", file);
      else fputc(*p, file);
      p++;
    }
  }
  fputs("</i> ", file);
  if (name->defs) {
    Scrap_Node *dp = name->defs;
    write_single_scrap_ref(file, dp->scrap);
    dp = dp->next;
    if (dp)
      fputs(", ...", file);
  }
  else {
    putc('?', file);
    fprintf(stderr, "%s: never defined <%s>\n",
            command_name, name->spelling);
  }
  fputs("</a>&gt;", file);
}◇
```

Fragment referenced in 95a.

Uses: `Arglist` 170b, `ARG_CHR` 168c, `collect_scrap_name` 171, `command_name` 29a, `defs` 123c, `fprintf` 21, `fputs` 21, `md_write_ArglistElement` 99, `md_write_literal` 98, `Name` 151b, `next` 123b, `putc` 21, `Scrap_Node` 151a, `stderr` 21, `write_single_scrap_ref` 125b.

"markdown.c" 98≡

```
static void md_write_literal(FILE * md_file, char * p)
{
   while (*p!= '\000') {
     if (*p == '<') fputs("&lt;", md_file);
     else if (*p == '>') fputs("&gt;", md_file);
     else if (*p == '&') fputs("&amp;", md_file);
     else fputc(*p, md_file);
     p++;
   }
}
◇
```

File defined by 23a, 84ab, 89a, 92, 93a, 98, 99, 100b, 101b, 102a, 104b, 106a, 107b.
Defines: `md_write_literal` 84a, 97, 99.
Uses: `FILE` 21, `fputs` 21.

"markdown.c" 99≡

```
static void
md_write_ArglistElement(FILE * file, Arglist * args, char ** params)
{
  Name *name = args->name;
  Arglist *q = args->args;

  if (name == NULL) {
    char * p = (char*)q;

    if (p[0] == ARG_CHR) {
       md_write_arg(file, params[p[1] - '1']);
    } else {
       md_write_literal(file, (char *)q);
    }
  } else if (name == (Name *)1) {
    Scrap_Node * qq = (Scrap_Node *)q;
    qq->quoted = TRUE;
    write_scraps(file, "", qq,
                 -1, "", 0, 0, 0, 0, 0,
                 NULL, params, 0, "");
    extra_scraps++;
    qq->quoted = FALSE;
  } else {
    char * p = name->spelling;
    fputs("&lt;<i>", file);
    while (*p != '\000') {
      if (*p == ARG_CHR) {
        md_write_ArglistElement(file, q, params);
        q = q->next;
        p++;
      }
      else {
        if (*p == '<') fputs("&lt;", file);
        else if (*p == '>') fputs("&gt;", file);
        else if (*p == '&') fputs("&amp;", file);
        else fputc(*p, file);
        p++;
      }
    }
    fputs("</i> ", file);
    if (name->defs) {
      Scrap_Node *dp = name->defs;
      write_single_scrap_ref(file, dp->scrap);
      dp = dp->next;
      if (dp)
        fputs(", ...", file);
    }
    else {
      putc('?', file);
      fprintf(stderr, "%s: never defined <%s>\n",
              command_name, name->spelling);
    }
    fputs("&gt;", file);
  }
}
◇
```

File defined by 23a, 84ab, 89a, 92, 93a, 98, 99, 100b, 101b, 102a, 104b, 106a, 107b.
Defines: `md_write_ArglistElement` 84a, 97.
Uses: `Arglist` 170b, `ARG_CHR` 168c, `command_name` 29a, `defs` 123c, `extra_scraps` 63b, `FALSE` 22a, `FILE` 21, `fprintf` 21, `fputs` 21, `md_write_arg` 89a, `md_write_literal` 98, `Name` 151b, `next` 123b, `putc` 21, `Scrap_Node` 151a, `stderr` 21, `TRUE` 22a, `write_scraps` 139a, `write_single_scrap_ref` 125b.

2.5.2 Generating the Indices in Markdown

⟨ *Write md index of file names* 100a ⟩ ≡

```
{
  if (file_names) {
    fputs("\n", md_file);
    md_format_file_entry(file_names, md_file);
    fputs("\n", md_file);
  }
  c = source_get();
}◇
```

Fragment referenced in 86a.
Uses: `file_names` 152a, `fputs` 21, `md_format_file_entry` 100b, `source_get` 118c.

"markdown.c" 100b≡

```
static void md_format_file_entry(Name *name, FILE *md_file)
{
  while (name) {
    md_format_file_entry(name->llink, md_file);
    ⟨ Format a md file index entry 100c ⟩
    name = name->rlink;
  }
}
◇
```

File defined by 23a, 84ab, 89a, 92, 93a, 98, 99, 100b, 101b, 102a, 104b, 106a, 107b.
Defines: `md_format_file_entry` 84a, 100a.
Uses: `FILE` 21, `Name` 151b.

⟨ *Format a md file index entry* 100c ⟩ ≡

```
fputs("- ", md_file);
fprintf(md_file, "`\"%s\"` ", name->spelling);
⟨ Write md file's defining scrap numbers 100d ⟩
putc('\n', md_file);◇
```

Fragment referenced in 100b.
Uses: `fprintf` 21, `fputs` 21, `putc` 21.

⟨ *Write md file's defining scrap numbers* 100d ⟩ ≡

```
{
  Scrap_Node *p = name->defs;
  fputs("*Defined by* ", md_file);
  if (p->next) {
    md_print_scrap_numbers(md_file, p);
  }
  else {
    fputs("<a href=\"#weft", md_file);
    write_single_scrap_ref(md_file, p->scrap);
    fputs("\">", md_file);
    write_single_scrap_ref(md_file, p->scrap);
    fputs("</a>.", md_file);
  }
}◇
```

Fragment referenced in 100c.
Uses: `defs` 123c, `fputs` 21, `md_print_scrap_numbers` 92, `next` 123b, `Scrap_Node` 151a, `write_single_scrap_ref` 125b.

⟨ *Write md index of macro names* 101a ⟩ ≡

```
{
  unsigned char sector = current_sector;
  int c = source_get();
  if (c == '+')
     sector = 0;
  else
     source_ungetc(&c);
  if (has_sector(macro_names, sector)) {
    fputs("\n", md_file);
    md_format_entry(macro_names, md_file, sector);
    fputs("\n", md_file);
  } else {
    fputs("None.\n", md_file);
  }
}
c = source_get();
◇
```

Fragment referenced in 86a.
Uses: `current_sector` 42c, `fputs` 21, `has_sector` 81b, `macro_names` 152a, `md_format_entry` 102a, `sector` 123c, `source_get` 118c, `source_ungetc` 119b.

"`markdown.c`" 101b≡

```
static int md_load_entry(Name * name, Name ** nms, int n)
{
   while (name) {
      n = md_load_entry(name->llink, nms, n);
      nms[n++] = name;
      name = name->rlink;
   }
   return n;
}
◇
```

File defined by 23a, 84ab, 89a, 92, 93a, 98, 99, 100b, 101b, 102a, 104b, 106a, 107b.
Defines: `md_load_entry` 102a.
Uses: `Name` 151b.

"markdown.c" 102a≡

```
static void md_format_entry(Name *name, FILE *md_file, unsigned char sector)
{
  Name ** nms = malloc(num_scraps()*sizeof(Name *));
  int n = md_load_entry(name, nms, 0);
  int i;

  ⟨ Sort nms of size n for ⟨ Rob's ordering 78c ⟩ 79a ⟩
  for (i = 0; i < n; i++)
  {
     Name * name = nms[i];

     ⟨ Format a md index entry 102b ⟩
  }
  free(nms);
}
◇
```

File defined by 23a, 84ab, 89a, 92, 93a, 98, 99, 100b, 101b, 102a, 104b, 106a, 107b.
Defines: `md_format_entry` 84a, 101a.
Uses: `FILE` 21, `malloc` 21, `md_load_entry` 101b, `Name` 151b, `num_scraps` 124a, `sector` 123c.

⟨ *Format a md index entry* 102b ⟩ ≡

```
if (name->sector == sector){
  fputs("- ", md_file);
  fputs("&lt;*", md_file);
  ⟨ Write the md macro's name 88c ⟩
  fputs("* ", md_file);
  ⟨ Write md defining scrap numbers 103a ⟩
  fputs("&gt; ", md_file);
  ⟨ Write md referencing scrap numbers 103b ⟩
  putc('\n', md_file);
}◇
```

Fragment referenced in 102a.
Uses: `fputs` 21, `putc` 21, `sector` 123c.

⟨ *Write md defining scrap numbers* 103a ⟩ ≡

```
{
  Scrap_Node *p = name->defs;
  if (p) {
    int page;
    fputs("<a href=\"#weft", md_file);
    write_scrap_ref(md_file, p->scrap, -1, &page);
    fputs("\">", md_file);
    write_scrap_ref(md_file, p->scrap, TRUE, &page);
    fputs("</a>", md_file);
    p = p->next;
    while (p) {
      fputs("<a href=\"#weft", md_file);
      write_scrap_ref(md_file, p->scrap, -1, &page);
      fputs("\">", md_file);
      write_scrap_ref(md_file, p->scrap, FALSE, &page);
      fputs("</a>", md_file);
      p = p->next;
    }
  }
  else
    putc('?', md_file);
}◇
```

Fragment referenced in 102b.
Uses: `defs` 123c, `FALSE` 22a, `fputs` 21, `next` 123b, `page` 123c, `putc` 21, `Scrap_Node` 151a, `TRUE` 22a, `write_scrap_ref` 125a.

⟨ *Write md referencing scrap numbers* 103b ⟩ ≡

```
{
  Scrap_Node *p = name->uses;
  if (p) {
    fputs("*Referenced in* ", md_file);
    if (p->next) {
      md_print_scrap_numbers(md_file, p);
    }
    else {
      fputs("<a href=\"#weft", md_file);
      write_single_scrap_ref(md_file, p->scrap);
      fputs("\">", md_file);
      write_single_scrap_ref(md_file, p->scrap);
      fputs("</a>.", md_file);
    }
  }
  else
    fputs("*Not referenced*.", md_file);
}◇
```

Fragment referenced in 102b.
Uses: `fputs` 21, `md_print_scrap_numbers` 92, `next` 123b, `Scrap_Node` 151a, `uses` 123c, `write_single_scrap_ref` 125b.

⟨ *Write md index of user-specified names* 104a ⟩ ≡

```
{
    unsigned char sector = current_sector;
    c = source_get();
    if (c == '+') {
       sector = 0;
       c = source_get();
    }
    if (has_sector(user_names, sector)) {
      fputs("\n", md_file);
      md_format_user_entry(user_names, md_file, sector);
      fputs("\n", md_file);
    }
}◇
```

Fragment referenced in 86a.
Uses: `current_sector` 42c, `fputs` 21, `has_sector` 81b, `md_format_user_entry` 104b, `sector` 123c, `source_get` 118c, `user_names` 152a.

"`markdown.c`" 104b≡

```
static void md_format_user_entry(Name *name, FILE *md_file, unsigned char sector)
{
  while (name) {
    md_format_user_entry(name->llink, md_file, sector);
    ⟨ Format a md user index entry 104c ⟩
    name = name->rlink;
  }
}
◇
```

File defined by 23a, 84ab, 89a, 92, 93a, 98, 99, 100b, 101b, 102a, 104b, 106a, 107b.
Defines: `md_format_user_entry` 84a, 104a.
Uses: `FILE` 21, `Name` 151b, `sector` 123c.

⟨ *Format a md user index entry* 104c ⟩ ≡

```
if (name->sector == sector){
  Scrap_Node *uses = name->uses;
  if ( uses || dangling_flag ) {
    int page;
    Scrap_Node *defs = name->defs;
    fprintf(md_file, "- `%s`: ", name->spelling);
    if (!uses) {
        fputs("(**", md_file);
        fputs("<a href=\"#weft", md_file);
        write_single_scrap_ref(md_file, defs->scrap);
        fputs("\">", md_file);
        write_single_scrap_ref(md_file, defs->scrap);
        fputs("</a>**)", md_file);
        page = -2;
        defs = defs->next;
    }
    else
      if (!defs || uses->scrap < defs->scrap) {
      fputs("<a href=\"#weft", md_file);
      write_scrap_ref(md_file, uses->scrap, -1, &page);
      fputs("\">", md_file);
      write_scrap_ref(md_file, uses->scrap, TRUE, &page);
```

```
      fputs("</a>", md_file);
      uses = uses->next;
    }
    else {
      if (defs->scrap == uses->scrap)
        uses = uses->next;
      fputs("**", md_file);

      fputs("<a href=\"#weft", md_file);
      write_single_scrap_ref(md_file, defs->scrap);
      fputs("\">", md_file);
      write_single_scrap_ref(md_file, defs->scrap);
      fputs("</a>**", md_file);
      page = -2;
      defs = defs->next;
    }
    while (uses || defs) {
      if (uses && (!defs || uses->scrap < defs->scrap)) {
        fputs("<a href=\"#weft", md_file);
        write_scrap_ref(md_file, uses->scrap, -1, &page);
        fputs("\">", md_file);
        write_scrap_ref(md_file, uses->scrap, FALSE, &page);
        fputs("</a>", md_file);
        uses = uses->next;
      }
      else {
        if (uses && defs->scrap == uses->scrap)
          uses = uses->next;
        fputs(", **", md_file);

        fputs("<a href=\"#weft", md_file);
        write_single_scrap_ref(md_file, defs->scrap);
        fputs("\">", md_file);
        write_single_scrap_ref(md_file, defs->scrap);
        fputs("</a>**", md_file);
        page = -2;
        defs = defs->next;
      }
    }
    fputs(".\n", md_file);
  }
}◇
```

Fragment referenced in 104b.
Uses: `dangling_flag` 27, `defs` 123c, `FALSE` 22a, `fprintf` 21, `fputs` 21, `next` 123b, `page` 123c, `Scrap_Node` 151a, `sector` 123c, `TRUE` 22a, `uses` 123c, `write_scrap_ref` 125a, `write_single_scrap_ref` 125b.

2.5.3 Identifier Cross-References in Markdown

These functions are the Markdown equivalents of `format_uses_refs` and `format_defs_refs` from `scraps.c`, but emit Markdown blockquote format.

"markdown.c" 106a≡

```
static void
md_format_uses_refs(FILE * md_file, int scrap)
{
  Uses * p = get_scrap_uses(scrap);
  if (p != NULL)
    ⟨ Write md uses references 106b ⟩
}
◇
```

File defined by 23a, 84ab, 89a, 92, 93a, 98, 99, 100b, 101b, 102a, 104b, 106a, 107b.
Defines: `md_format_uses_refs` 84a, 90bd.
Uses: `FILE` 21, `get_scrap_uses` 125c, `Uses` 185c.

⟨ *Write md uses references* 106b ⟩ ≡

```
{
  char join = ' ';
  fputs("> *Uses:*", md_file);
  do {
    ⟨ Write one md use reference 106c ⟩
    join = ',';
    p = p->next;
  }while (p != NULL);
  fputs(".\n", md_file);
}◇
```

Fragment referenced in 106a.
Uses: `fputs` 21, `next` 123b, `Uses` 185c.

⟨ *Write one md use reference* 106c ⟩ ≡

```
Name * name = p->defn;
Scrap_Node *defs = name->defs;
int first = TRUE, page = -1;
fprintf(md_file,
        "%c `%s` ",
        join, name->spelling);
if (defs)
{
  do {
    ⟨ Write one md referenced scrap 107a ⟩
    first = FALSE;
    defs = defs->next;
  }while (defs!= NULL);
}
else
{
  fputs("*(not defined globally)*", md_file);
}
◇
```

Fragment referenced in 106b.
Uses: `defs` 123c, `FALSE` 22a, `first` 193a, `fprintf` 21, `fputs` 21, `Name` 151b, `next` 123b, `page` 123c, `Scrap_Node` 151a, `TRUE` 22a.

⟨ *Write one md referenced scrap* 107a ⟩ ≡

```
fputs("<a href=\"#weft", md_file);
write_scrap_ref(md_file, defs->scrap, -1, &page);
fputs("\">", md_file);
write_scrap_ref(md_file, defs->scrap, first, &page);
fputs("</a>", md_file);◇
```

Fragment referenced in 106c, 108.
Uses: `defs` 123c, `first` 193a, `fputs` 21, `page` 123c, `write_scrap_ref` 125a.

`"markdown.c"` 107b≡

```
static void
md_format_defs_refs(FILE * md_file, int scrap)
{
  Uses * p = get_scrap_defs(scrap);
  if (p != NULL)
    ⟨ Write md defs references 107c ⟩
}
◇
```

File defined by 23a, 84ab, 89a, 92, 93a, 98, 99, 100b, 101b, 102a, 104b, 106a, 107b.
Defines: `md_format_defs_refs` 84a, 90bd.
Uses: `FILE` 21, `get_scrap_defs` 126a, `Uses` 185c.

⟨ *Write md defs references* 107c ⟩ ≡

```
{
  char join = ' ';
  fputs("> *Defines:*", md_file);
  do {
    ⟨ Write one md def reference 108 ⟩
    join = ',';
    p = p->next;
  }while (p != NULL);
  fputs(".\n", md_file);
}◇
```

Fragment referenced in 107b.
Uses: `fputs` 21, `next` 123b.

⟨ *Write one md def reference* 108 ⟩ ≡

```
Name * name = p->defn;
Scrap_Node *defs = name->uses;
int first = TRUE, page = -1;
fprintf(md_file,
        "%c `%s` ",
        join, name->spelling);
if (defs == NULL
    || (defs->scrap == scrap && defs->next == NULL)) {
  fputs("*Never used*", md_file);
}
else {
  do {
    if (defs->scrap != scrap) {
      ⟨ Write one md referenced scrap 107a ⟩
      first = FALSE;
    }
    defs = defs->next;
  }while (defs!= NULL);
}
◇
```

Fragment referenced in 107c.
Uses: `defs` 123c, `FALSE` 22a, `first` 193a, `fprintf` 21, `fputs` 21, `Name` 151b, `next` 123b, `page` 123c, `Scrap_Node` 151a, `TRUE` 22a, `uses` 123c.

2.6 Map and Extract Output

Two modes support AI-assisted navigation of weft projects:

`-m` (map) emits a JSON object to stdout describing every output file and every fragment, their definition locations, usage relationships, and transitive dependencies.

`-e` *name* (extract) emits the scrap code and dependency information for a single fragment (matched by prefix) as Markdown to stdout.

Both modes run after `pass1` has built the name trees and do *not* invoke the weave or tangle phases.

2.6.1 JSON Map Output

The `write_map` function performs an in-order traversal of the `file_names` and `macro_names` binary search trees, emitting a JSON object with two arrays: `"files"` and `"fragments"`.

For each `Name` node, it records:

- The spelling (name of the file or fragment).
- The sector number.
- The language tag (if any).
- Each definition scrap with its source file and line.
- Each usage scrap (scraps that reference this name).
- For definition scraps, the list of fragments referenced inside (via `get_scrap_uses`).

A helper function escapes strings for JSON output.

"output.c" 109≡

```
static void json_puts(FILE *f, const char *s)
{
  fputc('"', f);
  while (*s) {
    switch (*s) {
      case '"':  fputs("\\\"", f); break;
      case '\\': fputs("\\\\", f); break;
      case '\n': fputs("\\n", f);  break;
      case '\r': fputs("\\r", f);  break;
      case '\t': fputs("\\t", f);  break;
      default:
        if ((unsigned char)*s < 0x20)
          fprintf(f, "\\u%04x", (unsigned char)*s);
        else
          fputc(*s, f);
    }
    s++;
  }
  fputc('"', f);
}
◇
```

File defined by 23b, 109, 110, 111ab, 112, 113ab, 114b.
Defines: `json_puts` 110.
Uses: `FILE` 21, `fprintf` 21, `fputs` 21.

The `emit_name_json` helper emits a single name entry as a JSON object. The `is_file` flag distinguishes file names (which use `"uses_fragments"` for fragments referenced inside their definition scraps) from macro fragments (which use `"used_by"` for the scraps that invoke them, and `"uses"` for fragments they invoke).

"output.c" 110≡

```
static void emit_name_json(FILE *f, Name *name, int is_file, int *first)
{
  Scrap_Node *sn;
  int first_inner;

  if (!name) return;
  emit_name_json(f, name->llink, is_file, first);

  if (!*first) fputs(",\n", f);
  *first = 0;

  fputs("    {\n", f);
  fputs("      \"name\": ", f); json_puts(f, name->spelling);
  if (name->lang[0]) {
    fputs(",\n      \"lang\": ", f); json_puts(f, name->lang);
  }
  fprintf(f, ",\n      \"sector\": %d", name->sector);

  /* defs array */
  fputs(",\n      \"defs\": [", f);
  first_inner = 1;
  for (sn = name->defs; sn; sn = sn->next) {
    if (!first_inner) fputs(", ", f);
    first_inner = 0;
    fprintf(f, "{\"scrap\": %d, \"file\": ", sn->scrap);
    json_puts(f, scrap_file_name(sn->scrap));
    fprintf(f, ", \"line\": %d}", scrap_file_line(sn->scrap));
  }
  fputs("]", f);

  if (is_file) {
    /* uses_fragments: fragments referenced inside this file's scraps */
    fputs(",\n      \"uses_fragments\": [", f);
    first_inner = 1;
    for (sn = name->defs; sn; sn = sn->next) {
      Uses *u;
      for (u = get_scrap_uses(sn->scrap); u; u = u->next) {
        if (!first_inner) fputs(", ", f);
        first_inner = 0;
        json_puts(f, u->defn->spelling);
      }
    }
    fputs("]", f);
  } else {
    /* used_by: scraps that invoke this fragment */
    fputs(",\n      \"used_by\": [", f);
    first_inner = 1;
    for (sn = name->uses; sn; sn = sn->next) {
      if (!first_inner) fputs(", ", f);
      first_inner = 0;
      fprintf(f, "{\"scrap\": %d, \"file\": ", sn->scrap);
      json_puts(f, scrap_file_name(sn->scrap));
      fprintf(f, ", \"line\": %d}", scrap_file_line(sn->scrap));
    }
    fputs("]", f);

    /* uses: fragments referenced inside this fragment's def scraps */
    fputs(",\n      \"uses\": [", f);
    first_inner = 1;
    for (sn = name->defs; sn; sn = sn->next) {
      Uses *u;
      for (u = get_scrap_uses(sn->scrap); u; u = u->next) {
        if (!first_inner) fputs(", ", f);
        first_inner = 0;
        json_puts(f, u->defn->spelling);
      }
    }
    fputs("]", f);
  }
```

"output.c" 111a≡

```
void write_map(Name *files, Name *macros)
{
  int first;

  fputs("{\n  \"files\": [\n", stdout);
  first = 1;
  emit_name_json(stdout, files, 1, &first);
  fputs("\n  ],\n  \"fragments\": [\n", stdout);
  first = 1;
  emit_name_json(stdout, macros, 0, &first);
  fputs("\n  ]\n}\n", stdout);
}
◇
```

File defined by 23b, 109, 110, 111ab, 112, 113ab, 114b.
Defines: `write_map` 37, 114a.
Uses: `emit_name_json` 110, `first` 193a, `fputs` 21, `Name` 151b.

2.6.2 Fragment Extraction

The `write_extract` function finds a fragment by name (using prefix matching, like `@<name...@>` in the source), then emits the scrap code and dependency information as Markdown. It recurses into referenced fragments to provide the full transitive closure of the requested fragment.

To find a name in the BST without creating a new entry, we use a read-only search function:

"output.c" 111b≡

```
static Name *find_name(Name *node, const char *spelling)
{
  Name *found;
  if (!node) return NULL;
  if (robs_strcmp(node->spelling, (char *)spelling) == 0)
    return node;
  found = find_name(node->llink, spelling);
  if (found) return found;
  return find_name(node->rlink, spelling);
}
◇
```

File defined by 23b, 109, 110, 111ab, 112, 113ab, 114b.
Defines: `find_name` 113b.
Uses: `Name` 151b, `robs_strcmp` 156a.

A recursive helper emits one fragment and all its transitive dependencies. A visited array (indexed by Name pointer cast to a simple mark) prevents infinite recursion on cyclic references.

"output.c" 112≡

```
static void extract_fragment(FILE *f, Name *name, Name *files, Name *macros)
{
  Scrap_Node *sn;

  if (!name || name->mark) return;
  name->mark = TRUE;

  for (sn = name->defs; sn; sn = sn->next) {
    fprintf(f, "\n## \"%s\" --- %s:%d\n\n",
            name->spelling,
            scrap_file_name(sn->scrap),
            scrap_file_line(sn->scrap));
    if (name->lang[0])
      fprintf(f, "```%s\n", name->lang);
    else
      fputs("```\n", f);
    dump_scrap_text(f, sn->scrap);
    fputs("\n```\n", f);

    /* Show dependencies */
    {
      Uses *u = get_scrap_uses(sn->scrap);
      if (u) {
        fputs("\n### Uses\n", f);
        for (; u; u = u->next)
          fprintf(f, "- %s\n", u->defn->spelling);
      }
    }
  }

  /* Show used_by */
  if (name->uses) {
    Scrap_Node *usn;
    fputs("\n### Used by\n", f);
    for (usn = name->uses; usn; usn = usn->next) {
      fprintf(f, "- scrap %d (%s:%d)\n",
              usn->scrap,
              scrap_file_name(usn->scrap),
              scrap_file_line(usn->scrap));
    }
  }

  /* Recurse into dependencies */
  for (sn = name->defs; sn; sn = sn->next) {
    Uses *u;
    for (u = get_scrap_uses(sn->scrap); u; u = u->next) {
      extract_fragment(f, u->defn, files, macros);
    }
  }
}
◇
```

File defined by 23b, 109, 110, 111ab, 112, 113ab, 114b.
Defines: `extract_fragment` 113b.
Uses: `defs` 123c, `dump_scrap_text` 135, `FILE` 21, `fprintf` 21, `fputs` 21, `get_scrap_uses` 125c, `Name` 151b, `next` 123b, `scrap_file_line` 126d, `scrap_file_name` 126c, `Scrap_Node` 151a, `TRUE` 22a, `Uses` 185c, `uses` 123c.

Clear marks after extraction so the name trees remain usable:

"output.c" 113a≡
```
static void clear_marks(Name *node)
{
  if (!node) return;
  clear_marks(node->llink);
  node->mark = FALSE;
  clear_marks(node->rlink);
}
◇
```
File defined by 23b, 109, 110, 111ab, 112, 113ab, 114b.
Defines: clear_marks 113b.
Uses: FALSE 22a, Name 151b.

"output.c" 113b≡
```
void write_extract(char *name, Name *files, Name *macros)
{
  Name *found = NULL;

  /* Search in macro_names first, then file_names */
  found = find_name(macros, name);
  if (!found)
    found = find_name(files, name);

  if (!found) {
    fprintf(stderr, "%s: fragment \"%s\" not found\n", command_name, name);
    exit(1);
  }

  fprintf(stdout, "# Extract: \"%s\"\n", found->spelling);
  extract_fragment(stdout, found, files, macros);

  /* Clean up marks */
  clear_marks(files);
  clear_marks(macros);
}
◇
```
File defined by 23b, 109, 110, 111ab, 112, 113ab, 114b.
Defines: write_extract 37, 114a.
Uses: clear_marks 113a, command_name 29a, exit 21, extract_fragment 112, file_names 152a, find_name 111b, first 193a, fprintf 21, macro_names 152a, Name 151b, stderr 21.

2.7 Writing the Output Files

⟨ *Function prototypes* 113c ⟩ ≡
```
extern void write_files(Name *files);
◇
```
Fragment defined by 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c.
Fragment referenced in 20.
Uses: Name 151b, write_files 114b.

⟨ *Function prototypes* 114a ⟩ ≡

```
extern void write_map(Name *file_names, Name *macro_names);
extern void write_extract(char *name, Name *file_names, Name *macro_names);
◇
```

Fragment defined by 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c.
Fragment referenced in 20.
Uses: `file_names` 152a, `macro_names` 152a, `Name` 151b, `write_extract` 113b, `write_map` 111a.

"`output.c`" 114b≡

```
void write_files(Name *files)
{
  while (files) {
    write_files(files->llink);
    ⟨ Write out files->spelling 115a ⟩
    files = files->rlink;
  }
}
◇
```

File defined by 23b, 109, 110, 111ab, 112, 113ab, 114b.
Defines: `write_files` 37, 113c.
Uses: `Name` 151b.

`MAX_INDENT` defines the maximum number of characters tracked per line for indentation purposes. When a fragment is expanded mid-line, the characters preceding it are recorded so that subsequent lines of the fragment can reproduce the same indentation. Languages like HTML, XML, and SVG routinely contain lines longer than 500 characters, so we set this to 8192. If a line exceeds this limit, indentation beyond the recorded portion falls back to spaces instead of causing a fatal error.

⟨ *Type declarations* 114c ⟩ ≡

```
#define MAX_INDENT 8192
◇
```

Fragment defined by 22a, 50c, 114c, 151ab, 168c, 170b, 173d, 185c, 192a.
Fragment referenced in 20.
Defines: `MAX_INDENT` 58b, 87b, 115a, 142a, 143b.

⟨ *Write out* `files->spelling` 115a ⟩ ≡

```
{
  static char temp_name[FILENAME_MAX];
  static char real_name[FILENAME_MAX];
  static int temp_name_count = 0;
  char indent_chars[MAX_INDENT];
  FILE *temp_file;

  ⟨ Find a free temporary file 115b ⟩

  sprintf(real_name, "%s%s%s", dirpath, path_sep, files->spelling);
  if (verbose_flag)
    fprintf(stderr, "writing %s [%s]\n", files->spelling, temp_name);
  write_scraps(temp_file, files->spelling, files->defs, 0, indent_chars,
               files->debug_flag, files->tab_flag, files->indent_flag,
               files->comment_flag, files->location_flag,
               NULL, NULL, 0, files->spelling);
  fclose(temp_file);

  ⟨ Move the temporary file to the target, if required 116a ⟩
}◇
```

Fragment referenced in 114b.
Uses: `defs` 123c, `fclose` 21, `FILE` 21, `fprintf` 21, `MAX_INDENT` 114c, `stderr` 21, `verbose_flag` 27, `write_scraps` 139a.

⟨ *Find a free temporary file* 115b ⟩ ≡

```
for( temp_name_count = 0; temp_name_count < 10000; temp_name_count++) {
  sprintf(temp_name,"%s%snw%06d", dirpath, path_sep, temp_name_count);
#ifdef O_EXCL
  if (-1 != (temp_file_fd = open(temp_name, O_CREAT|O_WRONLY|O_EXCL))) {
     temp_file = fdopen(temp_file_fd, "w");
     break;
  }
#else
  if (0 != (temp_file = fopen(temp_name, "a"))) {
     if ( 0L == ftell(temp_file)) {
        break;
     } else {
        fclose(temp_file);
        temp_file = 0;
     }
  }
#endif
}
if (!temp_file) {
  fprintf(stderr, "%s: can't create %s for a temporary file\n",
          command_name, temp_name);
  exit(-1);
}
◇
```

Fragment referenced in 115a.
Uses: `command_name` 29a, `exit` 21, `fclose` 21, `fopen` 21, `fprintf` 21, `stderr` 21.

Note the call to `remove` before `rename` – the ANSI/ISO C standard does *not* guarantee that renaming a file to an existing filename will overwrite the file.

⟨ *Move the temporary file to the target, if required* 116a ⟩ ≡

```
if (compare_flag)
  ⟨ Compare the temp file and the old file 116b ⟩
else {
  remove(real_name);
  ⟨ Rename the temporary file to the target 116c ⟩
}
◇
```

Fragment referenced in 115a.
Uses: `compare_flag` 27, `remove` 21.

Again, we use a call to `remove` before `rename`.

⟨ *Compare the temp file and the old file* 116b ⟩ ≡

```
{
  FILE *old_file = fopen(real_name, "r");
  if (old_file) {
    int x, y;
    temp_file = fopen(temp_name, "r");
    do {
      x = getc(old_file);
      y = getc(temp_file);
    } while (x == y && x != EOF);
    fclose(old_file);
    fclose(temp_file);
    if (x == y)
      remove(temp_name);
    else {
      remove(real_name);
      ⟨ Rename the temporary file to the target 116c ⟩
    }
  }
  else
    ⟨ Rename the temporary file to the target 116c ⟩
}◇
```

Fragment referenced in 116a.
Uses: `fclose` 21, `FILE` 21, `fopen` 21, `getc` 21, `remove` 21.

⟨ *Rename the temporary file to the target* 116c ⟩ ≡

```
if (0 != rename(temp_name, real_name)) {
  fprintf(stderr, "%s: can't rename output file to %s\n",
          command_name, real_name);
}
◇
```

Fragment referenced in 116ab.
Uses: `command_name` 29a, `fprintf` 21, `stderr` 21.

Chapter 3

The Support Routines

3.1 Source Files

3.1.1 Global Declarations

We need two routines to handle reading the source files.

⟨ *Function prototypes* 117a ⟩ ≡

```
extern void source_open(char *name); /* pass in the name of the source file */
extern int source_get(void);   /* no args; returns the next char or EOF */
extern int source_last;   /* what last source_get() returned. */
extern int source_peek;   /* The next character to get */
◇
```

Fragment defined by 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c.
Fragment referenced in 20.
Uses: `next` 123b, `source_get` 118c, `source_last` 118c, `source_open` 123a, `source_peek` 118c.

There are also two global variables maintained for use in error messages and such.

⟨ *Global variable declarations* 117b ⟩ ≡

```
extern char *source_name;  /* name of the current file */
extern int source_line;    /* current line in the source file */
◇
```

Fragment defined by 27, 28b, 29a, 42c, 47c, 63b, 117b, 127a, 152a, 192b.
Fragment referenced in 20.
Defines: `source_line` 41, 43c, 44a, 117c, 118c, 119b, 120, 121, 122ab, 123a, 128c, 130, 132, 154b, 161, 162, 163, 164ab, 167, 169b, 171, 172, `source_name` 34, 35, 36, 37, 41, 43c, 44a, 117c, 120, 121, 122ab, 123a, 128c, 130, 132, 154b, 161, 162, 163, 164ab, 167, 168a, 169bc, 171, 172.

⟨ *Global variable definitions* 117c ⟩ ≡

```
char *source_name = NULL;
int source_line = 0;
◇
```

Fragment defined by 28ac, 29b, 42d, 47d, 63c, 117c, 127b, 152b.
Fragment referenced in 24e.
Uses: `source_line` 117b, `source_name` 117b.

3.1.2 Local Declarations

"input.c" 118a≡

```
static FILE *source_file;  /* the current input file */
static int double_at;
static int include_depth;
◇
```

File defined by 24a, 118abc, 119b, 123a.
Defines: `double_at` 120, 123a, `include_depth` 121, 122b, 123a, `source_file` 118c, 119b, 120, 121, 122ab, 123a.
Uses: `FILE` 21.

"input.c" 118b≡

```
static struct {
  FILE *file;
  char *name;
  int line;
} stack[10];
◇
```

File defined by 24a, 118abc, 119b, 123a.
Defines: `stack` 121, 122b.
Uses: `FILE` 21.

3.1.3 Reading a File

The routine `source_get` returns the next character from the current source file. It notices newlines and keeps the line counter `source_line` up to date. It also catches `EOF` and watches for `@` characters. All other characters are immediately returned. We define `source_last` to let us tell which type of scrap we are defining.

"input.c" 118c≡

```
int source_peek;
int source_last;
int source_get(void)
{
  int c;
  source_last = c = source_peek;
  switch (c) {
    case EOF:  ⟨ Handle EOF 122b ⟩
               return c;
    case '\n': source_line++;
    default:
           if (c==nw_char)
             {
               ⟨ Handle an "at" character 120 ⟩
               return c;
             }
           source_peek = getc(source_file);
               return c;
  }
}
◇
```

File defined by 24a, 118abc, 119b, 123a.
Defines: `source_get` 38c, 39a, 40, 41, 42ab, 43c, 44a, 45ab, 46a, 47ab, 52, 54a, 56c, 57a, 58ab, 63ad, 68, 70a, 71cde, 72a, 76a, 77b, 81c, 85ab, 86a, 87ab, 89c, 90a, 93a, 95a, 96b, 100a, 101a, 104a, 117a, 121, 122b, 129, 130, 131b, 132, 133c, 161, 162, 163, 164ab, 167, 168ab, 169c, 171, 172, 173a, `source_last` 54a, 68, 93a, 117a, `source_peek` 45a, 46a, 47a, 117a, 119b, 120, 121, 122b, 123a.
Uses: `getc` 21, `nw_char` 28bc, `source_file` 118a, `source_line` 117b.

`source_ungetc` pushes a read character back to the `source_file`.

⟨ *Function prototypes* 119a ⟩ ≡

```
extern void source_ungetc(int*);
◇
```

Fragment defined by 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c.
Fragment referenced in 20.
Uses: `source_ungetc` 119b.

"input.c" 119b≡

```
void source_ungetc(int *c)
{
  ungetc(source_peek, source_file);
  if(*c == '\n')
    source_line--;
  source_peek=*c;
}
◇
```

File defined by 24a, 118abc, 119b, 123a.
Defines: `source_ungetc` 45b, 46a, 47b, 77b, 101a, 119a.
Uses: `source_file` 118a, `source_line` 117b, `source_peek` 118c.

This whole `@` character handling mess is pretty annoying. I want to recognize `@i` so I can handle include files correctly. At the same time, it makes sense to recognize illegal `@` sequences and complain; this avoids

ever having to check anywhere else. Unfortunately, I need to avoid tripping over the `@@` sequence; hence this whole unsatisfactory `double_at` business.

⟨ *Handle an "at" character* 120 ⟩ ≡

```
{
  c = getc(source_file);
  if (double_at) {
    source_peek = c;
    double_at = FALSE;
    c = nw_char;
  }
  else
    switch (c) {
      case 'i': ⟨ Open an include file 121 ⟩
                break;
      case '#': case 'f': case 'm': case 'u': case 'v':
      case 'd': case 'o': case 'D': case 'O': case 's':
      case 'q': case 'Q': case 'S': case 'l': case 'L': case 't':
      case 'W':
      case '+':
      case '-':
      case '*':
      case '\'':
      case '{': case '}': case '<': case '>': case '|':
      case '(': case ')': case '[': case ']':
      case '%': case '_':
      case ':': case ',': case 'x': case 'c':
      case '1': case '2': case '3': case '4': case '5':
      case '6': case '7': case '8': case '9':
      case 'r':
                source_peek = c;
                c = nw_char;
                break;
      default:
            if (c==nw_char)
              {
                source_peek = c;
                double_at = TRUE;
                break;
              }
             fprintf(stderr, "%s: bad %c sequence %c[%d] (%s, line %d)\n",
                     command_name, nw_char, c, c, source_name, source_line);
             exit(-1);
    }
}◇
```

Fragment referenced in 118c.

Uses: `command_name` 29a, `double_at` 118a, `exit` 21, `FALSE` 22a, `fprintf` 21, `getc` 21, `nw_char` 28bc, `source_file` 118a, `source_line` 117b, `source_name` 117b, `source_peek` 118c, `stderr` 21, `TRUE` 22a.

⟨ *Open an include file* 121 ⟩ ≡

```
{
  char name[FILENAME_MAX];
  char fullname[FILENAME_MAX];
  struct incl * p = include_list;

  if (include_depth >= 10) {
    fprintf(stderr, "%s: include nesting too deep (%s, %d)\n",
            command_name, source_name, source_line);
    exit(-1);
  }
  ⟨ Collect include-file name 122a ⟩
  stack[include_depth].file = source_file;
  fullname[0] = '\0';
  for (;;) {
     strcat(fullname, name);
     source_file = fopen(fullname, "r");
     if (source_file || !p)
        break;
     strcpy(fullname, p->name);
     strcat(fullname, "/");
     p = p->next;
  }
  if (!source_file) {
    fprintf(stderr, "%s: can't open include file %s\n",
            command_name, name);
    source_file = stack[include_depth].file;
  }
  else
  {
     stack[include_depth].name = source_name;
     stack[include_depth].line = source_line + 1;
     include_depth++;
     source_line = 1;
     source_name = save_string(fullname);
  }
  source_peek = getc(source_file);
  c = source_get();
}◇
```

Fragment referenced in 120.

Uses: `command_name` 29a, `exit` 21, `fopen` 21, `fprintf` 21, `getc` 21, `incl` 27, `include_depth` 118a, `next` 123b, `save_string` 153b, `source_file` 118a, `source_get` 118c, `source_line` 117b, `source_name` 117b, `source_peek` 118c, `stack` 118b, `stderr` 21.

⟨ *Collect include-file name* 122a ⟩ ≡

```
{
    char *p = name;
    do
      c = getc(source_file);
    while (c == ' ' || c == '\t');
    while (isgraph(c)) {
      *p++ = c;
      c = getc(source_file);
    }
    *p = '\0';
    if (c != '\n') {
      fprintf(stderr, "%s: unexpected characters after file name (%s, %d)\n",
              command_name, source_name, source_line);
      exit(-1);
    }
}◇
```

Fragment referenced in 121.
Uses: `command_name` 29a, `exit` 21, `fprintf` 21, `getc` 21, `isgraph` 21, `source_file` 118a, `source_line` 117b, `source_name` 117b, `stderr` 21.

If an `EOF` is discovered, the current file must be closed and input from the next stacked file must be resumed. If no more files are on the stack, the `EOF` is returned.

⟨ *Handle* `EOF` 122b ⟩ ≡

```
{
  fclose(source_file);
  if (include_depth) {
    include_depth--;
    source_file = stack[include_depth].file;
    source_line = stack[include_depth].line;
    source_name = stack[include_depth].name;
    source_peek = getc(source_file);
    c = source_get();
  }
}◇
```

Fragment referenced in 118c.
Uses: `fclose` 21, `getc` 21, `include_depth` 118a, `source_file` 118a, `source_get` 118c, `source_line` 117b, `source_name` 117b, `source_peek` 118c, `stack` 118b.

3.1.4 Opening a File

The routine `source_open` takes a file name and tries to open the file. If unsuccessful, it complains and halts. Otherwise, it sets `source_name`, `source_line`, and `double_at`.

"input.c" 123a≡

```
void source_open(char *name)
{
  source_file = fopen(name, "r");
  if (!source_file) {
    fprintf(stderr, "%s: couldn't open %s\n", command_name, name);
    exit(-1);
  }
  nw_char = '@';
  source_name = name;
  source_line = 1;
  source_peek = getc(source_file);
  double_at = FALSE;
  include_depth = 0;
}
◇
```

File defined by 24a, 118abc, 119b, 123a.
Defines: `source_open` 38b, 55b, 84b, 117a.
Uses: `command_name` 29a, `double_at` 118a, `exit` 21, `FALSE` 22a, `fopen` 21, `fprintf` 21, `getc` 21, `include_depth` 118a, `nw_char` 28bc, `source_file` 118a, `source_line` 117b, `source_name` 117b, `source_peek` 118c, `stderr` 21.

3.2 Scraps

"scraps.c" 123b≡

```
#define SLAB_SIZE 1024

typedef struct slab {
  struct slab *next;
  char chars[SLAB_SIZE];
} Slab;
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `next` 27, 30, 31b, 36, 43b, 50d, 64cd, 65ab, 73, 74, 75, 77a, 80ab, 82b, 90c, 91ab, 92, 97, 99, 100d, 103ab, 104c, 106bc, 107c, 108, 110, 112, 117a, 121, 127d, 133ab, 134ac, 136b, 137ac, 138a, 139a, 147a, 148a, 151a, 170bc, 173c, 174abd, 175abc, 176a, 178a, 181, 182, 183, 185abc, 186c, 187a, 188ab, 192d, 193c, 194a, 195, `Slab` 123c, 127cd, 128c, 134c, `SLAB_SIZE` 127d, 134c, 136a, 179b.
Uses: `slab` 123c.

"scraps.c" 123c≡

```
typedef struct {
  char *file_name;
  Slab *slab;
  struct uses *uses;
  struct uses *defs;
  int file_line;
  int page;
  char letter;
  unsigned char sector;
} ScrapEntry;
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.

Defines: `defs` 43b, 50d, 58b, 64cd, 73, 74, 75, 77a, 80a, 82b, 87b, 90c, 91a, 97, 99, 100d, 103a, 104c, 106c, 107a, 108, 110, 112, 115a, 124b, 126a, 128c, 133b, 137c, 139a, 141, 142bcd, 145ab, 146ac, 147ac, 151b, 158, 173d, 174c, 183, 187abd, 188b, `file_line` 126d, 128c, 129, 141, `file_name` 38ab, 54b, 55b, 83, 84b, 126c, 128c, 129, 142bc, `letter` 125a, 128c, 150d, `page` 65b, 80a, 82b, 92, 103a, 104c, 106c, 107a, 108, 124b, 125ab, 128c, 149ab, 150bd, 187ab, 188b, 195, `ScrapEntry` 124ac, 128c, `sector` 54b, 55ab, 77b, 78b, 79b, 81bc, 82ab, 83, 84ab, 101a, 102ab, 104abc, 110, 128c, 132, 133abd, 135, 145b, 148a, 151b, 152c, 154ab, 155a, 157, 158, 167, 169b, 171, 183, `slab` 123b, 128c, 135, 141, 183, `uses` 65a, 80b, 82b, 91b, 103b, 104c, 108, 110, 112, 125c, 128c, 133a, 134a, 151b, 158, 159, 174c, 183, 185c, 186b, 188b.

Uses: `Slab` 123b.

There's some issue with reading the auxiliary file if it has more than 255 lines, so (as a workround only) increase the size of the array of `ScrapEntry`s held in `SCRAP[n]`.

"scraps.c" 124a≡

```
#define SCRAP_BITS 10
#define SCRAP_SIZE (1<<SCRAP_BITS)
#define SCRAP_MASK (SCRAP_SIZE - 1)
#define SCRAP_SHIFT SCRAP_BITS
static ScrapEntry *SCRAP[SCRAP_SIZE];

#define scrap_array(i) SCRAP[(i) >> SCRAP_SHIFT][(i) & SCRAP_MASK]

static int scraps;
int num_scraps(void)
{
   return scraps;
}
⟨ Forward declarations for scraps.c 139b, ... ⟩
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.

Defines: `num_scraps` 78b, 102a, 124b, `SCRAP` 124c, 128c, `scraps` 22e, 23a, 27, 54a, 55a, 59, 60a, 61b, 63a, 65b, 84a, 88ab, 89c, 90bd, 92, 110, 124c, 128c, 149ab, 150bc, 183, 195, `scrap_array` 125ac, 126acd, 128c, 129, 133ab, 135, 141, 142bc, 149ab, 150bd, 183, 186b, 187d, `SCRAP_BITS` Never used, `SCRAP_MASK` 128c, `SCRAP_SHIFT` 128c, `SCRAP_SIZE` 124c, 128c.

Uses: `ScrapEntry` 123c.

⟨ *Function prototypes* 124b ⟩ ≡

```
extern void init_scraps(void);
extern int collect_scrap(void);
extern int write_scraps(FILE *file, char *spelling, Scrap_Node *defs, int global_indent, char *indent_char
extern void write_scrap_ref(FILE *file, int num, int first, int *page);
extern void write_single_scrap_ref(FILE *file, int num);
extern int num_scraps(void);
extern Uses * get_scrap_uses(int scrap);
extern Uses * get_scrap_defs(int scrap);
◇
```

Fragment defined by 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c.
Fragment referenced in 20.
Uses: `Arglist` 170b, `collect_scrap` 128b, `defs` 123c, `FILE` 21, `first` 193a, `get_scrap_defs` 126a, `get_scrap_uses` 125c, `init_scraps` 124c, `num_scraps` 124a, `page` 123c, `Parameters` 50c, `Scrap_Node` 151a, `Uses` 185c, `write_scraps` 139a, `write_scrap_ref` 125a, `write_single_scrap_ref` 125b.

"scraps.c" 124c≡

```
void init_scraps(void)
{
  scraps = 1;
  SCRAP[0] = (ScrapEntry *) arena_getmem(SCRAP_SIZE * sizeof(ScrapEntry));
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `init_scraps` 38b, 124b.
Uses: `arena_getmem` 193b, `SCRAP` 124a, `ScrapEntry` 123c, `scraps` 124a, `SCRAP_SIZE` 124a.

"scraps.c" 125a≡

```
void write_scrap_ref(FILE *file, int num, int first, int *page)
{
  if (scrap_array(num).page >= 0) {
    if (first!=0)
      fprintf(file, "%d", scrap_array(num).page);
    else if (scrap_array(num).page != *page)
      fprintf(file, ", %d", scrap_array(num).page);
    if (scrap_array(num).letter > 0)
      fputc(scrap_array(num).letter, file);
  }
  else {
    if (first!=0)
      putc('?', file);
    else
      fputs(", ?", file);
    ⟨ Warn (only once) about needing to rerun after Latex 126e ⟩
  }
  if (first>=0)
  *page = scrap_array(num).page;
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `write_scrap_ref` 65b, 80a, 82b, 92, 103a, 104c, 107a, 124b, 125b, 187b.
Uses: `FILE` 21, `first` 193a, `fprintf` 21, `fputs` 21, `letter` 123c, `page` 123c, `putc` 21, `scrap_array` 124a.

"scraps.c" 125b≡

```
void write_single_scrap_ref(FILE *file, int num)
{
  int page;
  write_scrap_ref(file, num, TRUE, &page);
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `write_single_scrap_ref` 54a, 59, 60a, 65a, 74, 77a, 80b, 82b, 88ab, 91b, 97, 99, 100d, 103b, 104c, 124b, 146ac, 190c.
Uses: `FILE` 21, `page` 123c, `TRUE` 22a, `write_scrap_ref` 125a.

"scraps.c" 125c≡

```
Uses * get_scrap_uses(int scrap)
{
  return scrap_array(scrap).uses;
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `get_scrap_uses` 106a, 110, 112, 124b.
Uses: `scrap_array` 124a, `Uses` 185c, `uses` 123c.

"scraps.c" 126a≡

```
Uses * get_scrap_defs(int scrap)
{
  return scrap_array(scrap).defs;
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `get_scrap_defs` 107b, 124b.
Uses: `defs` 123c, `scrap_array` 124a, `Uses` 185c.

These accessors expose scrap metadata that is needed by the `--map` and `--extract` modes (Section 2.6) without leaking the internal `ScrapEntry` or `Slab` types.

⟨ *Function prototypes* 126b ⟩ ≡

```
extern const char *scrap_file_name(int i);
extern int scrap_file_line(int i);
extern void dump_scrap_text(FILE *file, int scrap_idx);
◇
```

Fragment defined by 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c.
Fragment referenced in 20.
Uses: `dump_scrap_text` 135, `FILE` 21, `scrap_file_line` 126d, `scrap_file_name` 126c.

"scraps.c" 126c≡

```
const char *scrap_file_name(int i) { return scrap_array(i).file_name; }
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `scrap_file_name` 110, 112, 126b.
Uses: `file_name` 123c, `scrap_array` 124a.

"scraps.c" 126d≡

```
int scrap_file_line(int i) { return scrap_array(i).file_line; }
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `scrap_file_line` 110, 112, 126b.
Uses: `file_line` 123c, `scrap_array` 124a.

⟨ *Warn (only once) about needing to rerun after Latex* 126e ⟩ ≡

```
{
  if (!already_warned) {
    fprintf(stderr, "%s: you'll need to rerun weft after running latex\n",
            command_name);
    already_warned = TRUE;
  }
}◇
```

Fragment referenced in 125a, 149b.
Uses: `already_warned` 127a, `command_name` 29a, `fprintf` 21, `stderr` 21, `TRUE` 22a.

⟨ *Global variable declarations* 127a ⟩ ≡

```
extern int already_warned;
◇
```

Fragment defined by 27, 28b, 29a, 42c, 47c, 63b, 117b, 127a, 152a, 192b.
Fragment referenced in 20.
Defines: `already_warned` 126e, 127b, 149a.

⟨ *Global variable definitions* 127b ⟩ ≡

```
int already_warned = 0;
◇
```

Fragment defined by 28ac, 29b, 42d, 47d, 63c, 117c, 127b, 152b.
Fragment referenced in 24e.
Uses: `already_warned` 127a.

"scraps.c" 127c≡

```
typedef struct {
  Slab *scrap;
  Slab *prev;
  int index;
} Manager;
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `Manager` 127d, 128ab, 134c, 135, 136a, 138b, 141, 176c, 178c, 183.
Uses: `Slab` 123b.

"scraps.c" 127d≡

```
static void push(char c, Manager *manager)
{
  Slab *scrap = manager->scrap;
  int index = manager->index;
  scrap->chars[index++] = c;
  if (index == SLAB_SIZE) {
    Slab *new_slab = (Slab *) arena_getmem(sizeof(Slab));
    scrap->next = new_slab;
    manager->scrap = new_slab;
    index = 0;
  }
  manager->index = index;
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `push` 48b, 52, 128a, 129, 130, 131a, 133cd.
Uses: `arena_getmem` 193b, `Manager` 127c, `next` 123b, `Slab` 123b, `SLAB_SIZE` 123b.

"scraps.c" 128a≡

```
static void pushs(char *s, Manager *manager)
{
  while (*s)
    push(*s++, manager);
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `pushs` 52, 131a, 133d.
Uses: `Manager` 127c, `push` 127d.

"scraps.c" 128b≡

```
int collect_scrap(void)
{
  int current_scrap, lblseq = 0;
  int depth = 1;
  Manager writer;
  ⟨ Create new scrap, managed by writer 128c ⟩
  ⟨ Accumulate scrap and return scraps++ 129 ⟩
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `collect_scrap` 42e, 43a, 52, 124b, 173c.
Uses: `Manager` 127c.

⟨ *Create new scrap, managed by* `writer` 128c ⟩ ≡

```
{
  Slab *scrap = (Slab *) arena_getmem(sizeof(Slab));
  if ((scraps & SCRAP_MASK) == 0)
    SCRAP[scraps >> SCRAP_SHIFT] = (ScrapEntry *) arena_getmem(SCRAP_SIZE * sizeof(ScrapEntry));
  scrap_array(scraps).slab = scrap;
  scrap_array(scraps).file_name = save_string(source_name);
  scrap_array(scraps).file_line = source_line;
  scrap_array(scraps).page = -1;
  scrap_array(scraps).letter = 0;
  scrap_array(scraps).uses = NULL;
  scrap_array(scraps).defs = NULL;
  scrap_array(scraps).sector = current_sector;
  writer.scrap = scrap;
  writer.index = 0;
  current_scrap = scraps++;
}◇
```

Fragment referenced in 128b.
Uses: `arena_getmem` 193b, `current_sector` 42c, `defs` 123c, `file_line` 123c, `file_name` 123c, `letter` 123c, `page` 123c, `save_string` 153b, `SCRAP` 124a, `ScrapEntry` 123c, `scraps` 124a, `scrap_array` 124a, `SCRAP_MASK` 124a, `SCRAP_SHIFT` 124a, `SCRAP_SIZE` 124a, `sector` 123c, `Slab` 123b, `slab` 123c, `source_line` 117b, `source_name` 117b, `uses` 123c.

⟨ *Accumulate scrap and return* `scraps++` 129 ⟩ ≡

```
{
  int c = source_get();
  while (1) {
    switch (c) {
      case EOF: fprintf(stderr, "%s: unexpect EOF in (%s, %d)\n",
                        command_name, scrap_array(current_scrap).file_name,
                        scrap_array(current_scrap).file_line);
                exit(-1);
      default:
        if (c==nw_char)
          {
            ⟨ Handle at-sign during scrap accumulation 130 ⟩
                break;
          }
        push(c, &writer);
                c = source_get();
                break;
    }
  }
}◇
```

Fragment referenced in 128b.
Uses: `command_name` 29a, `exit` 21, `file_line` 123c, `file_name` 123c, `fprintf` 21, `nw_char` 28bc, `push` 127d, `scrap_array` 124a, `source_get` 118c, `stderr` 21.

⟨ *Handle at-sign during scrap accumulation* 130 ⟩ ≡

```
{
  c = source_get();
  switch (c) {
    case '(':
    case '[':
    case '{': depth++;
              break;
    case '+':
    case '-':
    case '*':
    case '|': ⟨ Collect user-specified index entries 132 ⟩
              /* Fall through */
    case ')':
    case ']':
    case '}': if (--depth > 0)
                break;
              /* else fall through */
    case ',':
              push('\0', &writer);
              scrap_ended_with = c;
              return current_scrap;
    case '<': ⟨ Handle macro invocation in scrap 133c ⟩
              break;
    case '%': ⟨ Skip commented-out code 71e ⟩
              /* emit line break to the output file to keep #line in sync. */
              push('\n', &writer);
              c = source_get();
              break;
    case 'x': ⟨ Get label while collecting scrap 131a ⟩
              break;
    case 'c': ⟨ Include block comment in a scrap 48b ⟩
              break;
    case '1': case '2': case '3':
    case '4': case '5': case '6':
    case '7': case '8': case '9':
    case 'f': case '#': case 'v':
    case 't': case 's':
              push(nw_char, &writer);
              break;
    case '_': c = source_get();
              break;
    default :
          if (c==nw_char)
            {
              push(nw_char, &writer);
              push(nw_char, &writer);
              c = source_get();
              break;
            }
          fprintf(stderr, "%s: unexpected %c%c in scrap (%s, %d)\n",
                      command_name, nw_char, c, source_name, source_line);
              exit(-1);
  }
}◇
```

Fragment referenced in 129.
Uses: `command_name` 29a, `exit` 21, `fprintf` 21, `nw_char` 28bc, `push` 127d, `source_get` 118c, `source_line` 117b, `source_name` 117b, `stderr` 21.

⟨ *Get label while collecting scrap* 131a ⟩ ≡

```
{
    ⟨ Get label from (131b source_get() ) 189c ⟩
    ⟨ Save label to label store 190e ⟩
    push(nw_char, &writer);
    push('x', &writer);
    pushs(label_name, &writer);
    push(nw_char, &writer);
}◇
```

Fragment referenced in 130.
Uses: `source_get` 118c.

⟨ *Collect user-specified index entries* 132 ⟩ ≡

```
{
  do {
    int type = c;
    do {
      char new_name[MAX_NAME_LEN];
      char *p = new_name;
      unsigned int sector = 0;
      do
        c = source_get();
      while (isspace(c));
      if (c != nw_char) {
        Name *name;
        do {
          *p++ = c;
          c = source_get();
        } while (c != nw_char && !isspace(c));
        *p = '\0';
        switch (type) {
        case '*':
           sector = current_sector;
           ⟨ Add user identifier use 133a ⟩
           break;
        case '-':
           ⟨ Add user identifier use 133a ⟩
           /* Fall through */
        case '|':
           sector = current_sector;
           /* Fall through */
        case '+':
           ⟨ Add user identifier definition 133b ⟩
           break;
        }
      }
    } while (c != nw_char);
    c = source_get();
  }while (c == '|' || c == '*' || c == '-' || c == '+');
  if (c != '}' && c != ']' && c != ')') {
    fprintf(stderr, "%s: unexpected %c%c in index entry (%s, %d)\n",
            command_name, nw_char, c, source_name, source_line);
    exit(-1);
  }
}◇
```

Fragment referenced in 130.

Uses: `command_name` 29a, `current_sector` 42c, `exit` 21, `fprintf` 21, `isspace` 21, `MAX_NAME_LEN` 22b, `Name` 151b, `nw_char` 28bc, `sector` 123c, `source_get` 118c, `source_line` 117b, `source_name` 117b, `stderr` 21.

⟨ *Add user identifier use* 133a ⟩ ≡

```
name = name_add(&user_names, new_name, sector);
if (!name->uses || name->uses->scrap != current_scrap) {
  Scrap_Node *use = (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
  use->scrap = current_scrap;
  use->next = name->uses;
  name->uses = use;
  add_uses(&(scrap_array(current_scrap).uses), name);
}◇
```

Fragment referenced in 132.
Uses: `add_uses` 185b, `arena_getmem` 193b, `name_add` 157, `next` 123b, `scrap_array` 124a, `Scrap_Node` 151a, `sector` 123c, `user_names` 152a, `uses` 123c.

⟨ *Add user identifier definition* 133b ⟩ ≡

```
name = name_add(&user_names, new_name, sector);
if (!name->defs || name->defs->scrap != current_scrap) {
  Scrap_Node *def = (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
  def->scrap = current_scrap;
  def->next = name->defs;
  name->defs = def;
  add_uses(&(scrap_array(current_scrap).defs), name);
}◇
```

Fragment referenced in 132.
Uses: `add_uses` 185b, `arena_getmem` 193b, `defs` 123c, `name_add` 157, `next` 123b, `scrap_array` 124a, `Scrap_Node` 151a, `sector` 123c, `user_names` 152a.

⟨ *Handle macro invocation in scrap* 133c ⟩ ≡

```
{
  Arglist * args = collect_scrap_name(current_scrap);
  Name *name = args->name;
  ⟨ Save macro name 133d ⟩
  add_to_use(name, current_scrap);
  if (scrap_name_has_parameters) {
    ⟨ Save macro parameters 52 ⟩
  }
  push(nw_char, &writer);
  push('>', &writer);
  c = source_get();
}◇
```

Fragment referenced in 130.
Uses: `add_to_use` 134a, `Arglist` 170b, `collect_scrap_name` 171, `Name` 151b, `nw_char` 28bc, `push` 127d, `source_get` 118c.

⟨ *Save macro name* 133d ⟩ ≡

```
{
  char buff[24];

  push(nw_char, &writer);
  push('<', &writer);
  push(name->sector, &writer);
  sprintf(buff, "%d", arglist_register(args));
  pushs(buff, &writer);
}◇
```

Fragment referenced in 133c.
Uses: `arglist_register` 140, `nw_char` 28bc, `push` 127d, `pushs` 128a, `sector` 123c.

"scraps.c" 134a≡

```
void
add_to_use(Name * name, int current_scrap)
{
  if (!name->uses || name->uses->scrap != current_scrap) {
    Scrap_Node *use = (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
    use->scrap = current_scrap;
    use->next = name->uses;
    name->uses = use;
  }
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `add_to_use` 52, 133c, 134b, 174a.
Uses: `arena_getmem` 193b, `Name` 151b, `next` 123b, `Scrap_Node` 151a, `uses` 123c.

⟨ *Function prototypes* 134b ⟩ ≡

```
extern void add_to_use(Name * name, int current_scrap);
◇
```

Fragment defined by 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c.
Fragment referenced in 20.
Uses: `add_to_use` 134a, `Name` 151b.

"scraps.c" 134c≡

```
static char pop(Manager *manager)
{
  Slab *scrap = manager->scrap;
  int index = manager->index;
  char c = scrap->chars[index++];
  if (index == SLAB_SIZE) {
    manager->prev = scrap;
    manager->scrap = scrap->next;
    index = 0;
  }
  manager->index = index;
  return c;
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `pop` 49abd, 53, 135, 138bc, 141, 143a, 144ac, 177b.
Uses: `Manager` 127c, `next` 123b, `Slab` 123b, `SLAB_SIZE` 123b.

The function `dump_scrap_text` walks the slab chain for a given scrap and writes every printable character (plus newlines and tabs) to the supplied file handle. The internal meta-character sequences that encode fragment references are decoded back to their `@<name@>` form; other control sequences are silently skipped.

"scraps.c" 135≡

```
void dump_scrap_text(FILE *file, int scrap_idx)
{
  Manager reader;
  char c;
  reader.scrap = scrap_array(scrap_idx).slab;
  reader.prev = NULL;
  reader.index = 0;
  c = pop(&reader);
  while (c) {
    if (c == nw_char) {
      /* handle internal control sequences */
      c = pop(&reader);
      if (c == '<') {
        /* fragment reference encoded as: sector_byte + decimal_index
           followed by nw_char + '>' */
        char idx_buf[MAX_NAME_LEN];
        char *p = idx_buf;
        int idx;
        (void)pop(&reader); /* sector byte, discard */
        c = pop(&reader);
        while (c != nw_char && c != '\0') {
          *p++ = c;
          c = pop(&reader);
        }
        *p = '\0';
        if (sscanf(idx_buf, "%d", &idx) == 1) {
          Arglist *a = arglist_lookup(idx);
          if (a && a->name) {
            fprintf(file, "%c<%s%c>", nw_char, a->name->spelling, nw_char);
          }
        }
        /* skip past closing '>' marker */
        if (c == nw_char)
          c = pop(&reader);
        /* skip optional parameter block */
        if (c == '(') {
          while (c && c != nw_char)
            c = pop(&reader);
          if (c == nw_char)
            c = pop(&reader);
        }
      } else {
        /* other control seq: skip */
      }
      c = pop(&reader);
      continue;
    }
    fputc(c, file);
    c = pop(&reader);
  }
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.

Defines: `dump_scrap_text` 112, 126b.

Uses: `Arglist` 170b, `arglist_lookup` 140, `FILE` 21, `fprintf` 21, `Manager` 127c, `MAX_NAME_LEN` 22b, `nw_char` 28bc, `pop` 134c, `scrap_array` 124a, `sector` 123c, `slab` 123c.

"scraps.c" 136a≡

```
static void backup(int n, Manager *manager)
{
  int index = manager->index;
  if (n > index
      && manager->prev != NULL)
  {
     manager->scrap = manager->prev;
     manager->prev = NULL;
     index += SLAB_SIZE;
  }
  manager->index = (n <= index ? index - n : 0);
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `backup` 143a.
Uses: `Manager` 127c, `SLAB_SIZE` 123b.

"scraps.c" 136b≡

```
void
lookup(int n, Arglist * par, char * arg[9], Name **name, Arglist ** args)
{
  int i;
  Arglist * p = par;

  for (i = 0; i < n && p != NULL; i++)
    p = p->next;
  if (p == NULL) {
    char * a = arg[n];

    *name = NULL;
    *args = (Arglist *)a;
  }
  else {
    *name = p->name;
    *args = p->args;
  }
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `lookup` 50d, 137c.
Uses: `Arglist` 170b, `Name` 151b, `next` 123b.

"scraps.c" 137a≡

```
Arglist * instance(Arglist * a, Arglist * par, char * arg[9], int * ch)
{
   if (a != NULL) {
      int changed = 0;
      Arglist *args, *next;
      Name* name;
      ⟨ Set up name, args and next 137c ⟩
      if (changed){
        ⟨ Build a new arglist 138a ⟩
        *ch = 1;
      }
   }

   return a;
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `instance` 58b, 87b, 137bc, 145b.
Uses: `Arglist` 170b, `Name` 151b, `next` 123b.

⟨ *Function prototypes* 137b ⟩ ≡

```
Arglist *instance(Arglist *a, Arglist *par, char *arg[9], int *ch);
◇
```

Fragment defined by 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c.
Fragment referenced in 20.
Uses: `Arglist` 170b, `instance` 137a.

⟨ *Set up name, args and next* 137c ⟩ ≡

```
next = instance(a->next, par, arg, &changed);
name = a->name;
if (name == (Name *)1) {
   Embed_Node * q = (Embed_Node *)arena_getmem(sizeof(Embed_Node));
   q->defs = (Scrap_Node *)a->args;
   q->args = par;
   args = (Arglist *)q;
   changed = 1;
} else if (name != NULL)
  args = instance(a->args, par, arg, &changed);
else {
  char * p = (char *)a->args;
   if (p[0] == ARG_CHR) {
      lookup(p[1] - '1', par, arg, &name, &args);
      changed = 1;
   }
   else {
      args = a->args;
   }
}◇
```

Fragment referenced in 137a.
Uses: `arena_getmem` 193b, `Arglist` 170b, `ARG_CHR` 168c, `defs` 123c, `Embed_Node` 173d, `instance` 137a, `lookup` 136b, `Name` 151b, `next` 123b, `Scrap_Node` 151a.

⟨ *Build a new arglist* 138a ⟩ ≡

```
a = (Arglist *)arena_getmem(sizeof(Arglist));
a->name = name;
a->args = args;
a->next = next;◇
```

Fragment referenced in 137a.
Uses: `arena_getmem` 193b, `Arglist` 170b, `next` 123b.

"scraps.c" 138b≡

```
static Arglist *pop_scrap_name(Manager *manager, Parameters *parameters)
{
  char name[MAX_NAME_LEN];
  char *p = name;
  Arglist * args;
  int c;
  int idx;

  (void)pop(manager); /* not sure why we have to pop twice */
  c = pop(manager);

  while (c != nw_char) {
    *p++ = c;
    c = pop(manager);
  }
  *p = '\000';
  if (sscanf(name, "%d", &idx) != 1)
  {
    fprintf(stderr,  "%s: found an internal problem (2)\n", command_name);
    exit(-1);
  }
  args = arglist_lookup(idx);
  ⟨ Check for end of scrap name 138c ⟩
  return args;
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `pop_scrap_name` 145b.
Uses: `Arglist` 170b, `arglist_lookup` 140, `command_name` 29a, `exit` 21, `fprintf` 21, `Manager` 127c, `MAX_NAME_LEN` 22b, `nw_char` 28bc, `Parameters` 50c, `pop` 134c, `stderr` 21.

⟨ *Check for end of scrap name* 138c ⟩ ≡

```
{
  c = pop(manager);
  ⟨ Check for macro parameters 53 ⟩
}◇
```

Fragment referenced in 138b.
Uses: `pop` 134c.

"scraps.c" 139a≡

```
int write_scraps(FILE *file, char *spelling, Scrap_Node *defs,
                 int global_indent, char *indent_chars,
                 char debug_flag, char tab_flag, char indent_flag,
                 unsigned char comment_flag, char location_flag,
                 Arglist *inArgs,
                 char *inParams[9], Parameters parameters, char *title)
{
  /* This is in file file name */
  int indent = 0;
  int newline = 1;
  while (defs) {
    ⟨ Copy defs->scrap to file 141 ⟩
    defs = defs->next;
  }
  return indent + global_indent;
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `write_scraps` 50d, 58b, 75, 87b, 99, 115a, 124b, 145b, 147c.
Uses: `Arglist` 170b, `defs` 123c, `FILE` 21, `next` 123b, `Parameters` 50c, `Scrap_Node` 151a.

⟨ *Forward declarations for scraps.c* 139b ⟩ ≡

```
int delayed_indent = 0;
◇
```

Fragment defined by 139b, 140, 147b, 166, 184c.
Fragment referenced in 124a.
Defines: `delayed_indent` 50d, 141, 143c, 145b, 146ab.

⟨ *Forward declarations for scraps.c* 140 ⟩ ≡

```
#define MAX_ARGLISTS 4096
static Arglist *arglist_table[MAX_ARGLISTS];
static int arglist_count = 0;

void arglist_reset(void)
{
  arglist_count = 0;
}

static int arglist_register(Arglist *a)
{
  if (arglist_count >= MAX_ARGLISTS) {
    fprintf(stderr, "%s: too many macro argument lists (max %d)\n",
            command_name, MAX_ARGLISTS);
    exit(EXIT_FAILURE);
  }
  arglist_table[arglist_count] = a;
  return arglist_count++;
}

static Arglist *arglist_lookup(int idx)
{
  if (idx < 0 || idx >= arglist_count) {
    fprintf(stderr, "%s: invalid arglist index %d\n", command_name, idx);
    exit(EXIT_FAILURE);
  }
  return arglist_table[idx];
}
◇
```

Fragment defined by 139b, 140, 147b, 166, 184c.
Fragment referenced in 124a.
Defines: `arglist_count` Never used, `arglist_lookup` 135, 138b, 184b, `arglist_register` 133d, `arglist_reset` 192c, 194b, `arglist_table` Never used, `MAX_ARGLISTS` Never used.
Uses: `Arglist` 170b, `command_name` 29a, `exit` 21, `fprintf` 21, `stderr` 21.

⟨ *Copy* `defs->scrap` *to* `file` 141 ⟩ ≡

```
{
  char c;
  Manager reader;
  Parameters local_parameters = 0;
  int line_number = scrap_array(defs->scrap).file_line;
  reader.scrap = scrap_array(defs->scrap).slab;
  reader.index = 0;
  ⟨ Insert debugging information if required 142b ⟩
  ⟨ Insert section opening marker 142c ⟩
  if (delayed_indent)
  {
    ⟨ Insert appropriate indentation 143a ⟩
  }
  c = pop(&reader);
  while (c) {
    switch (c) {
      case '\n':
         if (global_indent >= 0) {
           putc(c, file);
           line_number++;
           newline = 1;
           delayed_indent = 0;
           ⟨ Insert appropriate indentation 143a ⟩
           break;
         } else {
           /* Don't show newlines in embedded fragmants */
           fputs(". . .", file);
           return 0;
         }
      case '\t': ⟨ Handle tab characters on output 143d ⟩
                 delayed_indent = 0;
                 break;
      default:
         if (c==nw_char)
           {
             ⟨ Check for macro invocation in scrap 144a ⟩
             break;
           }
         putc(c, file);
         if (global_indent >= 0) {
           ⟨ Add more indentation ' ' 142a ⟩
         }
         indent++;
         if (c > ' ') newline = 0;
         delayed_indent = 0;
         break;
    }
    c = pop(&reader);
  }
  ⟨ Insert section closing marker 142d ⟩
}◇
```

Fragment referenced in 139a.
Uses: `defs` 123c, `delayed_indent` 139b, `file_line` 123c, `fputs` 21, `Manager` 127c, `nw_char` 28bc, `Parameters` 50c, `pop` 134c, `putc` 21, `scrap_array` 124a, `slab` 123c.

We need to make sure that we don't overflow `indent_chars[]`. If the line is longer than `MAX_INDENT`, we

simply stop recording; the `Put out the indent` scrap will use spaces for any positions beyond the recorded range.

⟨ *Add more indentation* *char* 142a ⟩ ≡

```
{
  if (global_indent + indent < MAX_INDENT)
    indent_chars[global_indent + indent] = char;
}◇
```

Fragment referenced in 141, 143d, 144a.
Uses: `MAX_INDENT` 114c.

⟨ *Insert debugging information if required* 142b ⟩ ≡

```
if (debug_flag) {
  fprintf(file, "\n#line %d \"%s\"\n",
          line_number, scrap_array(defs->scrap).file_name);
  ⟨ Insert appropriate indentation 143a ⟩
}◇
```

Fragment referenced in 141, 144a.
Uses: `defs` 123c, `file_name` 123c, `fprintf` 21, `scrap_array` 124a.

Section markers in the Knuth style delimit each scrap with opening and closing comments that include the scrap number and source location. For example, in C: `/* {5: weft.w:28} */` and `/* {:5} */`.

⟨ *Insert section opening marker* 142c ⟩ ≡

```
if (location_flag && comment_flag) {
  fprintf(file, "%s{%d: %s:%d}%s\n",
          comment_begin[comment_flag],
          defs->scrap,
          scrap_array(defs->scrap).file_name,
          line_number,
          comment_end[comment_flag]);
}◇
```

Fragment referenced in 141.
Uses: `comment_begin` 166, `comment_end` 166, `defs` 123c, `file_name` 123c, `fprintf` 21, `scrap_array` 124a.

⟨ *Insert section closing marker* 142d ⟩ ≡

```
if (location_flag && comment_flag) {
  fprintf(file, "%s{:%d}%s\n",
          comment_begin[comment_flag],
          defs->scrap,
          comment_end[comment_flag]);
}◇
```

Fragment referenced in 141.
Uses: `comment_begin` 166, `comment_end` 166, `defs` 123c, `fprintf` 21.

⟨ *Insert appropriate indentation* 143a ⟩ ≡

```
{
  char c1 = pop(&reader);
  char c2 = pop(&reader);

  if (indent_flag && !(⟨ Indent suppressed 143c ⟩)) {
    ⟨ Put out the indent 143b ⟩
  }
  indent = 0;
  backup(2, &reader);
}◇
```

Fragment referenced in 141, 142b.
Uses: `backup` 136a, `pop` 134c.

⟨ *Put out the indent* 143b ⟩ ≡

```
if (tab_flag)
    for (indent=0; indent<global_indent; indent++)
      putc(' ', file);
  else
    for (indent=0; indent<global_indent; indent++)
      putc(indent < MAX_INDENT ? indent_chars[indent] : ' ', file);
◇
```

Fragment referenced in 50d, 143a.
Uses: `MAX_INDENT` 114c, `putc` 21.

Indent will be suppressed if the next character is a newline or if the next two characters are `@#`. If the next two characters are `@<`, we suppress the indent for now but mark that it may be needed when the next fragment is started.

⟨ *Indent suppressed* 143c ⟩ ≡

```
c1 == '\n'
|| (c1 == nw_char && (c2 == '#' || (delayed_indent |= (c2 == '<'))))◇
```

Fragment referenced in 143a.
Uses: `delayed_indent` 139b, `nw_char` 28bc.

⟨ *Handle tab characters on output* 143d ⟩ ≡

```
{
  if (tab_flag)
    ⟨ Expand tab into spaces 69c ⟩
  else {
    putc('\t', file);
    if (global_indent >= 0) {
      ⟨ Add more indentation '\t' 142a ⟩
    }
    indent++;
  }
}◇
```

Fragment referenced in 141.
Uses: `putc` 21.

⟨ *Check for macro invocation in scrap* 144a ⟩ ≡

```
{
  int oldin = indent;
  char oldcf = comment_flag;
  c = pop(&reader);
  switch (c) {
    case 't': ⟨ Copy fragment title into file 146c ⟩
              break;
    case 'c': ⟨ Copy block comment from scrap 49a ⟩
              break;
    case 'f': ⟨ Copy file name into file 145a ⟩
              break;
    case 'x': ⟨ Copy label from scrap into file 144b ⟩
    case '_': break;
    case 'v': ⟨ Copy version info into file 71a ⟩
              break;
    case 's': indent = -global_indent;
              comment_flag = 0;
              break;
    case '<': ⟨ Copy macro into file 145b ⟩
              ⟨ Insert debugging information if required 142b ⟩
              indent = oldin;
              comment_flag = oldcf;
              break;
    ⟨ Handle macro parameter substitution 50d ⟩
              indent = oldin;
              break;
    default:
          if(c==nw_char)
            {
              putc(c, file);
              if (global_indent >= 0) {
                 ⟨ Add more indentation ' ' 142a ⟩
              }
              indent++;
              break;
            }
          /* ignore, since we should already have a warning */
              break;
  }
}◇
```

Fragment referenced in 141.
Uses: `nw_char` 28bc, `pop` 134c, `putc` 21.

⟨ *Copy label from scrap into file* 144b ⟩ ≡

```
{
   ⟨ Get label from (144c pop(&reader) ) 189c ⟩
   write_label(label_name, file);
}◇
```

Fragment referenced in 144a.
Uses: `pop` 134c.

⟨ *Copy file name into file* 145a ⟩ ≡

```
if (defs->quoted)
   fprintf(file, "%cf", nw_char);
else
   fputs(spelling, file);
◇
```

Fragment referenced in 144a.
Uses: `defs` 123c, `fprintf` 21, `fputs` 21, `nw_char` 28bc.

⟨ *Copy macro into* `file` 145b ⟩ ≡

```
{
  Arglist *a = pop_scrap_name(&reader, &local_parameters);
  Name *name = a->name;
  int changed;
  Arglist * args = instance(a->args, inArgs, inParams, &changed);
  int i, narg;
  char * p = name->spelling;
  char * * inParams = name->arg;
  Arglist *q = args;

  if (name->mark) {
    fprintf(stderr, "%s: recursive macro discovered involving <%s>\n",
            command_name, name->spelling);
    exit(-1);
  }
  if (name->defs && !defs->quoted) {
    ⟨ Perhaps comment this macro 146a ⟩
    name->mark = TRUE;
    indent = write_scraps(file, spelling, name->defs, global_indent + indent,
                          indent_chars, debug_flag, tab_flag, indent_flag,
                          comment_flag, location_flag, args, name->arg,
                          local_parameters, name->spelling);
    indent -= global_indent;
    name->mark = FALSE;
  }
  else
  {
    if (delayed_indent)
    {
      for (i = indent + global_indent; --i >= 0; )
         putc(' ', file);
    }

    fprintf(file, "%c<",  nw_char);
    if (name->sector == 0)
       fputc('+', file);
    ⟨ Comment this macro use 147a ⟩
    fprintf(file, "%c>",  nw_char);
    if (!defs->quoted && !tex_flag)
      fprintf(stderr, "%s: macro never defined <%s>\n",
            command_name, name->spelling);
  }
}◇
```

Fragment referenced in 144a.
Uses: `Arglist` 170b, `command_name` 29a, `defs` 123c, `delayed_indent` 139b, `exit` 21, `FALSE` 22a, `fprintf` 21, `instance` 137a, `Name` 151b, `nw_char` 28bc, `pop_scrap_name` 138b, `putc` 21, `sector` 123c, `stderr` 21, `tex_flag` 27, `TRUE` 22a, `write_scraps` 139a.

⟨ *Perhaps comment this macro* 146a ⟩ ≡

```
if (comment_flag && newline) {
   ⟨ Perhaps put a delayed indent 146b ⟩
   fputs(comment_begin[comment_flag], file);
   ⟨ Comment this macro use 147a ⟩
   if (xref_flag) {
      putc(' ', file);
      write_single_scrap_ref(file, name->defs->scrap);
   }
   fputs(comment_end[comment_flag], file);
   putc('\n', file);
   if (!delayed_indent)
      for (i = indent + global_indent; --i >= 0; )
         putc(' ', file);
}
◇
```

Fragment referenced in 50d, 145b.
Uses: `comment_begin` 166, `comment_end` 166, `defs` 123c, `delayed_indent` 139b, `fputs` 21, `putc` 21, `write_single_scrap_ref` 125b, `xref_flag` 27.

⟨ *Perhaps put a delayed indent* 146b ⟩ ≡

```
if (delayed_indent)
   for (i = indent + global_indent; --i >= 0; )
      putc(' ', file);
◇
```

Fragment referenced in 49a, 146a.
Uses: `delayed_indent` 139b, `putc` 21.

⟨ *Copy fragment title into file* 146c ⟩ ≡

```
{
   char * p = title;
   Arglist *q = inArgs;
   int narg;

   ⟨ Comment this macro use 147a ⟩
   if (xref_flag) {
      putc(' ', file);
      write_single_scrap_ref(file, defs->scrap);
   }
}◇
```

Fragment referenced in 144a.
Uses: `Arglist` 170b, `defs` 123c, `putc` 21, `write_single_scrap_ref` 125b, `xref_flag` 27.

⟨ *Comment this macro use* 147a ⟩ ≡

```
narg = 0;
while (*p != '\000') {
  if (*p == ARG_CHR) {
    if (q == NULL) {
       if (defs->quoted)
          fprintf(file, "%c'%s%c'", nw_char, inParams[narg], nw_char);
       else
          fprintf(file, "'%s'", inParams[narg]);
    }
    else {
      comment_ArglistElement(file, q, defs->quoted);
      q = q->next;
    }
    p++;
    narg++;
  }
  else
     fputc(*p++, file);
}◇
```

Fragment referenced in 145b, 146ac.
Uses: `ARG_CHR` 168c, `comment_ArglistElement` 147b, `defs` 123c, `fprintf` 21, `next` 123b, `nw_char` 28bc.

⟨ *Forward declarations for scraps.c* 147b ⟩ ≡

```
static void
comment_ArglistElement(FILE * file, Arglist * args, int quote)
{
  Name *name = args->name;
  Arglist *q = args->args;

  if (name == NULL) {
    if (quote)
       fprintf(file, "%c'%s%c'", nw_char, (char *)q, nw_char);
    else
       fprintf(file, "'%s'", (char *)q);
  } else if (name == (Name *)1) {
     ⟨ Include an embedded scrap in comment 147c ⟩
  } else {
     ⟨ Include a fragment use in comment 148a ⟩
  }
}
◇
```

Fragment defined by 139b, 140, 147b, 166, 184c.
Fragment referenced in 124a.
Defines: `comment_ArglistElement` 147a, 148a.
Uses: `Arglist` 170b, `FILE` 21, `fprintf` 21, `Name` 151b, `nw_char` 28bc.

⟨ *Include an embedded scrap in comment* 147c ⟩ ≡

```
Embed_Node * e = (Embed_Node *)q;
fputc('{', file);
write_scraps(file, "", e->defs, -1, "", 0, 0, 0, 0, 0, e->args, NULL, NULL, "");
fputc('}', file);◇
```

Fragment referenced in 147b.
Uses: `defs` 123c, `Embed_Node` 173d, `write_scraps` 139a.

⟨ *Include a fragment use in comment* 148a ⟩ ≡

```
char * p = name->spelling;
if (quote)
   fputc(nw_char, file);
fputc('<', file);
if (quote && name->sector == 0)
   fputc('+', file);
while (*p != '\000') {
  if (*p == ARG_CHR) {
    comment_ArglistElement(file, q, quote);
    q = q->next;
    p++;
  }
  else
     fputc(*p++, file);
}
if (quote)
   fputc(nw_char, file);
fputc('>', file);◇
```

Fragment referenced in 147b.
Uses: `ARG_CHR` 168c, `comment_ArglistElement` 147b, `next` 123b, `nw_char` 28bc, `sector` 123c.

3.2.1 Collecting Page Numbers

⟨ *Function prototypes* 148b ⟩ ≡

```
extern void collect_numbers(char *aux_name);
◇
```

Fragment defined by 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c.
Fragment referenced in 20.
Uses: `collect_numbers` 149a.

"scraps.c" 149a≡

```
void collect_numbers(char *aux_name)
{
  if (number_flag) {
    int i;
    for (i=1; i<scraps; i++)
      scrap_array(i).page = i;
  }
  else {
    FILE *aux_file = fopen(aux_name, "r");
    already_warned = FALSE;
    if (aux_file) {
      char aux_line[500];
      while (fgets(aux_line, 500, aux_file)) {
        ⟨ Read line in .aux file 149b ⟩
      }
      fclose(aux_file);
      ⟨ Add letters to scraps with duplicate page numbers 150a ⟩
    }
  }
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `collect_numbers` 37, 148b.
Uses: `already_warned` 127a, `FALSE` 22a, `fclose` 21, `FILE` 21, `fopen` 21, `number_flag` 27, `page` 123c, `scraps` 124a, `scrap_array` 124a.

The *memoir* document class creates references in the `.aux` file with nested braces, so after each open brace we have to wait for the matching closing brace.

⟨ *Read line in* `.aux` *file* 149b ⟩ ≡

```
int scrap_number;
int page_number;
int i;
int dummy_idx;
int bracket_depth = 1;
if (1 == sscanf(aux_line,
                "\\newlabel{scrap%d}{{%n",
                &scrap_number,
                &dummy_idx)) {
  for (i = dummy_idx; i < (int)strlen(aux_line) && bracket_depth > 0; i++) {
    if (aux_line[i] == '{') bracket_depth++;
    else if (aux_line[i] == '}') bracket_depth--;
  }
  if (i > dummy_idx
      && i < (int)strlen(aux_line)
      && 1 == sscanf(aux_line+i, "{%d}" ,&page_number)) {
    if (scrap_number < scraps)
      scrap_array(scrap_number).page = page_number;
    else
      ⟨ Warn (only once) about needing to rerun after Latex 126e ⟩
  }
}
◇
```

Fragment referenced in 149a.
Uses: `page` 123c, `scraps` 124a, `scrap_array` 124a, `strlen` 21.

⟨ *Add letters to scraps with duplicate page numbers* 150a ⟩ ≡

```
{
   int i = 0;

   ⟨ Step i to the next valid scrap 150b ⟩
   ⟨ For all remaining scraps 150c ⟩ {
      int j = i;
      ⟨ Step j to the next valid scrap 150b ⟩
      ⟨ Perhaps add letters to the page numbers 150d ⟩
      i = j;
   }
}
◇
```

Fragment referenced in 149a.

⟨ *Step i to the next valid scrap* 150b ⟩ ≡

```
do
   i++;
while (i < scraps && scrap_array(i).page == -1);
◇
```

Fragment referenced in 150a.
Uses: `page` 123c, `scraps` 124a, `scrap_array` 124a.

⟨ *For all remaining scraps* 150c ⟩ ≡

```
while (i < scraps)◇
```

Fragment referenced in 150a.
Uses: `scraps` 124a.

⟨ *Perhaps add letters to the page numbers* 150d ⟩ ≡

```
if (scrap_array(i).page == scrap_array(j).page) {
   if (scrap_array(i).letter == 0)
      scrap_array(i).letter = 'a';
   scrap_array(j).letter = scrap_array(i).letter + 1;
}
◇
```

Fragment referenced in 150a.
Uses: `letter` 123c, `page` 123c, `scrap_array` 124a.

3.3 Names

Each scrap is linked through a simple node:

⟨ *Type declarations* 151a ⟩ ≡

```
typedef struct scrap_node {
  struct scrap_node *next;
  int scrap;
  char quoted;
} Scrap_Node;
◇
```

Fragment defined by 22a, 50c, 114c, 151ab, 168c, 170b, 173d, 185c, 192a.
Fragment referenced in 20.
Defines: `Scrap_Node` 43b, 50d, 55a, 65b, 74, 75, 77a, 80ab, 82b, 84a, 92, 97, 99, 100d, 103ab, 104c, 106c, 108, 110, 112, 124b, 133ab, 134a, 137c, 139a, 151b, 173cd, 174cd, 183, 184c, 185a, 187a, 188b.
Uses: `next` 123b.

A `Name` represents a file or fragment definition. Beyond the spelling and the binary tree links, several flags control how the tangled output is formatted:

- `debug_flag` — if `TRUE`, emit `#line` directives (set by `-d` or auto-detected for C/C++).
- `comment_flag` — selects the comment style for section markers and block comments: 0=none, 1=C, 2=C++, 3=shell, 4=Lua/SQL, 5=HTML/XML (see Section 3.3.5).
- `location_flag` — if `TRUE`, emit Knuth-style opening and closing section markers around each scrap in the tangled output. Activated automatically by `@l` auto-detection or the `@L` command.
- `lang` — the language tag (e.g. `"c"`, `"python"`) used for syntax highlighting in the Markdown weave output. Set from a preceding `@l` command or inferred from the file extension of an `@o` name. Empty if unknown.

⟨ *Type declarations* 151b ⟩ ≡

```
typedef struct name {
  char *spelling;
  struct name *llink;
  struct name *rlink;
  Scrap_Node *defs;
  Scrap_Node *uses;
  char * arg[9];
  int mark;
  char tab_flag;
  char indent_flag;
  char debug_flag;
  char location_flag;
  char suppress_markers;
  unsigned char comment_flag;
  char lang[64];
  unsigned char sector;
} Name;
◇
```

Fragment defined by 22a, 50c, 114c, 151ab, 168c, 170b, 173d, 185c, 192a.
Fragment referenced in 20.
Defines: `Name` 42e, 43a, 50d, 55a, 58b, 59, 60a, 68, 73, 75, 76b, 78ab, 79a, 81ab, 82a, 84a, 87b, 88ab, 93a, 97, 99, 100b, 101b, 102a, 104b, 106c, 108, 110, 111ab, 112, 113abc, 114ab, 132, 133c, 134ab, 136b, 137ac, 145b, 147b, 152abc, 154a, 155a, 157, 158, 163, 167, 169d, 170abc, 172, 173c, 174c, 175a, 178a, 180ab, 183, 184c, 185bc, 187a, 188b, 189b.
Uses: `defs` 123c, `Scrap_Node` 151a, `sector` 123c, `uses` 123c.

The variable `pending_lang` acts as a one-shot channel between the `@l` command in pass1 (which fills it with a language tag like `"js"` or `"python"`) and `collect_file_name` or `collect_macro_name` (which consume it to set the flags and the `lang` field on the next `@o` or `@d` name).

⟨ *Global variable declarations* 152a ⟩ ≡

```
extern Name *file_names;
extern Name *macro_names;
extern Name *user_names;
extern int scrap_name_has_parameters;
extern int scrap_ended_with;
extern char pending_lang[64];
◇
```

Fragment defined by 27, 28b, 29a, 42c, 47c, 63b, 117b, 127a, 152a, 192b.
Fragment referenced in 20.
Defines: `file_names` 37, 38b, 50b, 76a, 100a, 113b, 114a, 152b, 163, `macro_names` 37, 38b, 50b, 77b, 101a, 113b, 114a, 152b, 169b, `pending_lang` 39a, 152b, 165ab, `user_names` 38b, 50b, 81c, 104a, 133ab, 152b, 180a.
Uses: `Name` 151b.

⟨ *Global variable definitions* 152b ⟩ ≡

```
Name *file_names = NULL;
Name *macro_names = NULL;
Name *user_names = NULL;
int scrap_name_has_parameters;
int scrap_ended_with;
char pending_lang[64] = "";
◇
```

Fragment defined by 28ac, 29b, 42d, 47d, 63c, 117c, 127b, 152b.
Fragment referenced in 24e.
Uses: `file_names` 152a, `macro_names` 152a, `Name` 151b, `pending_lang` 152a, `user_names` 152a.

⟨ *Function prototypes* 152c ⟩ ≡

```
extern Name *collect_file_name(void);
extern Name *collect_macro_name(void);
extern Arglist *collect_scrap_name(int current_scrap);
extern Name *name_add(Name **rt, char *spelling, unsigned char sector);
extern Name *prefix_add(Name **rt, char *spelling, unsigned char sector);
extern char *save_string(char *s);
extern void reverse_lists(Name *names);
extern void collect_lang_def(void);
extern void collect_weave_format(void);
◇
```

Fragment defined by 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c.
Fragment referenced in 20.
Uses: `Arglist` 170b, `collect_file_name` 163, `collect_lang_def` 161, `collect_macro_name` 167, `collect_scrap_name` 171, `collect_weave_format` 162, `Name` 151b, `name_add` 157, `prefix_add` 154a, `reverse_lists` 174c, `save_string` 153b, `sector` 123c.

"names.c" 153a≡

```
enum { LESS, GREATER, EQUAL, PREFIX, EXTENSION };

static int compare(char *x, char *y)
{
  int len, result;
  int xl = strlen(x);
  int yl = strlen(y);
  int xp = x[xl - 1] == ' ';
  int yp = y[yl - 1] == ' ';
  if (xp) xl--;
  if (yp) yl--;
  len = xl < yl ? xl : yl;
  result = strncmp(x, y, len);
  if (result < 0) return GREATER;
  else if (result > 0) return LESS;
  else if (xl < yl) {
    if (xp) return EXTENSION;
    else return LESS;
  }
  else if (xl > yl) {
    if (yp) return PREFIX;
    else return GREATER;
  }
  else return EQUAL;
}
◇
```

File defined by 24c, 153ab, 154a, 155a, 156a, 157, 159, 161, 162, 163, 167, 170ac, 171, 174cd.
Defines: `compare` 154a, 155a, `EQUAL` 154a, 155a, `EXTENSION` 154a, 155a, `GREATER` 154a, 155a, `LESS` 154a, 155a, `PREFIX` 154a, 155a.
Uses: `strlen` 21.

"names.c" 153b≡

```
char *save_string(char *s)
{
  char *new_str = (char *) arena_getmem((strlen(s) + 1) * sizeof(char));
  strcpy(new_str, s);
  return new_str;
}
◇
```

File defined by 24c, 153ab, 154a, 155a, 156a, 157, 159, 161, 162, 163, 167, 170ac, 171, 174cd.
Defines: `save_string` 31b, 36, 121, 128c, 152c, 154a, 158, 170a, 174b.
Uses: `arena_getmem` 193b, `strlen` 21.

"names.c" 154a≡

```
static int ambiguous_prefix(Name *node, char *spelling, unsigned char sector);

static char * found_name = NULL;

Name *prefix_add(Name **rt, char *spelling, unsigned char sector)
{
  Name *node = *rt;
  int cmp;

  while (node) {
    switch ((cmp = compare(node->spelling, spelling))) {
    case GREATER:   rt = &node->rlink;
                    break;
    case LESS:      rt = &node->llink;
                    break;
    case EQUAL:
                    found_name = node->spelling;
    case EXTENSION: if (node->sector > sector) {
                       rt = &node->rlink;
                       break;
                    }
                    else if (node->sector < sector) {
                       rt = &node->llink;
                       break;
                    }
                    if (cmp == EXTENSION)
                       node->spelling = save_string(spelling);
                    return node;
    case PREFIX:    ⟨ Check for ambiguous prefix 154b ⟩
                    return node;
    }
    node = *rt;
  }
  ⟨ Create new name entry 158 ⟩
}
◇
```

File defined by 24c, 153ab, 154a, 155a, 156a, 157, 159, 161, 162, 163, 167, 170ac, 171, 174cd.
Defines: `prefix_add` 152c, 169b.
Uses: `compare` 153a, `EQUAL` 153a, `EXTENSION` 153a, `GREATER` 153a, `LESS` 153a, `Name` 151b, `PREFIX` 153a, `save_string` 153b, `sector` 123c.

Since a very short prefix might match more than one fragment name, I need to check for other matches to avoid mistakes. Basically, I simply continue the search down *both* branches of the tree.

⟨ *Check for ambiguous prefix* 154b ⟩ ≡

```
{
  if (ambiguous_prefix(node->llink, spelling, sector) ||
      ambiguous_prefix(node->rlink, spelling, sector))
    fprintf(stderr,
            "%s: ambiguous prefix %c<%s...%c> (%s, line %d)\n",
            command_name, nw_char, spelling, nw_char, source_name, source_line);
}◇
```

Fragment referenced in 154a.
Uses: `command_name` 29a, `fprintf` 21, `nw_char` 28bc, `sector` 123c, `source_line` 117b, `source_name` 117b, `stderr` 21.

"names.c" 155a≡

```
static int ambiguous_prefix(Name *node, char *spelling, unsigned char sector)
{
  while (node) {
    switch (compare(node->spelling, spelling)) {
    case GREATER:   node = node->rlink;
                    break;
    case LESS:      node = node->llink;
                    break;
    case EXTENSION:
    case PREFIX:
    case EQUAL:     if (node->sector > sector) {
                       node = node->rlink;
                       break;
                    }
                    else if (node->sector < sector) {
                       node = node->llink;
                       break;
                    }
                    return TRUE;
    }
  }
  return FALSE;
}
◇
```

File defined by 24c, 153ab, 154a, 155a, 156a, 157, 159, 161, 162, 163, 167, 170ac, 171, 174cd.
Uses: `compare` 153a, `EQUAL` 153a, `EXTENSION` 153a, `FALSE` 22a, `GREATER` 153a, `LESS` 153a, `Name` 151b, `PREFIX` 153a, `sector` 123c, `TRUE` 22a.

Rob Shillingsburg suggested that I organize the index of user-specified identifiers more traditionally; that is, not relying on strict ASCII comparisons via `strcmp`. Ideally, we'd like to see the index ordered like this:

aardvark
Adam
atom
Atomic
atoms

The function `robs_strcmp` implements the desired predicate. It returns -2 for alphabetically less-than, -1 for less-than but only differing in case, zero for equal, 1 for greater-than but only in case and 2 for alphabetically greater-than.

⟨ *Function prototypes* 155b ⟩ ≡

```
extern int robs_strcmp(char*, char*);
◇
```

Fragment defined by 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c.
Fragment referenced in 20.
Uses: `robs_strcmp` 156a.

"names.c" 156a≡

```
int robs_strcmp(char* x, char* y)
{
   int cmp = 0;

   for (; *x && *y; x++, y++)
   {
      ⟨ Skip invisibles on x 156b ⟩
      ⟨ Skip invisibles on y 156b ⟩
      if (*x == *y)
         continue;
      if (islower(*x) && toupper(*x) == *y)
      {
         if (!cmp) cmp = 1;
         continue;
      }
      if (islower(*y) && *x == toupper(*y))
      {
         if (!cmp) cmp = -1;
         continue;
      }
      return 2*(toupper(*x) - toupper(*y));
   }
   if (*x)
      return 2;
   if (*y)
      return -2;
   return cmp;
}
◇
```

File defined by 24c, 153ab, 154a, 155a, 156a, 157, 159, 161, 162, 163, 167, 170ac, 171, 174cd.
Defines: `robs_strcmp` 78c, 111b, 155b, 157, 158, 185b.
Uses: `islower` 21, `toupper` 21.

Certain character sequences are invisible when printed. We don't want them to be considered for the alphabetical ordering.

⟨ *Skip invisibles on* *p* 156b ⟩ ≡

```
if (*p == '|')
   p++;
◇
```

Fragment referenced in 156a.

"names.c" 157≡

```
Name *name_add(Name **rt, char *spelling, unsigned char sector)
{
  Name *node = *rt;
  while (node) {
    int result = robs_strcmp(node->spelling, spelling);
    if (result > 0)
      rt = &node->llink;
    else if (result < 0)
      rt = &node->rlink;
    else
    {
       found_name = node->spelling;
       if (node->sector > sector)
         rt = &node->llink;
       else if (node->sector < sector)
         rt = &node->rlink;
       else
         return node;
    }
    node = *rt;
  }
  ⟨ Create new name entry 158 ⟩
}
◇
```

File defined by 24c, 153ab, 154a, 155a, 156a, 157, 159, 161, 162, 163, 167, 170ac, 171, 174cd.
Defines: `name_add` 133ab, 152c, 163.
Uses: `Name` 151b, `robs_strcmp` 156a, `sector` 123c.

⟨ *Create new name entry* 158 ⟩ ≡

```
{
  node = (Name *) arena_getmem(sizeof(Name));
  if (found_name && robs_strcmp(found_name, spelling) == 0)
     node->spelling = found_name;
  else
     node->spelling = save_string(spelling);
  node->mark = FALSE;
  node->llink = NULL;
  node->rlink = NULL;
  node->uses = NULL;
  node->defs = NULL;
  node->arg[0] =
  node->arg[1] =
  node->arg[2] =
  node->arg[3] =
  node->arg[4] =
  node->arg[5] =
  node->arg[6] =
  node->arg[7] =
  node->arg[8] = NULL;
  node->tab_flag = TRUE;
  node->indent_flag = TRUE;
  node->debug_flag = FALSE;
  node->location_flag = FALSE;
  node->suppress_markers = FALSE;
  node->comment_flag = 0;
  node->lang[0] = '\0';
  node->sector = sector;
  *rt = node;
  return node;
}◇
```

Fragment referenced in 154a, 157.
Uses: `arena_getmem` 193b, `defs` 123c, `FALSE` 22a, `Name` 151b, `robs_strcmp` 156a, `save_string` 153b, `sector` 123c, `TRUE` 22a, `uses` 123c.

3.3.1 Language Auto-Detection

When a Markdown fence (```c, ```python) or a LaTeX marker (%%%c) precedes an `@o` directive, weft auto-detects the language's comment style and, for C/C++, enables `#line` directives. This is driven by a data table that maps fence tags to one of the six comment styles defined in Section 3.3.5.

Each entry records:

- `name` — the fence tag (lowercase, e.g. `"js"`, `"python"`, `"lua"`).
- `comment` — index into the `comment_begin`/`comment_end` arrays (1–5).
- `line_directive` — `TRUE` only for languages whose toolchains honour `#line` (C family).

The built-in table covers over 250 entries (languages, file extensions, and aliases) organised by comment style. For any language not listed, the `@L` command (Section 3.3.2) lets users add entries at the document level. User-defined entries are searched first, so they can also override built-in defaults. If a fence tag matches neither table, weft defaults to C++ line comments (//) with no `#line`.

"names.c" 159≡

```
typedef struct {
  const char *name;
  unsigned char comment;
  char line_directive;
} LangEntry;

static const LangEntry lang_table[] = {
  /* ---- Style 1: C block comments WITH #line ---- */
  {"c",            1, TRUE},
  {"h",            1, TRUE},
  {"hpp",          2, TRUE},
  {"hxx",          2, TRUE},
  {"hh",           2, TRUE},
  {"cpp",          2, TRUE},
  {"c++",          2, TRUE},
  {"cxx",          2, TRUE},
  {"cc",           2, TRUE},
  {"objc",         1, TRUE},
  {"objcpp",       2, TRUE},
  {"m",            1, TRUE},
  {"mm",           2, TRUE},
  {"ino",          2, TRUE},   /* Arduino */
  {"pde",          2, TRUE},   /* Processing (C-like) */
  /* ---- Style 1: C block comments WITHOUT #line ---- */
  {"css",          1, FALSE},
  {"scss",         1, FALSE},
  {"sass",         1, FALSE},
  {"less",         1, FALSE},
  {"stylus",       1, FALSE},
  {"styl",         1, FALSE},
  /* ---- Style 2: C++ line comments (//) ---- */
  /* --- JavaScript / TypeScript --- */
  {"js",           2, FALSE},
  {"javascript",   2, FALSE},
  {"jsx",          2, FALSE},
  {"mjs",          2, FALSE},
  {"cjs",          2, FALSE},
  {"ts",           2, FALSE},
  {"typescript",   2, FALSE},
  {"tsx",          2, FALSE},
  {"mts",          2, FALSE},
  {"cts",          2, FALSE},
  {"svelte",       2, FALSE},
  {"vue",          2, FALSE},
  /* --- JVM languages --- */
  {"java",         2, FALSE},
  {"scala",        2, FALSE},
  {"sc",           2, FALSE},
  {"kotlin",       2, FALSE},
  {"kt",           2, FALSE},
  {"kts",          2, FALSE},
  {"groovy",       2, FALSE},
  {"gradle",       2, FALSE},
  {"clj",          2, FALSE},  /* Clojure (;; but // is close enough) */
  {"cljs",         2, FALSE},
  {"cljc",         2, FALSE},
  /* --- .NET --- */
  {"cs",           2, FALSE},
  {"csharp",       2, FALSE},
  {"fs",           2, FALSE},
  {"fsharp",       2, FALSE},
  {"fsx",          2, FALSE},
  {"vb",           2, FALSE},  /* VB.NET uses ' but // is a safe fallback */
  /* --- Systems languages --- */
  {"go",           2, FALSE},
  {"rust",         2, FALSE},
  {"rs",           2, FALSE},
  {"zig",          2, FALSE},
  {"d",            2, FALSE},
  {"v",            2, FALSE},
```

3.3.2 User-Defined Languages

The `@L` command lets users register new language mappings (or override built-in ones) directly in the document preamble. This is useful for languages that are not in the built-in table, or when the default comment style for a language is not what the user wants.

Syntax: `@L` *name style* [`+d`]

- *name* — the fence tag that will trigger this mapping (e.g. `nim`, `glsl`, `wgsl`).
- *style* — one of `//`, `#`, `--`, `/*`, or `<!--`, selecting the comment delimiters.
- `+d` — optional; if present, enables `#line` directive generation for this language.

Examples:

```
@L nim #           % Nim uses # comments
@L glsl // +d      % GLSL uses // and supports #line
@L c //            % override: use // instead of /* */ for C
```

Up to 32 user-defined languages may be registered. They are searched before the built-in table, so a definition like `@L c //` effectively overrides the built-in mapping for C.

The `@L` command is processed during pass1 and silently skipped during weaving. It should appear in the document preamble (before any `@o` or `@d` directives that depend on it).

"names.c" 161≡

```
void collect_lang_def(void)
{
  char name[64];
  char *p = name;
  char style[16];
  char *sp;
  int c;
  unsigned char comment = 0;
  char line_dir = FALSE;

  /* Skip whitespace after @L */
  c = source_get();
  while (c == ' ' || c == '\t')
    c = source_get();

  /* Read language name */
  while (c != EOF && c != ' ' && c != '\t' && c != '\n'
         && (p - name) < 62) {
    *p++ = c;
    c = source_get();
  }
  *p = '\0';

  if (name[0] == '\0') {
    fprintf(stderr, "%s: expected language name after %cL (%s, %d)\n",
            command_name, nw_char, source_name, source_line);
    while (c != EOF && c != '\n') c = source_get();
    return;
  }

  /* Skip whitespace */
  while (c == ' ' || c == '\t')
    c = source_get();

  /* Read comment style token */
  sp = style;
  while (c != EOF && c != ' ' && c != '\t' && c != '\n'
         && (sp - style) < 14) {
    *sp++ = c;
    c = source_get();
  }
  *sp = '\0';

  /* Map style string to comment index */
  if (strcmp(style, "//") == 0)        comment = 2;
  else if (strcmp(style, "#") == 0)    comment = 3;
  else if (strcmp(style, "--") == 0)   comment = 4;
  else if (strcmp(style, "/*") == 0)   comment = 1;
  else if (strcmp(style, "<!--") == 0) comment = 5;
  else {
    fprintf(stderr,
            "%s: unknown comment style '%s' for %cL (%s, %d)\n",
            command_name, style, nw_char, source_name, source_line);
    fprintf(stderr, "  Valid styles: // # -- /* <!--\n");
    while (c != EOF && c != '\n') c = source_get();
    return;
  }

  /* Check for optional +d flag (#line directives) */
  while (c == ' ' || c == '\t')
    c = source_get();
  if (c == '+') {
    c = source_get();
    if (c == 'd') line_dir = TRUE;
  }

  /* Consume rest of line */
  while (c != EOF && c != '\n') c = source_get();

  /* Register the user language */
```

3.3.3 Weave Format Declaration

The `@W` command declares the weave output format in the source file. Valid tokens are `md` (Markdown) and `tex` (LaTeX). If the user has already specified a format on the command line (`-w md` or `-w tex`), the declaration is silently ignored.

"names.c" 162≡

```
void collect_weave_format(void)
{
  char token[64];
  char *p = token;
  int c;

  /* Skip whitespace after @W */
  c = source_get();
  while (c == ' ' || c == '\t')
    c = source_get();

  /* Read format token */
  while (c != EOF && c != ' ' && c != '\t' && c != '\n'
         && (p - token) < 62) {
    *p++ = c;
    c = source_get();
  }
  *p = '\0';

  /* Consume rest of line */
  while (c != EOF && c != '\n') c = source_get();

  if (token[0] == '\0') {
    fprintf(stderr, "%s: expected format after %cW (md or tex) (%s, %d)\n",
            command_name, nw_char, source_name, source_line);
    return;
  }

  /* If CLI already set the format, skip silently */
  if (weave_format_from_cli)
    return;

  if (strcmp(token, "md") == 0)
    weave_format = 2;
  else if (strcmp(token, "tex") == 0)
    weave_format = 1;
  else {
    fprintf(stderr, "%s: unknown weave format '%s' for %cW (%s, %d)\n",
            command_name, token, nw_char, source_name, source_line);
    fprintf(stderr, "  Valid formats: md tex\n");
  }
}
◇
```

File defined by 24c, 153ab, 154a, 155a, 156a, 157, 159, 161, 162, 163, 167, 170ac, 171, 174cd.
Defines: `collect_weave_format` 39c, 152c.
Uses: `command_name` 29a, `fprintf` 21, `nw_char` 28bc, `source_get` 118c, `source_line` 117b, `source_name` 117b, `stderr` 21, `weave_format` 27, `weave_format_from_cli` 27.

3.3.4 Collecting File and Fragment Names

An output file name (`@o`) is terminated by whitespace. After reading the name we check for optional per-file flags (`-d`, `-t`, `-i`, `-c`) and then, if no explicit comment or debug flags were given, apply any pending language defaults from a preceding fence, `%%%` marker, or `@L` definition.

"names.c" 163≡

```
Name *collect_file_name(void)
{
  Name *new_name;
  char name[MAX_NAME_LEN];
  char *p = name;
  int start_line = source_line;
  int c = source_get(), c2;
  while (isspace(c))
    c = source_get();
  while (isgraph(c)) {
    *p++ = c;
    c = source_get();
  }
  if (p == name) {
    fprintf(stderr, "%s: expected file name (%s, %d)\n",
            command_name, source_name, start_line);
    exit(-1);
  }
  *p = '\0';
  /* File names are always global. */
  new_name = name_add(&file_names, name, 0);
  ⟨ Handle optional per-file flags 164a ⟩
  ⟨ Apply pending language if no explicit flags 165a ⟩
  c2 = source_get();
  if (c != nw_char || (c2 != '{' && c2 != '(' && c2 != '[')) {
    fprintf(stderr, "%s: expected %c{, %c[, or %c( after file name (%s, %d)\n",
            command_name, nw_char, nw_char, nw_char, source_name, start_line);
    exit(-1);
  }
  return new_name;
}
◇
```

File defined by 24c, 153ab, 154a, 155a, 156a, 157, 159, 161, 162, 163, 167, 170ac, 171, 174cd.
Defines: `collect_file_name` 42e, 59, 88a, 152c.
Uses: `command_name` 29a, `exit` 21, `file_names` 152a, `fprintf` 21, `isgraph` 21, `isspace` 21, `MAX_NAME_LEN` 22b, `Name` 151b, `name_add` 157, `nw_char` 28bc, `source_get` 118c, `source_line` 117b, `source_name` 117b, `stderr` 21.

⟨ *Handle optional per-file flags* 164a ⟩ ≡

```
{
  while (1) {
    while (isspace(c))
      c = source_get();
    if (c == '-') {
      c = source_get();
      do {
        switch (c) {
          case 't': new_name->tab_flag = FALSE;
                    break;
          case 'd': new_name->debug_flag = TRUE;
                    break;
          case 'i': new_name->indent_flag = FALSE;
                    break;
          case 's': new_name->suppress_markers = TRUE;
                    break;
          case 'c': ⟨ Get comment delimiters 164b ⟩
                    break;
          default : fprintf(stderr, "%s: unexpected per-file flag (%s, %d)\n",
                            command_name, source_name, source_line);
                    break;
        }
        c = source_get();
      } while (!isspace(c));
    }
    else break;
  }
}◇
```

Fragment referenced in 163.
Uses: `command_name` 29a, `FALSE` 22a, `fprintf` 21, `isspace` 21, `source_get` 118c, `source_line` 117b, `source_name` 117b, `stderr` 21, `TRUE` 22a.

The explicit per-file comment flags (`-cc`, `-c+`, `-cp`) select among the first three comment styles. These flags predate the auto-detection system; they still work and take precedence over any fence-based or `@L`-based setting.

The `-s` flag suppresses `#line` directives and section markers in the tangled output for that file, while still allowing language auto-detection to set the comment style and the `lang` tag (used by the Markdown weaver for syntax highlighting classes). This is useful for bootstrap source files or any output where tangle markers are unwanted.

⟨ *Get comment delimiters* 164b ⟩ ≡

```
c = source_get();
if (c == 'c')
   new_name->comment_flag = 1;
else if (c == '+')
   new_name->comment_flag = 2;
else if (c == 'p')
   new_name->comment_flag = 3;
else
   fprintf(stderr, "%s: Unrecognised comment flag (%s, %d)\n",
           command_name, source_name, source_line);
◇
```

Fragment referenced in 164a.
Uses: `command_name` 29a, `fprintf` 21, `source_get` 118c, `source_line` 117b, `source_name` 117b, `stderr` 21.

If there is a pending language tag from a fence or %%% marker, and the user has not specified explicit comment or debug flags, we apply the language defaults automatically.

⟨ *Apply pending language if no explicit flags* 165a ⟩ ≡

```
if (pending_lang[0] != '\0') {
  strncpy(new_name->lang, pending_lang, 63);
  new_name->lang[63] = '\0';
  if (new_name->comment_flag == 0 && !new_name->debug_flag) {
    lang_to_flags(pending_lang,
                  &new_name->comment_flag,
                  &new_name->debug_flag,
                  &new_name->location_flag);
  }
  pending_lang[0] = '\0';
} else if (new_name->lang[0] == '\0') {
  const char *dot = strrchr(new_name->spelling, '.');
  if (dot && dot[1]) {
    const LangEntry *e = find_lang(dot + 1);
    if (e) {
      strncpy(new_name->lang, dot + 1, 63);
      new_name->lang[63] = '\0';
      if (new_name->comment_flag == 0 && !new_name->debug_flag) {
        lang_to_flags(new_name->lang,
                      &new_name->comment_flag,
                      &new_name->debug_flag,
                      &new_name->location_flag);
      }
    }
  }
}
if (new_name->suppress_markers) {
  new_name->debug_flag = FALSE;
  new_name->location_flag = FALSE;
}◇
```

Fragment referenced in 163.
Uses: `FALSE` 22a, `find_lang` 159, `LangEntry` 159, `lang_to_flags` 159, `pending_lang` 152a.

For `@d` (macro) definitions, the language tag is stored but no flags are inferred—macros inherit their flags from the output file that eventually includes them.

⟨ *Apply pending language to macro* 165b ⟩ ≡

```
if (pending_lang[0] != '\0') {
  strncpy(node->lang, pending_lang, 63);
  node->lang[63] = '\0';
  pending_lang[0] = '\0';
}◇
```

Fragment referenced in 168a, 169c.
Uses: `pending_lang` 152a.

3.3.5 Comment Styles

weft supports six comment styles, indexed 0–5. The arrays below provide the opening, continuation, and closing delimiters for each. Style 0 means "no comments"; the remaining five cover the vast majority of programming languages:

Index	Style	Begin	Mid	End
0	none	(empty)	(empty)	(empty)
1	C	`/*`	`*`	`*/`
2	C++	`//`	`//`	(empty)
3	Shell	`#`	`#`	(empty)
4	Lua/SQL	`--`	`--`	(empty)
5	HTML	`<!--`	(spaces)	`-->`

These are used both for block comments (`@c`) and for the Knuth-style section markers generated by the language auto-detection system.

⟨ *Forward declarations for scraps.c* 166 ⟩ ≡

```
char * comment_begin[6] = { "", "/* ", "// ", "# ", "-- ", "<!-- "};
char * comment_mid[6] = { "", " * ", "// ", "# ", "-- ", "     "};
char * comment_end[6] = { "", " */", "", "", "", " -->"};
◇
```

Fragment defined by 139b, 140, 147b, 166, 184c.
Fragment referenced in 124a.
Defines: `comment_begin` 49a, 142cd, 146a, `comment_end` 49a, 142cd, 146a, `comment_mid` 49a.

Name terminated by `\n` or `@{`; but keep skipping until `@{`

"names.c" 167≡

```
Name *collect_macro_name(void)
{
  char name[MAX_NAME_LEN];
  char args[1000];
  char * arg[9];
  char * argp = args;
  int argc = 0;
  char *p = name;
  int start_line = source_line;
  int c = source_get(), c2;
  unsigned char sector = current_sector;

  if (c == '+') {
    sector = 0;
    c = source_get();
  }
  while (isspace(c))
    c = source_get();
  while (c != EOF) {
    Name * node;
    switch (c) {
      case '\t':
      case ' ':  *p++ = ' ';
                 do
                   c = source_get();
                 while (c == ' ' || c == '\t');
                 break;
      case '\n': ⟨ Skip until scrap begins, then return name 169c ⟩
      default:
         if (c==nw_char)
           {
             ⟨ Check for terminating at-sequence and return name 168a ⟩
             break;
           }
         *p++ = c;
                 c = source_get();
                 break;
    }
  }
  fprintf(stderr, "%s: expected fragment name (%s, %d)\n",
          command_name, source_name, start_line);
  exit(-1);
  return NULL;  /* unreachable return to avoid warnings on some compilers */
}
◇
```

File defined by 24c, 153ab, 154a, 155a, 156a, 157, 159, 161, 162, 163, 167, 170ac, 171, 174cd.
Defines: `collect_macro_name` 43a, 60a, 88b, 152c.
Uses: `command_name` 29a, `current_sector` 42c, `exit` 21, `fprintf` 21, `isspace` 21, `MAX_NAME_LEN` 22b, `Name` 151b, `nw_char` 28bc, `sector` 123c, `source_get` 118c, `source_line` 117b, `source_name` 117b, `stderr` 21.

⟨ *Check for terminating at-sequence and return name* 168a ⟩ ≡

```
{
  c = source_get();
  switch (c) {
    case '(':
    case '[':
    case '{': ⟨ Cleanup and install name 169b ⟩
              ⟨ Apply pending language to macro 165b ⟩
              return install_args(node, argc, arg);
    case '\'': ⟨ Enter the next argument 168b ⟩
               break;
    default:
          if (c==nw_char)
            {
              *p++ = c;
              break;
            }
          fprintf(stderr,
                      "%s: unexpected %c%c in fragment definition name (%s, %d)\n",
                      command_name, nw_char, c, source_name, start_line);
              exit(-1);
  }
}◇
```

Fragment referenced in 167.
Uses: `command_name` 29a, `exit` 21, `fprintf` 21, `nw_char` 28bc, `source_get` 118c, `source_name` 117b, `stderr` 21.

⟨ *Enter the next argument* 168b ⟩ ≡

```
arg[argc] = argp;
while ((c = source_get()) != EOF) {
   if (c==nw_char) {
      c2 = source_get();
      if (c2=='\'') {
        ⟨ Make this argument 169a ⟩
        c = source_get();
        break;
      }
      else
        *argp++ = c2;
   }
   else
     *argp++ = c;
}
*p++ = ARG_CHR;
◇
```

Fragment referenced in 168a.
Uses: `ARG_CHR` 168c, `nw_char` 28bc, `source_get` 118c.

⟨ *Type declarations* 168c ⟩ ≡

```
#define ARG_CHR '\001'
◇
```

Fragment defined by 22a, 50c, 114c, 151ab, 168c, 170b, 173d, 185c, 192a.
Fragment referenced in 20.
Defines: `ARG_CHR` 60b, 73, 75, 88c, 97, 99, 137c, 147a, 148a, 168b, 172, 173b.

⟨ *Make this argument* 169a ⟩ ≡

```
if (argc < 9) {
  *argp++ = '\000';
  argc += 1;
}
◇
```

Fragment referenced in 168b.

⟨ *Cleanup and install name* 169b ⟩ ≡

```
{
  if (p > name && p[-1] == ' ')
    p--;
  if (p - name > 3 && p[-1] == '.' && p[-2] == '.' && p[-3] == '.') {
    p[-3] = ' ';
    p -= 2;
  }
  if (p == name || name[0] == ' ') {
    fprintf(stderr, "%s: empty name (%s, %d)\n",
            command_name, source_name, source_line);
    exit(-1);
  }
  *p = '\0';
  node = prefix_add(&macro_names, name, sector);
}◇
```

Fragment referenced in 168a, 169c, 172.
Uses: `command_name` 29a, `exit` 21, `fprintf` 21, `macro_names` 152a, `prefix_add` 154a, `sector` 123c, `source_line` 117b, `source_name` 117b, `stderr` 21.

⟨ *Skip until scrap begins, then return name* 169c ⟩ ≡

```
{
  do
    c = source_get();
  while (isspace(c));
  c2 = source_get();
  if (c != nw_char || (c2 != '{' && c2 != '(' && c2 != '[')) {
    fprintf(stderr, "%s: expected %c{ after fragment name (%s, %d)\n",
            command_name, nw_char, source_name, start_line);
    exit(-1);
  }
  ⟨ Cleanup and install name 169b ⟩
  ⟨ Apply pending language to macro 165b ⟩
  return install_args(node, argc, arg);
}◇
```

Fragment referenced in 167.
Uses: `command_name` 29a, `exit` 21, `fprintf` 21, `isspace` 21, `nw_char` 28bc, `source_get` 118c, `source_name` 117b, `stderr` 21.

⟨ *Function prototypes* 169d ⟩ ≡

```
extern Name *install_args(Name *name, int argc, char *arg[9]);
◇
```

Fragment defined by 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c.
Fragment referenced in 20.
Uses: `Name` 151b.

"names.c" 170a≡

```
Name *install_args(Name * name, int argc, char *arg[9])
{
  int i;

  for (i = 0; i < argc; i++) {
    if (name->arg[i] == NULL)
      name->arg[i] = save_string(arg[i]);
  }
  return name;
}
◇
```

File defined by 24c, 153ab, 154a, 155a, 156a, 157, 159, 161, 162, 163, 167, 170ac, 171, 174cd.
Uses: `Name` 151b, `save_string` 153b.

⟨ *Type declarations* 170b ⟩ ≡

```
typedef struct arglist
{Name * name;
struct arglist * args;
struct arglist * next;
} Arglist;
◇
```

Fragment defined by 22a, 50c, 114c, 151ab, 168c, 170b, 173d, 185c, 192a.
Fragment referenced in 20.
Defines: `Arglist` 50d, 55a, 58b, 73, 75, 84a, 87b, 97, 99, 124b, 133c, 135, 136b, 137abc, 138ab, 139a, 140, 145b, 146c, 147b, 152c, 170c, 171, 173cd, 174b, 176b, 177a, 178a, 184b.
Uses: `Name` 151b, `next` 123b.

"names.c" 170c≡

```
Arglist * buildArglist(Name * name, Arglist * a)
{
  Arglist * args = (Arglist *)arena_getmem(sizeof(Arglist));

  args->args = a;
  args->next = NULL;
  args->name = name;
  return args;
}
◇
```

File defined by 24c, 153ab, 154a, 155a, 156a, 157, 159, 161, 162, 163, 167, 170ac, 171, 174cd.
Defines: `buildArglist` 172, 173c, 174b.
Uses: `arena_getmem` 193b, `Arglist` 170b, `Name` 151b, `next` 123b.

Terminated by `@>`

"names.c" 171≡

```
Arglist * collect_scrap_name(int current_scrap)
{
  char name[MAX_NAME_LEN];
  char *p = name;
  int c = source_get();
  unsigned char sector = current_sector;
  Arglist * head = NULL;
  Arglist ** tail = &head;

  if (c == '+')
  {
    sector = 0;
    c = source_get();
  }
  while (c == ' ' || c == '\t')
    c = source_get();
  while (c != EOF) {
    switch (c) {
      case '\t':
      case ' ':  *p++ = ' ';
                 do
                   c = source_get();
                 while (c == ' ' || c == '\t');
                 break;
      default:
         if (c==nw_char)
           {
             ⟨ Look for end of scrap name and return 172 ⟩
             break;
           }
         if (!isgraph(c)) {
                   fprintf(stderr,
                           "%s: unexpected character in fragment name (%s, %d)\n",
                           command_name, source_name, source_line);
                   exit(-1);
                 }
                 *p++ = c;
                 c = source_get();
                 break;
    }
  }
  fprintf(stderr, "%s: unexpected end of file (%s, %d)\n",
          command_name, source_name, source_line);
  exit(-1);
  return NULL;  /* unreachable return to avoid warnings on some compilers */
}
◇
```

File defined by 24c, 153ab, 154a, 155a, 156a, 157, 159, 161, 162, 163, 167, 170ac, 171, 174cd.
Defines: `collect_scrap_name` 58b, 73, 87b, 97, 133c, 152c, 174a.
Uses: `Arglist` 170b, `command_name` 29a, `current_sector` 42c, `exit` 21, `fprintf` 21, `isgraph` 21, `MAX_NAME_LEN` 22b, `nw_char` 28bc, `sector` 123c, `source_get` 118c, `source_line` 117b, `source_name` 117b, `stderr` 21.

⟨ *Look for end of scrap name and return* 172 ⟩ ≡

```
{
  Name * node;

  c = source_get();
  switch (c) {

    case '\'': {
          ⟨ Add plain string argument 173a ⟩
        }
        *p++ = ARG_CHR;
        c = source_get();
        break;
    case '1': case '2': case '3':
    case '4': case '5': case '6':
    case '7': case '8': case '9': {
          ⟨ Add a propagated argument 173b ⟩
        }
        *p++ = ARG_CHR;
        c = source_get();
        break;
    case '{': {
        ⟨ Add an inline scrap argument 173c ⟩
        }
        *p++ = ARG_CHR;
        c = source_get();
        break;
    case '<':
        ⟨ Add macro call argument 174a ⟩
        *p++ = ARG_CHR;
        c = source_get();
        break;
    case '(':
        scrap_name_has_parameters = 1;
        ⟨ Cleanup and install name 169b ⟩
        return buildArglist(node, head);
    case '>':
        scrap_name_has_parameters = 0;
        ⟨ Cleanup and install name 169b ⟩
        return buildArglist(node, head);

    default:
       if (c==nw_char)
         {
           *p++ = c;
              c = source_get();
              break;
         }
       fprintf(stderr,
                      "%s: unexpected %c%c in fragment invocation name (%s, %d)\n",
                      command_name, nw_char, c, source_name, source_line);
              exit(-1);
  }
}◇
```

Fragment referenced in 171.

Uses: `ARG_CHR` 168c, `buildArglist` 170c, `command_name` 29a, `exit` 21, `fprintf` 21, `Name` 151b, `nw_char` 28bc, `source_get` 118c, `source_line` 117b, `source_name` 117b, `stderr` 21.

A plain string argument has no name and a string as a value.

⟨ *Add plain string argument* 173a ⟩ ≡

```
char buff[MAX_NAME_LEN];
char * s = buff;
int c, c2;

while ((c = source_get()) != EOF) {
  if (c==nw_char) {
    c2 = source_get();
    if (c2=='\'')
      break;
    *s++ = c2;
  }
  else
    *s++ = c;
}
*s = '\000';
⟨ Add buff to current arg list 174b ⟩◇
```

Fragment referenced in 172.
Uses: `MAX_NAME_LEN` 22b, `nw_char` 28bc, `source_get` 118c.

A parameter argument propagated into an interior fragment has no name and and a string of `ARG_CHAR` followed by a digit as value.

⟨ *Add a propagated argument* 173b ⟩ ≡

```
char buff[3];
buff[0] = ARG_CHR;
buff[1] = c;
buff[2] = '\000';
⟨ Add buff to current arg list 174b ⟩◇
```

Fragment referenced in 172.
Uses: `ARG_CHR` 168c.

⟨ *Add an inline scrap argument* 173c ⟩ ≡

```
int s = collect_scrap();
Scrap_Node * d = (Scrap_Node *)arena_getmem(sizeof(Scrap_Node));
d->scrap = s;
d->quoted = 0;
d->next = NULL;
*tail = buildArglist((Name *)1, (Arglist *)d);
tail = &(*tail)->next;◇
```

Fragment referenced in 172.
Uses: `arena_getmem` 193b, `Arglist` 170b, `buildArglist` 170c, `collect_scrap` 128b, `Name` 151b, `next` 123b, `Scrap_Node` 151a.

⟨ *Type declarations* 173d ⟩ ≡

```
typedef struct embed {
   Scrap_Node * defs;
   Arglist * args;
} Embed_Node;
◇
```

Fragment defined by 22a, 50c, 114c, 151ab, 168c, 170b, 173d, 185c, 192a.
Fragment referenced in 20.
Defines: `Embed_Node` 50d, 137c, 147c.
Uses: `Arglist` 170b, `defs` 123c, `Scrap_Node` 151a.

⟨ *Add macro call argument* 174a ⟩ ≡

```
*tail = collect_scrap_name(current_scrap);
if (current_scrap >= 0)
  add_to_use((*tail)->name, current_scrap);
tail = &(*tail)->next;
◇
```

Fragment referenced in 172.
Uses: `add_to_use` 134a, `collect_scrap_name` 171, `next` 123b.

⟨ *Add buff to current arg list* 174b ⟩ ≡

```
*tail = buildArglist(NULL, (Arglist *)save_string(buff));
tail = &(*tail)->next;
◇
```

Fragment referenced in 173ab.
Uses: `Arglist` 170b, `buildArglist` 170c, `next` 123b, `save_string` 153b.

"names.c" 174c≡

```
static Scrap_Node *reverse(Scrap_Node *a); /* a forward declaration */

void reverse_lists(Name *names)
{
  while (names) {
    reverse_lists(names->llink);
    names->defs = reverse(names->defs);
    names->uses = reverse(names->uses);
    names = names->rlink;
  }
}
◇
```

File defined by 24c, 153ab, 154a, 155a, 156a, 157, 159, 161, 162, 163, 167, 170ac, 171, 174cd.
Defines: `reverse_lists` 50b, 152c.
Uses: `defs` 123c, `Name` 151b, `reverse` 174d, `Scrap_Node` 151a, `uses` 123c.

Just for fun, here's a non-recursive version of the traditional list reversal code. Note that it reverses the list in place; that is, it does no new allocations.

"names.c" 174d≡

```
static Scrap_Node *reverse(Scrap_Node *a)
{
  if (a) {
    Scrap_Node *b = a->next;
    a->next = NULL;
    while (b) {
      Scrap_Node *c = b->next;
      b->next = a;
      a = b;
      b = c;
    }
  }
  return a;
}
◇
```

File defined by 24c, 153ab, 154a, 155a, 156a, 157, 159, 161, 162, 163, 167, 170ac, 171, 174cd.
Defines: `reverse` 174c.
Uses: `next` 123b, `Scrap_Node` 151a.

3.4 Searching for Index Entries

Given the array of scraps and a set of index entries, we need to search all the scraps for occurrences of each entry. The obvious approach to this problem would be quite expensive for large documents; however, there is an interesting paper describing an efficient solution [1].

"scraps.c" 175a≡

```
typedef struct name_node {
  struct name_node *next;
  Name *name;
} Name_Node;
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `Name_Node` 175b, 181, 182, 183.
Uses: `Name` 151b, `next` 123b.

"scraps.c" 175b≡

```
typedef struct goto_node {
  Name_Node *output;            /* list of words ending in this state */
  struct move_node *moves;      /* list of possible moves */
  struct goto_node *fail;       /* and where to go when no move fits */
  struct goto_node *next;       /* next goto node with same depth */
} Goto_Node;
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `Goto_Node` 175cd, 176a, 180a, 181, 182, 183.
Uses: `Name_Node` 175a, `next` 123b.

"scraps.c" 175c≡

```
typedef struct move_node {
  struct move_node *next;
  Goto_Node *state;
  char c;
} Move_Node;
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `Move_Node` 176a, 181, 182.
Uses: `Goto_Node` 175b, `next` 123b.

"scraps.c" 175d≡

```
static Goto_Node *root[256];
static int max_depth;
static Goto_Node **depths;
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `depths` 180a, 181, 182, `max_depth` 180a, 181, 182, `root` 180a, 181, 182, 183, 184c, 185b.
Uses: `Goto_Node` 175b.

"scraps.c" 176a≡

```
static Goto_Node *goto_lookup(char c, Goto_Node *g)
{
  Move_Node *m = g->moves;
  while (m && m->c != c)
    m = m->next;
  if (m)
    return m->state;
  else
    return NULL;
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `goto_lookup` 181, 182, 183.
Uses: `Goto_Node` 175b, `Move_Node` 175c, `next` 123b.

3.4.1 Retrieving scrap uses

"scraps.c" 176b≡

```
typedef struct ArgMgr_s
{
   char * pv;
   char * bgn;
   Arglist * arg;
   struct ArgMgr_s * old;
} ArgMgr;
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `ArgMgr` 176c, 177ab, 178c.
Uses: `Arglist` 170b.

"scraps.c" 176c≡

```
typedef struct ArgManager_s
{
   Manager * m;
   ArgMgr * a;
} ArgManager;
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `ArgManager` 177ab, 178c, 180a, 183, 189b.
Uses: `ArgMgr` 176b, `Manager` 127c.

"scraps.c" 177a≡

```
static void
pushArglist(ArgManager * mgr, Arglist * a)
{
   ArgMgr * b = malloc(sizeof(ArgMgr));

   if (b == NULL)
   {
      fprintf(stderr, "Can't allocate space for an argument manager\n");
      exit(EXIT_FAILURE);
   }
   b->pv = b->bgn = NULL;
   b->arg = a;
   b->old = mgr->a;
   mgr->a = b;
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `pushArglist` 178a, 184b.
Uses: `Arglist` 170b, `ArgManager` 176c, `ArgMgr` 176b, `exit` 21, `fprintf` 21, `malloc` 21, `stderr` 21.

"scraps.c" 177b≡

```
static char argpop(ArgManager * mgr)
{
   while (mgr->a != NULL)
   {
      ArgMgr * a = mgr->a;

      ⟨ Perhaps —return— a character from the current arg 177c ⟩
      ⟨ Perhaps start a new arg 178a ⟩
      ⟨ Otherwise pop the current arg 178b ⟩
   }

   return (pop(mgr->m));
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `argpop` 183, 184ab.
Uses: `ArgManager` 176c, `ArgMgr` 176b, `pop` 134c.

We separate individual arguments using spaces.

⟨ *Perhaps —return— a character from the current arg* 177c ⟩ ≡

```
if (a->pv != NULL)
{
   char c = *a->pv++;

   if (c != '\0')
      return c;
   a->pv = NULL;
   return ' ';
}
◇
```

Fragment referenced in 177b.

I'm pretty sure that only the first case is ever used. I don't think the others can occur in this context, which makes the whole —ArgManager— thing doubtful.

⟨ *Perhaps start a new arg* 178a ⟩ ≡

```
if (a->arg) {
   Arglist * b = a->arg;

   a->arg = b->next;
   if (b->name == NULL) {
      a->bgn = a->pv = (char *)b->args;
   } else if (b->name == (Name *)1) {
      a->bgn = a->pv = "{Embedded Scrap}";
   } else {
      pushArglist(mgr, b->args);
   }◇
```

Fragment referenced in 177b.
Uses: `Arglist` 170b, `Name` 151b, `next` 123b, `pushArglist` 177a.

⟨ *Otherwise pop the current arg* 178b ⟩ ≡

```
} else {
   mgr->a = a->old;
   free(a);
}◇
```

Fragment referenced in 177b.

"scraps.c" 178c≡

```
static char
prev_char(ArgManager * mgr, int n)
{
   char c = '\0';
   ArgMgr * a = mgr->a;
   Manager * m = mgr->m;

   if (a != NULL) {
      ⟨ Get the nth previous character from an argument 179a ⟩
   } else {
      ⟨ Get the nth previous character from a scrap 179b ⟩
   }

   return c;
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `prev_char` 189b.
Uses: `ArgManager` 176c, `ArgMgr` 176b, `Manager` 127c.

⟨ *Get the nth previous character from an argument* 179a ⟩ ≡

```
if (a->pv && a->pv - n >= a->bgn)
   c = *a->pv;
else if (a->bgn) {
   int j = strlen(a->bgn) + 1;

   if (n >= j)
      c = a->bgn[j - n];
   else
      c = ' ';
}
◇
```

Fragment referenced in 178c.
Uses: `strlen` 21.

⟨ *Get the nth previous character from a scrap* 179b ⟩ ≡

```
int k = m->index - n - 2;

if (k >= 0)
   c = m->scrap->chars[k];
else if (m->prev)
   c = m->prev->chars[SLAB_SIZE - k];
◇
```

Fragment referenced in 178c.
Uses: `SLAB_SIZE` 123b.

3.4.2 Building the Automata

⟨ *Function prototypes* 179c ⟩ ≡

```
extern void search(void);
◇
```

Fragment defined by 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c.
Fragment referenced in 20.
Uses: `search` 180a.

"scraps.c" 180a≡

```
static void build_gotos(Name *tree);
static int reject_match(Name *name, char post, ArgManager *reader);

void search(void)
{
  int i;
  for (i=0; i<256; i++)
    root[i] = NULL;
  max_depth = 10;
  depths = (Goto_Node **) arena_getmem(max_depth * sizeof(Goto_Node *));
  for (i=0; i<max_depth; i++)
    depths[i] = NULL;
  build_gotos(user_names);
  ⟨ Build failure functions 182 ⟩
  ⟨ Search scraps 183 ⟩
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `search` 38b, 179c, 195.
Uses: `arena_getmem` 193b, `ArgManager` 176c, `build_gotos` 180b, `depths` 175d, `Goto_Node` 175b, `max_depth` 175d, `Name` 151b, `reject_match` 189b, `root` 175d, `user_names` 152a.

"scraps.c" 180b≡

```
static void build_gotos(Name *tree)
{
  while (tree) {
    ⟨ Extend goto graph with tree->spelling 181 ⟩
    build_gotos(tree->rlink);
    tree = tree->llink;
  }
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `build_gotos` 180a.
Uses: `Name` 151b.

⟨ *Extend goto graph with* `tree->spelling` 181 ⟩ ≡

```
{
  int depth = 2;
  char *p = tree->spelling;
  char c = *p++;
  Goto_Node *q = root[(unsigned char)c];
  Name_Node * last;
  if (!q) {
    q = (Goto_Node *) arena_getmem(sizeof(Goto_Node));
    root[(unsigned char)c] = q;
    q->moves = NULL;
    q->fail = NULL;
    q->output = NULL;
    q->next = depths[1];
    depths[1] = q;
  }
  while ((c = *p++)) {
    Goto_Node *new_node = goto_lookup(c, q);
    if (!new_node) {
      Move_Node *new_move = (Move_Node *) arena_getmem(sizeof(Move_Node));
      new_node = (Goto_Node *) arena_getmem(sizeof(Goto_Node));
      new_node->moves = NULL;
      new_node->fail = NULL;
      new_node->output = NULL;
      new_move->state = new_node;
      new_move->c = c;
      new_move->next = q->moves;
      q->moves = new_move;
      if (depth == max_depth) {
        int i;
        Goto_Node **new_depths =
            (Goto_Node **) arena_getmem(2*depth*sizeof(Goto_Node *));
        max_depth = 2 * depth;
        for (i=0; i<depth; i++)
          new_depths[i] = depths[i];
        depths = new_depths;
        for (i=depth; i<max_depth; i++)
          depths[i] = NULL;
      }
      new_node->next = depths[depth];
      depths[depth] = new_node;
    }
    q = new_node;
    depth++;
  }
  last = q->output;
  q->output = (Name_Node *) arena_getmem(sizeof(Name_Node));
  q->output->next = last;
  q->output->name = tree;
}◇
```

Fragment referenced in 180b.

Uses: `arena_getmem` 193b, `depths` 175d, `goto_lookup` 176a, `Goto_Node` 175b, `max_depth` 175d, `Move_Node` 175c, `Name_Node` 175a, `next` 123b, `root` 175d.

⟨ *Build failure functions* 182 ⟩ ≡

```
{
  int depth;
  for (depth=1; depth<max_depth; depth++) {
    Goto_Node *r = depths[depth];
    while (r) {
      Move_Node *m = r->moves;
      while (m) {
        char a = m->c;
        Goto_Node *s = m->state;
        Goto_Node *state = r->fail;
        while (state && !goto_lookup(a, state))
          state = state->fail;
        if (state)
          s->fail = goto_lookup(a, state);
        else
          s->fail = root[(unsigned char)a];
        if (s->fail) {
          Name_Node *p = s->fail->output;
          while (p) {
            Name_Node *q = (Name_Node *) arena_getmem(sizeof(Name_Node));
            q->name = p->name;
            q->next = s->output;
            s->output = q;
            p = p->next;
          }
        }
        m = m->next;
      }
      r = r->next;
    }
  }
}◇
```

Fragment referenced in 180a.

Uses: `arena_getmem` 193b, `depths` 175d, `goto_lookup` 176a, `Goto_Node` 175b, `max_depth` 175d, `Move_Node` 175c, `Name_Node` 175a, `next` 123b, `root` 175d.

3.4.3 Searching the Scraps

⟨ *Search scraps* 183 ⟩ ≡

```
{
  for (i=1; i<scraps; i++) {
    char c, last = '\0';
    Manager rd;
    ArgManager reader;
    Goto_Node *state = NULL;
    rd.prev = NULL;
    rd.scrap = scrap_array(i).slab;
    rd.index = 0;
    reader.m = &rd;
    reader.a = NULL;
    c = argpop(&reader);
    while (c) {
      while (state && !goto_lookup(c, state))
        state = state->fail;
      if (state)
        state = goto_lookup(c, state);
      else
        state = root[(unsigned char)c];
      ⟨ Skip over at at 184a ⟩
      ⟨ Skip over a scrap use 184b ⟩
      ⟨ Skip over a block comment 49d ⟩
      last = c;
      c = argpop(&reader);
      if (state && state->output) {
        Name_Node *p = state->output;
        do {
          Name *name = p->name;
          if (!reject_match(name, c, &reader) &&
              scrap_array(i).sector == name->sector &&
              (!name->uses || name->uses->scrap != i)) {
            Scrap_Node *new_use =
                (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
            new_use->scrap = i;
            new_use->next = name->uses;
            name->uses = new_use;
            if (!scrap_is_in(name->defs, i))
              add_uses(&(scrap_array(i).uses), name);
          }
          p = p->next;
        } while (p);
      }
    }
  }
}◇
```

Fragment referenced in 180a.

Uses: `add_uses` 185b, `arena_getmem` 193b, `ArgManager` 176c, `argpop` 177b, `defs` 123c, `goto_lookup` 176a, `Goto_Node` 175b, `Manager` 127c, `Name` 151b, `Name_Node` 175a, `next` 123b, `reject_match` 189b, `root` 175d, `scraps` 124a, `scrap_array` 124a, `scrap_is_in` 185a, `Scrap_Node` 151a, `sector` 123c, `slab` 123c, `uses` 123c.

⟨ *Skip over at at* 184a ⟩ ≡

```
if (last == nw_char && c == nw_char)
{
   last = '\0';
   c = argpop(&reader);
}
◇
```

Fragment referenced in 183.
Uses: `argpop` 177b, `nw_char` 28bc.

⟨ *Skip over a scrap use* 184b ⟩ ≡

```
if (last == nw_char && c == '<')
{
   char buf[MAX_NAME_LEN];
   char * p = buf;
   Arglist * args;
   int idx;

   c = argpop(&reader);
   while ((c = argpop(&reader)) != nw_char)
      *p++ = c;
   c = argpop(&reader);
   *p = '\0';
   if (sscanf(buf, "%d", &idx) != 1) {
      fprintf(stderr,  "%s: found an internal problem (3)\n", command_name);
      exit(-1);
   }
   args = arglist_lookup(idx);
   pushArglist(&reader, args);
}◇
```

Fragment referenced in 183.
Uses: `Arglist` 170b, `arglist_lookup` 140, `argpop` 177b, `command_name` 29a, `exit` 21, `fprintf` 21, `MAX_NAME_LEN` 22b, `nw_char` 28bc, `pushArglist` 177a, `stderr` 21.

⟨ *Forward declarations for scraps.c* 184c ⟩ ≡

```
static void add_uses(Uses **root, Name *name);
static int scrap_is_in(Scrap_Node *list, int i);
◇
```

Fragment defined by 139b, 140, 147b, 166, 184c.
Fragment referenced in 124a.
Uses: `add_uses` 185b, `Name` 151b, `root` 175d, `scrap_is_in` 185a, `Scrap_Node` 151a, `Uses` 185c.

"scraps.c" 185a≡

```
static int scrap_is_in(Scrap_Node * list, int i)
{
  while (list != NULL) {
    if (list->scrap == i)
      return TRUE;
    list = list->next;
  }
  return FALSE;
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `scrap_is_in` 183, 184c.
Uses: `FALSE` 22a, `next` 123b, `Scrap_Node` 151a, `TRUE` 22a.

"scraps.c" 185b≡

```
static void add_uses(Uses * * root, Name *name)
{
   int cmp;
   Uses *p, **q = root;

   while ((p = *q, p != NULL)
          && (cmp = robs_strcmp(p->defn->spelling, name->spelling)) < 0)
      q = &(p->next);
   if (p == NULL || cmp > 0)
   {
      Uses *new_use = arena_getmem(sizeof(Uses));
      new_use->next = p;
      new_use->defn = name;
      *q = new_use;
   }
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `add_uses` 133ab, 183, 184c.
Uses: `arena_getmem` 193b, `Name` 151b, `next` 123b, `robs_strcmp` 156a, `root` 175d, `Uses` 185c.

⟨ *Type declarations* 185c ⟩ ≡

```
typedef struct uses {
  struct uses *next;
  Name *defn;
} Uses;
◇
```

Fragment defined by 22a, 50c, 114c, 151ab, 168c, 170b, 173d, 185c, 192a.
Fragment referenced in 20.
Defines: `Uses` 56a, 106ab, 107b, 110, 112, 124b, 125c, 126a, 184c, 185b, 186b, 187d.
Uses: `Name` 151b, `next` 123b, `uses` 123c.

⟨ *Function prototypes* 186a ⟩ ≡

```
extern void format_uses_refs(FILE *, int);
◇
```

Fragment defined by 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c.
Fragment referenced in 20.
Uses: `FILE` 21, `format_uses_refs` 186b.

"scraps.c" 186b≡

```
void
format_uses_refs(FILE * tex_file, int scrap)
{
  Uses * p = scrap_array(scrap).uses;
  if (p != NULL)
    ⟨ Write uses references 186c ⟩
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `format_uses_refs` 59, 60a, 186a.
Uses: `FILE` 21, `scrap_array` 124a, `Uses` 185c, `uses` 123c.

⟨ *Write uses references* 186c ⟩ ≡

```
{
  char join = ' ';
  fputs("\\item \\NWtxtIdentsUsed\\nobreak\\", tex_file);
  do {
    ⟨ Write one use reference 187a ⟩
    join = ',';
    p = p->next;
  }while (p != NULL);
  fputs(".", tex_file);
}◇
```

Fragment referenced in 186b.
Uses: `fputs` 21, `next` 123b.

⟨ *Write one use reference* 187a ⟩ ≡

```
Name * name = p->defn;
Scrap_Node *defs = name->defs;
int first = TRUE, page = -1;
fprintf(tex_file,
        "%c \\verb%c%s%c\\nobreak\\ ",
        join, nw_char, name->spelling, nw_char);
if (defs)
{
  do {
    ⟨ Write one referenced scrap 187b ⟩
    first = FALSE;
    defs = defs->next;
  }while (defs!= NULL);
}
else
{
  fputs("\\NWnotglobal", tex_file);
}
◇
```

Fragment referenced in 186c.
Uses: `defs` 123c, `FALSE` 22a, `first` 193a, `fprintf` 21, `fputs` 21, `Name` 151b, `next` 123b, `nw_char` 28bc, `page` 123c, `Scrap_Node` 151a, `TRUE` 22a.

⟨ *Write one referenced scrap* 187b ⟩ ≡

```
fputs("\\NWlink{weft", tex_file);
write_scrap_ref(tex_file, defs->scrap, -1, &page);
fputs("}{", tex_file);
write_scrap_ref(tex_file, defs->scrap, first, &page);
fputs("}", tex_file);◇
```

Fragment referenced in 187a, 188b.
Uses: `defs` 123c, `first` 193a, `fputs` 21, `page` 123c, `write_scrap_ref` 125a.

⟨ *Function prototypes* 187c ⟩ ≡

```
extern void format_defs_refs(FILE *, int);
◇
```

Fragment defined by 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c.
Fragment referenced in 20.
Uses: `FILE` 21, `format_defs_refs` 187d.

"scraps.c" 187d≡

```
void
format_defs_refs(FILE * tex_file, int scrap)
{
  Uses * p = scrap_array(scrap).defs;
  if (p != NULL)
    ⟨ Write defs references 188a ⟩
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `format_defs_refs` 59, 60a, 187c.
Uses: `defs` 123c, `FILE` 21, `scrap_array` 124a, `Uses` 185c.

⟨ *Write defs references* 188a ⟩ ≡

```
{
  char join = ' ';
  fputs("\\item \\NWtxtIdentsDefed\\nobreak\\", tex_file);
  do {
    ⟨ Write one def reference 188b ⟩
    join = ',';
    p = p->next;
  }while (p != NULL);
  fputs(".", tex_file);
}◇
```

Fragment referenced in 187d.
Uses: `fputs` 21, `next` 123b.

⟨ *Write one def reference* 188b ⟩ ≡

```
Name * name = p->defn;
Scrap_Node *defs = name->uses;
int first = TRUE, page = -1;
fprintf(tex_file,
        "%c \\verb%c%s%c\\nobreak\\ ",
        join, nw_char, name->spelling, nw_char);
if (defs == NULL
    || (defs->scrap == scrap && defs->next == NULL)) {
  fputs("\\NWtxtIdentsNotUsed", tex_file);
}
else {
  do {
    if (defs->scrap != scrap) {
      ⟨ Write one referenced scrap 187b ⟩
      first = FALSE;
    }
    defs = defs->next;
  }while (defs!= NULL);
}
◇
```

Fragment referenced in 188a.
Uses: `defs` 123c, `FALSE` 22a, `first` 193a, `fprintf` 21, `fputs` 21, `Name` 151b, `next` 123b, `nw_char` 28bc, `page` 123c, `Scrap_Node` 151a, `TRUE` 22a, `uses` 123c.

Rejecting Matches

A problem with simple substring matching is that the string "he" would match longer strings like "she" and "her." Norman Ramsey suggested examining the characters occurring immediately before and after a match and rejecting the match if it appears to be part of a longer token. Of course, the concept of *token* is language-dependent, so we may be occasionally mistaken. For the present, we'll consider the mechanism an experiment.

"scraps.c" 189a≡

```
#define sym_char(c) (isalnum(c) || (c) == '_')

static int op_char(char c)
{
  switch (c) {
    case '!':           case '#': case '%': case '$': case '^':
    case '&': case '*': case '-': case '+': case '=': case '/':
    case '|': case '~': case '<': case '>':
      return TRUE;
    default:
      return c==nw_char ? TRUE : FALSE;
  }
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `op_char` 189b, `sym_char` 189b.
Uses: `FALSE` 22a, `nw_char` 28bc, `TRUE` 22a.

"scraps.c" 189b≡

```
static int reject_match(Name *name, char post, ArgManager *reader)
{
  int len = strlen(name->spelling);
  char first = name->spelling[0];
  char last = name->spelling[len - 1];
  char prev = prev_char(reader, len);
  if (sym_char(last) && sym_char(post)) return TRUE;
  if (sym_char(first) && sym_char(prev)) return TRUE;
  if (op_char(last) && op_char(post)) return TRUE;
  if (op_char(first) && op_char(prev)) return TRUE;
  return FALSE; /* Here is 189b-01 */
}
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `reject_match` 180a, 183.
Uses: `ArgManager` 176c, `FALSE` 22a, `first` 193a, `Name` 151b, `op_char` 189a, `prev_char` 178c, `strlen` 21, `sym_char` 189a, `TRUE` 22a.

3.5 Labels

Refer to 189c-01. And another one 189b-01.

⟨ *Get label from* 189c ⟩ ≡

```
char  label_name[MAX_NAME_LEN];
char * p = label_name;
while (c = @1, c != nw_char) /* Here is 189c-01 */
   *p++ = c;
*p = '\0';
c = @1;
◇
```

Fragment referenced in 71b, 131a, 144b.
Uses: `MAX_NAME_LEN` 22b, `nw_char` 28bc.

`"scraps.c"` 190a≡

```
void
write_label(char label_name[], FILE * file)
⟨ Search for label(⟨ Write the label to file 190c ⟩,⟨ Complain about missing label 190d ⟩) 191c ⟩
◇
```

File defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
Defines: `write_label` 71b, 144b, 190b.
Uses: `FILE` 21.

⟨ *Function prototypes* 190b ⟩ ≡

```
void write_label(char label_name[], FILE * file);
◇
```

Fragment defined by 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c.
Fragment referenced in 20.
Uses: `FILE` 21, `write_label` 190a.

⟨ *Write the label to file* 190c ⟩ ≡

```
write_single_scrap_ref(file, lbl->scrap);
fprintf(file, "-%02d", lbl->seq);◇
```

Fragment referenced in 190a.
Uses: `fprintf` 21, `write_single_scrap_ref` 125b.

⟨ *Complain about missing label* 190d ⟩ ≡

```
fprintf(stderr, "Can't find label %s.\n", label_name);◇
```

Fragment referenced in 190a.
Uses: `fprintf` 21, `stderr` 21.

⟨ *Save label to label store* 190e ⟩ ≡

```
if (label_name[0])
⟨ Search for label(⟨ Complain about duplicate labels 191a ⟩,⟨ Create a new label entry 190f ⟩) 191c ⟩
else
{
   ⟨ Complain about empty label 191b ⟩
}◇
```

Fragment referenced in 131a.

⟨ *Create a new label entry* 190f ⟩ ≡

```
lbl = (label_node *)arena_getmem(sizeof(label_node) + (p - label_name));
lbl->left = lbl->right = NULL;
strcpy(lbl->name, label_name);
lbl->scrap = current_scrap;
lbl->seq = ++lblseq;
*plbl = lbl;◇
```

Fragment referenced in 190e.
Uses: `arena_getmem` 193b, `label_node` 192a.

⟨ *Complain about duplicate labels* 191a ⟩ ≡

```
fprintf(stderr, "Duplicate label %s.\n", label_name);◇
```

Fragment referenced in 190e.
Uses: `fprintf` 21, `stderr` 21.

⟨ *Complain about empty label* 191b ⟩ ≡

```
fprintf(stderr, "Empty label.\n");◇
```

Fragment referenced in 190e.
Uses: `fprintf` 21, `stderr` 21.

⟨ *Search for label(Found,Notfound)* 191c ⟩ ≡

```
{
   label_node * * plbl = &label_tab;
   for (;;)
   {
      label_node * lbl = *plbl;

      if (lbl)
      {
         int cmp = label_name[0] - lbl->name[0];

         if (cmp == 0)
            cmp = strcmp(label_name + 1, lbl->name + 1);
         if (cmp < 0)
            plbl = &lbl->left;
         else if (cmp > 0)
            plbl = &lbl->right;
         else
         {
            Found
            break;
         }
      }
      else
      {
          Notfound
          break;
      }
   }
}
◇
```

Fragment referenced in 190ae.
Uses: `label_node` 192a, `label_tab` 191d.

"global.c" 191d≡

```
label_node * label_tab = NULL;
◇
```

File defined by 24e, 191d.
Defines: `label_tab` 191c, 192b.
Uses: `label_node` 192a.

⟨ *Type declarations* 192a ⟩ ≡

```
typedef struct l_node
{
   struct l_node * left, * right;
   int scrap, seq;
   char name[1];
} label_node;
◇
```

Fragment defined by 22a, 50c, 114c, 151ab, 168c, 170b, 173d, 185c, 192a.
Fragment referenced in 20.
Defines: `label_node` 190f, 191cd, 192b.

⟨ *Global variable declarations* 192b ⟩ ≡

```
extern label_node * label_tab;
◇
```

Fragment defined by 27, 28b, 29a, 42c, 47c, 63b, 117b, 127a, 152a, 192b.
Fragment referenced in 20.
Uses: `label_node` 192a, `label_tab` 191d.

3.6 Memory Management

I manage memory using a simple scheme inspired by Hanson's idea of *arenas* [2]. Basically, I allocate all the storage required when processing a source file (primarily for names and scraps) using calls to `arena_getmem(n)`, where `n` specifies the number of bytes to be allocated. When the storage is no longer required, the entire arena is freed with a single call to `arena_free()`. Both operations are quite fast.

⟨ *Function prototypes* 192c ⟩ ≡

```
extern void *arena_getmem(size_t n);
extern void arena_free(void);
extern void arglist_reset(void);
◇
```

Fragment defined by 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c.
Fragment referenced in 20.
Uses: `arena_free` 194b, `arena_getmem` 193b, `arglist_reset` 140, `size_t` 21.

"arena.c" 192d≡

```
typedef struct chunk {
  struct chunk *next;
  char *limit;
  char *avail;
} Chunk;
◇
```

File defined by 24d, 192d, 193ab, 194b.
Defines: `Chunk` 193ac, 194a.
Uses: `next` 123b.

We define an empty chunk called `first`. The variable `arena` points at the current chunk of memory; it's initially pointed at `first`. As soon as some storage is required, a "real" chunk of memory will be allocated and attached to `first->next`; storage will be allocated from the new chunk (and later chunks if necessary).

"arena.c" 193a≡

```
static Chunk first = { NULL, NULL, NULL };
static Chunk *arena = &first;
◇
```

File defined by 24d, 192d, 193ab, 194b.
Defines: `arena` 193bc, 194ab, `first` 106c, 107a, 108, 110, 111a, 113b, 124b, 125a, 187ab, 188b, 189b, 194b, 195.
Uses: `Chunk` 192d.

3.6.1 Allocating Memory

The routine `arena_getmem(n)` returns a pointer to (at least) `n` bytes of memory. Note that `n` is rounded up to ensure that returned pointers are always aligned. We align to the nearest 8-byte segment, since that'll satisfy the more common 2-byte and 4-byte alignment restrictions too.

"arena.c" 193b≡

```
void *arena_getmem(size_t n)
{
  char *q;
  char *p = arena->avail;
  n = (n + 7) & ~7;             /* ensuring alignment to 8 bytes */
  q = p + n;
  if (q <= arena->limit) {
    arena->avail = q;
    return p;
  }
  ⟨ Find a new chunk of memory 193c ⟩
}
◇
```

File defined by 24d, 192d, 193ab, 194b.
Defines: `arena_getmem` 31b, 36, 43b, 53, 124c, 127d, 128c, 133ab, 134a, 137c, 138a, 153b, 158, 170c, 173c, 180a, 181, 182, 183, 185b, 190f, 192c.
Uses: `arena` 193a, `size_t` 21.

If the current chunk doesn't have adequate space (at least `n` bytes) we examine the rest of the list of chunks (starting at `arena->next`) looking for a chunk with adequate space. If `n` is very large, we may not find it right away or we may not find a suitable chunk at all.

⟨ *Find a new chunk of memory* 193c ⟩ ≡

```
{
  Chunk *ap = arena;
  Chunk *np = ap->next;
  while (np) {
    char *v = sizeof(Chunk) + (char *) np;
    if (v + n <= np->limit) {
      np->avail = v + n;
      arena = np;
      return v;
    }
    ap = np;
    np = ap->next;
  }
  ⟨ Allocate a new chunk of memory 194a ⟩
}◇
```

Fragment referenced in 193b.
Uses: `arena` 193a, `Chunk` 192d, `next` 123b.

If there isn't a suitable chunk of memory on the free list, then we need to allocate a new one.

⟨ *Allocate a new chunk of memory* 194a ⟩ ≡

```
{
  size_t m = n + 10000;
  np = (Chunk *) malloc(m);
  np->limit = m + (char *) np;
  np->avail = n + sizeof(Chunk) + (char *) np;
  np->next = NULL;
  ap->next = np;
  arena = np;
  return sizeof(Chunk) + (char *) np;
}◇
```

Fragment referenced in 193c.
Uses: `arena` 193a, `Chunk` 192d, `malloc` 21, `next` 123b, `size_t` 21.

3.6.2 Freeing Memory

To free all the memory in the arena, we need only point `arena` back to the first empty chunk.

"`arena.c`" 194b≡

```
void arena_free(void)
{
  arena = &first;
  arglist_reset();
}
◇
```

File defined by 24d, 192d, 193ab, 194b.
Defines: `arena_free` 37, 192c.
Uses: `arena` 193a, `arglist_reset` 140, `first` 193a.

Chapter 4

Man page

Here is the UNIX man page for weft:

"weft.1" 195≡

```
.TH WEFT 1 "March 2026"
.SH NAME
weft, a literate programming tool
.SH SYNOPSIS
.B weft
.br
\fBweft\fP [options] [file] ...
.SH DESCRIPTION
.I weft
is a literate programming tool like Knuth's
.I WEB,
only simpler.
A
.I weft
file contains program source code interleaved with documentation.
When
.I weft
is given a
.I weft
file, it writes the program file(s).
By default only tangling is performed. Weaving (documentation
generation) is requested with \fB-w\fP.
.SH COMMAND LINE OPTIONS
.br
\fB-t\fP Suppresses weave output (even if \fB-w\fP is given).
.br
\fB-o\fP Suppresses generation of the output files.
.br
\fB-d\fP List dangling identifier references in indexes.
.br
\fB-c\fP Forces output files to overwrite old files of the same
  name without comparing for equality first.
.br
\fB-v\fP The verbose flag. Forces output of progress reports.
.br
\fB-n\fP Forces sequential numbering of scraps (instead of page
  numbers).
.br
\fB-s\fP Doesn't print list of scraps making up file at end of
```

```
  each scrap.
\fB-p path\fP Prepend path to the filenames for all the output files.
\fB-V string\fP Provide the string for replacement of the @v
operation. This is intended as a means for including version
information in generated output.
\fB-x\fP Include cross-references in comments in output files.
\fB-h options\fP Turn on hyperlinks using the hyperref package of
LaTeX and provide the options to the package.
\fB-r\fP Turn on hyperlinks using the hyperref package of
LaTeX, with the package options being in the text.
\fB-I path\fP Provide a directory to search for included files. This
may appear several times.
.br
\fB-m\fP Emit a JSON map of all output files, fragments, their definitions,
and dependency relationships to stdout. No weave or tangle is performed.
Useful for AI tooling and project navigation.
.br
\fB-e name\fP Extract the named fragment and all its transitive
dependencies as Markdown to stdout. Fragment names support prefix
matching. No weave or tangle is performed.
.br
\fB-w [format]\fP Request weave output. Without a format token,
the format is taken from the \fB@W\fP directive in the source file.
With \fBmd\fP, a \fB.md\fP file is generated; with \fBtex\fP,
a \fB.tex\fP file is generated. A CLI format overrides \fB@W\fP.
If \fB-w\fP is given without a format and no \fB@W\fP exists,
weft exits with an error.

.SH FORMAT OF WEFT FILES
A
.I weft
file contains mostly ordinary
.I LaTeX.
The file is read and copied to output (.tex file) unless a
.I weft
command is encountered. All
.I weft
commands start with an ``at-sign'' (@).
Files and fragments are defined with the following commands:
.PP
@o \fIfile-name flags  scrap\fP  where scrap is smaller than one page.
.br
@O \fIfile-name flags  scrap\fP  where scrap is bigger than one page.
.br
@d \fIfragment-name scrap\fP. Where scrap is smaller than one page.
.br
@D \fIfragment-name scrap\fP. Where scrap is bigger than one page.
.PP
@q \fIfragment-name scrap\fP. Where scrap is smaller than one page.
The scrap is not expanded in the output, allowing you to construct
output files which can, perhaps after further processing, be input to
.I weft.
.br
@Q \fIfragment-name scrap\fP. Where scrap is bigger than one page.
Likewise.
.PP
Scraps have specific begin and end
markers;
```

```
which begin and end marker you use determines how the scrap will be
typeset in the .tex file:
.br
\fB@{\fP...\fB@}\fP for verbatim "terminal style" formatting or,
with the -l flag, LaTeX listing package.
.br
\fB@[\fP...\fB@]\fP for LaTeX paragraph mode formatting, and
.br
\fB@(\fP...\fB@)\fP for LaTeX math mode formatting.
.br
Any amount of whitespace
(including carriage returns) may appear between a name and the
beginning of a scrap.
.PP
Several code/file scraps may have the same name;
.I weft
concatenates their definitions to produce a single scrap.
Code scrap definitions are like macro definitions;
.I weft
extracts a program by expanding one scrap.
The definition of that scrap contains references to other scraps, which are
themselves expanded, and so on.
\fIweft\fP's output is readable; it preserves the indentation of expanded
scraps with respect to the scraps in which they appear.
.PP
.SH PER FILE OPTIONS
When defining an output file, the programmer has the option of using flags
to control the output.
.PP
\fB-d\fR option,
.I weft
will emit line number indications at scrap boundaries.
.br
\fB-i\fR option,
.I weft
supresses the indentation of fragments (useful for \fBFortran\fR).
.br
\fB-t\fP option makes \fIweft\fP
copy tabs untouched from input to output.
.br
\fB-c\fIx\fP Include comments in the output file.
\fIx\fP may be \fBc\fP for C-style comments, \fB+\fP for C++ and
\fBp\fP for perl and similar.
.br
\fB-s\fP Suppress section markers and \fB#line\fP directives in the
tangled output for this file.  Language auto-detection still sets
the comment style and the \fBlang\fP tag (used for Markdown syntax
highlighting), but no markers appear in the output.
.PP
.SH MINOR COMMANDS
.br
@@    Causes a single ``at-sign'' to be copied into the output.
.br
@\_    Causes the text between it and the next {\tt @\_} to be made bold
         (for keywords, etc.) in the formatted document
.br
@%     Comments out a line so that it doesn't appear in the output.
.br
```

```
@i     \fBfilename\fR causes the file named to be included.
.br
@f     Creates an index of output files.
.br
@m     Creates an index of fragments.
.br
@u     Creates an index of user-specified identifiers.
.br
@W \fIformat\fP Declares the weave output format (\fBmd\fP or \fBtex\fP).
Used by \fB-w\fP when no explicit format is given on the command line.
.br
@l \fItag\fP Sets the language for the next @o or @d directive.
The tag is looked up in the language table to determine the comment
style for section markers and the \fBclass\fP attribute for Markdown
syntax highlighting.  Place it on the line before the directive it
applies to.
.br
@L \fIname style\fP [+d] Defines a custom language mapping.
\fIname\fP is the tag (e.g. \fBnim\fP, \fBglsl\fP),
\fIstyle\fP is one of \fB//\fP, \fB#\fP, \fB--\fP, \fB/*\fP, or \fB<!--\fP.
The optional \fB+d\fP flag enables \fB#line\fP directive generation.
User-defined languages override built-in entries.  Must appear in the
document preamble before any @o or @d that depends on it.
.PP
.SH SECTION MARKERS
By default, when a language is detected (via @l, @L, or file extension
of @o), weft wraps each scrap in the tangled output with opening and
closing section markers using the appropriate comment syntax.  For example,
C output includes:
.nf

  /* {1: source.w:10} */
  int main() { return 0; }
  /* {:1} */

.fi
The opening marker shows the scrap number and source location; the closing
marker shows only the scrap number.  Nested fragment expansions produce
nested markers.
.PP
Languages with \fB#line\fP support (C, C++, Objective-C, etc.) also
emit \fB#line\fP directives at scrap boundaries.
.PP
The \fB-s\fP per-file flag suppresses both section markers and \fB#line\fP
directives.
.PP
.SH LANGUAGE INFERENCE
When no @l tag precedes an @o directive, weft infers the language
from the output file's extension.  For example, \fB@o app.js\fP
automatically detects JavaScript (// comments, no #line).  The inferred
language is used for section markers in the tangle and for the
\fBclass="language-X"\fP attribute in the Markdown weave.
.PP
Over 250 file extensions are recognised, covering C, C++, Python,
JavaScript, Go, Rust, Ruby, Lua, SQL, HTML, and many more.
Extensions not in the table produce no markers.
.PP
To mark an identifier for inclusion in the index, it must be mentioned
```

```
at the end of the scrap it was defined in. The line starts
with @| and ends with the \fBend of scrap\fP mark \fB@}\fP.
.PP
.SH ERROR MESSAGES
.PP
.SH BUGS
.PP
.SH AUTHOR
Preston Briggs.
Internet address \fBpreston@cs.rice.edu\fP.
.SH MAINTAINER
Simon Wright
Internet address \fBsimon@pushface.org\fP
.br
Keith Harwood
Internet address \fBKeith.Harwood@vitalmis.com\fP
◇
```

Uses: FILE 21, first 193a, main 25a, next 123b, page 123c, scraps 124a, search 180a.

Chapter 5

Indices

Three sets of indices can be created automatically: an index of file names, an index of fragment names, and an index of user-specified identifiers. An index entry includes the name of the entry, where it was defined, and where it was referenced.

5.1 Files

"arena.c" Defined by 24d, 192d, 193ab, 194b.
"global.c" Defined by 24e, 191d.
"global.h" Defined by 20.
"input.c" Defined by 24a, 118abc, 119b, 123a.
"latex.c" Defined by 22e, 55ab, 61a, 65b, 66ab, 67bc, 68, 69a, 75, 76b, 78ab, 81b, 82a.
"main.c" Defined by 22c, 25a.
"markdown.c" Defined by 23a, 84ab, 89a, 92, 93a, 98, 99, 100b, 101b, 102a, 104b, 106a, 107b.
"names.c" Defined by 24c, 153ab, 154a, 155a, 156a, 157, 159, 161, 162, 163, 167, 170ac, 171, 174cd.
"output.c" Defined by 23b, 109, 110, 111ab, 112, 113ab, 114b.
"pass1.c" Defined by 22d, 38b.
"scraps.c" Defined by 24b, 123bc, 124ac, 125abc, 126acd, 127cd, 128ab, 134ac, 135, 136ab, 137a, 138b, 139a, 149a, 175abcd, 176abc, 177ab, 178c, 180ab, 185ab, 186b, 187d, 189ab, 190a.
"weft.1" Defined by 195.

5.2 Fragments

⟨ Accumulate scrap and return scraps++ 129 ⟩ Referenced in 128b.
⟨ Add a propagated argument 173b ⟩ Referenced in 172.
⟨ Add an at character to the block or break 46a ⟩ Referenced in 45b.
⟨ Add an inline scrap argument 173c ⟩ Referenced in 172.
⟨ Add any other character to the block 46b ⟩ Referenced in 45b.
⟨ Add buff to current arg list 174b ⟩ Referenced in 173ab.
⟨ Add letters to scraps with duplicate page numbers 150a ⟩ Referenced in 149a.
⟨ Add macro call argument 174a ⟩ Referenced in 172.
⟨ Add more indentation *char* 142a ⟩ Referenced in 141, 143d, 144a.
⟨ Add one char to the block buffer 45b ⟩ Referenced in 44b.
⟨ Add plain string argument 173a ⟩ Referenced in 172.
⟨ Add the source path to the include path list 36 ⟩ Referenced in 35.
⟨ Add user identifier definition 133b ⟩ Referenced in 132.
⟨ Add user identifier use 133a ⟩ Referenced in 132.
⟨ Add scrap to name's definition list 43b ⟩ Referenced in 42e, 43a.
⟨ Allocate a new chunk of memory 194a ⟩ Referenced in 193c.
⟨ Apply pending language if no explicit flags 165a ⟩ Referenced in 163.
⟨ Apply pending language to macro 165b ⟩ Referenced in 168a, 169c.

⟨ Begin or end a block comment 50a ⟩ Referenced in 57a.
⟨ Begin the cross-reference environment 64a ⟩ Referenced in 59, 60a.
⟨ Begin the scrap environment 61b ⟩ Referenced in 59, 60a.
⟨ Bold Keyword 72a ⟩ Referenced in 70a.
⟨ Bold md Keyword 96b ⟩ Referenced in 95a.
⟨ Build a new arglist 138a ⟩ Referenced in 137a.
⟨ Build failure functions 182 ⟩ Referenced in 180a.
⟨ Build fragment definition 43a ⟩ Referenced in 41.
⟨ Build output file definition 42e ⟩ Referenced in 41.
⟨ Build `source_name` and `tex_name` 35 ⟩ Referenced in 34.
⟨ Check at-sequence for end-of-scrap 70a ⟩ Referenced in 68.
⟨ Check for ambiguous prefix 154b ⟩ Referenced in 154a.
⟨ Check for cosmetic fence line 85b ⟩ Referenced in 85a.
⟨ Check for end of scrap name 138c ⟩ Referenced in 138b.
⟨ Check for macro invocation in scrap 144a ⟩ Referenced in 141.
⟨ Check for macro parameters 53 ⟩ Referenced in 138c.
⟨ Check for terminating at-sequence and return name 168a ⟩ Referenced in 167.
⟨ Check md at-sequence for end-of-scrap 95a ⟩ Referenced in 93a.
⟨ Cleanup and install name 169b ⟩ Referenced in 168a, 169c, 172.
⟨ Close the current sector 42b ⟩ Referenced in 41, 57a, 86a.
⟨ Collect a block comment 44b ⟩ Referenced in 41.
⟨ Collect include-file name 122a ⟩ Referenced in 121.
⟨ Collect language apply in pass1 39a ⟩ Referenced in 41.
⟨ Collect user language definition in pass1 39b ⟩ Referenced in 41.
⟨ Collect user-specified index entries 132 ⟩ Referenced in 130.
⟨ Collect weave format declaration in pass1 39c ⟩ Referenced in 41.
⟨ Comment this macro use 147a ⟩ Referenced in 145b, 146ac.
⟨ Compare the temp file and the old file 116b ⟩ Referenced in 116a.
⟨ Complain about duplicate labels 191a ⟩ Referenced in 190e.
⟨ Complain about empty label 191b ⟩ Referenced in 190e.
⟨ Complain about missing label 190d ⟩ Referenced in 190a.
⟨ Copy block comment from scrap 49a ⟩ Referenced in 144a.
⟨ Copy file name into file 145a ⟩ Referenced in 144a.
⟨ Copy fragment title into file 146c ⟩ Referenced in 144a.
⟨ Copy label from scrap into file 144b ⟩ Referenced in 144a.
⟨ Copy label from source into 71b ⟩ Referenced in 57a, 70a, 86a, 95a.
⟨ Copy macro into `file` 145b ⟩ Referenced in 144a.
⟨ Copy version info into file 71a ⟩ Referenced in 70a, 95a, 144a.
⟨ Copy version info into md file 87a ⟩ Referenced in 86a.
⟨ Copy version info into tex file 58a ⟩ Referenced in 57a.
⟨ Copy `defs->scrap` to `file` 141 ⟩ Referenced in 139a.
⟨ Copy `source_file` into `md_file` 85a ⟩ Referenced in 84b.
⟨ Copy `source_file` into `tex_file` 56c ⟩ Referenced in 55b.
⟨ Create a new label entry 190f ⟩ Referenced in 190e.
⟨ Create new name entry 158 ⟩ Referenced in 154a, 157.
⟨ Create new scrap, managed by `writer` 128c ⟩ Referenced in 128b.
⟨ End block 62c ⟩ Referenced in 50a, 61b, 63a.
⟨ Enter the next argument 168b ⟩ Referenced in 168a.
⟨ Expand macro into *file* 58b ⟩ Referenced in 57a.
⟨ Expand md macro into *file* 87b ⟩ Referenced in 86a.
⟨ Expand md tab into spaces 94 ⟩ Referenced in 93a.
⟨ Expand tab into spaces 69c ⟩ Referenced in 68, 143d.
⟨ Extend goto graph with `tree->spelling` 181 ⟩ Referenced in 180b.
⟨ Fill in the md scrap 89b ⟩ Referenced in 88ab.
⟨ Fill in the middle of the scrap environment 62d ⟩ Referenced in 59, 60a.
⟨ Find a free temporary file 115b ⟩ Referenced in 115a.
⟨ Find a new chunk of memory 193c ⟩ Referenced in 193b.
⟨ Finish md scrap 89c ⟩ Referenced in 88ab.

⟨ Finish the cross-reference environment 64b ⟩ Referenced in 59, 60a.
⟨ Finish the scrap environment 63a ⟩ Referenced in 59, 60a.
⟨ For all remaining scraps 150c ⟩ Referenced in 150a.
⟨ Format a file index entry 76c ⟩ Referenced in 76b.
⟨ Format a md file index entry 100c ⟩ Referenced in 100b.
⟨ Format a md index entry 102b ⟩ Referenced in 102a.
⟨ Format a md user index entry 104c ⟩ Referenced in 104b.
⟨ Format a user index entry 82b ⟩ Referenced in 82a.
⟨ Format an index entry 79b ⟩ Referenced in 78b.
⟨ Format macro name 73 ⟩ Referenced in 70a.
⟨ Format macro parameters 54a ⟩ Referenced in 73, 75.
⟨ Format md macro name 97 ⟩ Referenced in 95a.
⟨ Forward declarations for scraps.c 139b, 140, 147b, 166, 184c ⟩ Referenced in 124a.
⟨ Function prototypes 38a, 54b, 67a, 69b, 81a, 83, 113c, 114a, 117a, 119a, 124b, 126b, 134b, 137b, 148b, 152c, 155b, 169d, 179c, 186a, 187c, 190b, 192c ⟩ Referenced in 20.
⟨ Get comment delimiters 164b ⟩ Referenced in 164a.
⟨ Get label from 189c ⟩ Referenced in 71b, 131a, 144b.
⟨ Get label while collecting scrap 131a ⟩ Referenced in 130.
⟨ Get the nth previous character from a scrap 179b ⟩ Referenced in 178c.
⟨ Get the nth previous character from an argument 179a ⟩ Referenced in 178c.
⟨ Global variable declarations 27, 28b, 29a, 42c, 47c, 63b, 117b, 127a, 152a, 192b ⟩ Referenced in 20.
⟨ Global variable definitions 28ac, 29b, 42d, 47d, 63c, 117c, 127b, 152b ⟩ Referenced in 24e.
⟨ Handle an "at" character 120 ⟩ Referenced in 118c.
⟨ Handle at-sign during scrap accumulation 130 ⟩ Referenced in 129.
⟨ Handle macro invocation in scrap 133c ⟩ Referenced in 130.
⟨ Handle macro parameter substitution 50d ⟩ Referenced in 144a.
⟨ Handle optional per-file flags 164a ⟩ Referenced in 163.
⟨ Handle tab characters on output 143d ⟩ Referenced in 141.
⟨ Handle the file name in `argv[arg]` 34 ⟩ Referenced in 33.
⟨ Handle `EOF` 122b ⟩ Referenced in 118c.
⟨ HTML-escape character c 93b ⟩ Referenced in 93a.
⟨ If we break the line at this word 49c ⟩ Referenced in 49a.
⟨ Include a fragment use in comment 148a ⟩ Referenced in 147b.
⟨ Include an embedded scrap in comment 147c ⟩ Referenced in 147b.
⟨ Include block comment in a scrap 48b ⟩ Referenced in 130.
⟨ Include files 21 ⟩ Referenced in 20.
⟨ Indent suppressed 143c ⟩ Referenced in 143a.
⟨ Insert appropriate indentation 143a ⟩ Referenced in 141, 142b.
⟨ Insert debugging information if required 142b ⟩ Referenced in 141, 144a.
⟨ Insert section closing marker 142d ⟩ Referenced in 141.
⟨ Insert section opening marker 142c ⟩ Referenced in 141.
⟨ Interpret at-sequence 57a ⟩ Referenced in 56c.
⟨ Interpret command-line arguments 29cd ⟩ Referenced in 25a.
⟨ Interpret md at-sequence 86a ⟩ Referenced in 85a.
⟨ Interpret the argument string `s` 30 ⟩ Referenced in 29d.
⟨ Italic "*whatever*" 72b ⟩ Referenced in 70a.
⟨ Italic md "*whatever*" 96c ⟩ Referenced in 95a.
⟨ Limits 22b ⟩ Referenced in 20.
⟨ Look for end of scrap name and return 172 ⟩ Referenced in 171.
⟨ Make this argument 169a ⟩ Referenced in 168b.
⟨ Move a word to the file 49b ⟩ Referenced in 49a.
⟨ Move the temporary file to the target, if required 116a ⟩ Referenced in 115a.
⟨ Open an include file 121 ⟩ Referenced in 120.
⟨ Operating System Dependencies 25b ⟩ Referenced in 20, 24e.
⟨ Otherwise pop the current arg 178b ⟩ Referenced in 177b.
⟨ Perhaps add an include path 31b ⟩ Referenced in 29d.
⟨ Perhaps add letters to the page numbers 150d ⟩ Referenced in 150a.
⟨ Perhaps comment this macro 146a ⟩ Referenced in 50d, 145b.

⟨ Perhaps get the extract name 32b ⟩ Referenced in 29d.
⟨ Perhaps get the hyperref options 32a ⟩ Referenced in 29d.
⟨ Perhaps get the prepend path 31a ⟩ Referenced in 29d.
⟨ Perhaps get the version info string 31c ⟩ Referenced in 29d.
⟨ Perhaps put a delayed indent 146b ⟩ Referenced in 49a, 146a.
⟨ Perhaps —return— a character from the current arg 177c ⟩ Referenced in 177b.
⟨ Perhaps skip white-space 47a ⟩ Referenced in 46b.
⟨ Perhaps start a new arg 178a ⟩ Referenced in 177b.
⟨ Process a file 37 ⟩ Referenced in 34.
⟨ Process the remaining arguments (file names) 33 ⟩ Referenced in 25a.
⟨ Put out the indent 143b ⟩ Referenced in 50d, 143a.
⟨ Read line in `.aux` file 149b ⟩ Referenced in 149a.
⟨ Rename the temporary file to the target 116c ⟩ Referenced in 116ab.
⟨ Reverse cross-reference lists 50b ⟩ Referenced in 38b.
⟨ Rob's ordering 78c ⟩ Referenced in 78b, 102a.
⟨ Save label to label store 190e ⟩ Referenced in 131a.
⟨ Save macro name 133d ⟩ Referenced in 133c.
⟨ Save macro parameters 52 ⟩ Referenced in 133c.
⟨ Scan at-sequence 41 ⟩ Referenced in 38c.
⟨ Scan the source file, looking for at-sequences 38c ⟩ Referenced in 38b.
⟨ Search for label(*Found*,*Notfound*) 191c ⟩ Referenced in 190ae.
⟨ Search scraps 183 ⟩ Referenced in 180a.
⟨ Set locale information 32c ⟩ Referenced in 25a.
⟨ Set up name, args and next 137c ⟩ Referenced in 137a.
⟨ Show md presence of a block comment 96a ⟩ Referenced in 95a.
⟨ Show presence of a block comment 48a ⟩ Referenced in 70a.
⟨ Skip at-L language definition line 40 ⟩ Referenced in 57a, 86a.
⟨ Skip commented-out code 71e ⟩ Referenced in 70a, 95a, 130.
⟨ Skip invisibles on *p* 156b ⟩ Referenced in 156a.
⟨ Skip over a block comment 49d ⟩ Referenced in 183.
⟨ Skip over a scrap use 184b ⟩ Referenced in 183.
⟨ Skip over an in-text scrap 43c ⟩ Referenced in 41.
⟨ Skip over at at 184a ⟩ Referenced in 183.
⟨ Skip over at-sign or go to skipped 44a ⟩ Referenced in 43c.
⟨ Skip over index entries 71d ⟩ Referenced in 70a, 95a.
⟨ Skip to the next nw-char 47b ⟩ Referenced in 44b.
⟨ Skip until scrap begins, then return name 169c ⟩ Referenced in 167.
⟨ Skip whitespace 45a ⟩ Referenced in 44b.
⟨ Sort *key* of size *n* for *ordering* 79a ⟩ Referenced in 78b, 102a.
⟨ Start block 62a ⟩ Referenced in 50a, 61b.
⟨ Step *i* to the next valid scrap 150b ⟩ Referenced in 150a.
⟨ Step to next sector 42a ⟩ Referenced in 41, 57a, 86a.
⟨ Switch block 62b ⟩ Referenced in 61b.
⟨ Type declarations 22a, 50c, 114c, 151ab, 168c, 170b, 173d, 185c, 192a ⟩ Referenced in 20.
⟨ Warn (only once) about needing to rerun after Latex 126e ⟩ Referenced in 125a, 149b.
⟨ Write abbreviated definition list 74 ⟩ Referenced in 73, 75.
⟨ Write defining scrap numbers 80a ⟩ Referenced in 79b.
⟨ Write defs references 188a ⟩ Referenced in 187d.
⟨ Write file defs 64c ⟩ Referenced in 59.
⟨ Write file's defining scrap numbers 77a ⟩ Referenced in 76c.
⟨ Write in-text scrap 63d ⟩ Referenced in 57a.
⟨ Write index of file names 76a ⟩ Referenced in 57a.
⟨ Write index of macro names 77b ⟩ Referenced in 57a.
⟨ Write index of user-specified names 81c ⟩ Referenced in 57a.
⟨ Write LaTeX limbo definitions 56a ⟩ Referenced in 55b.
⟨ Write macro definition 60a ⟩ Referenced in 57a.
⟨ Write macro defs 64d ⟩ Referenced in 60a.
⟨ Write macro refs 65a ⟩ Referenced in 60a.

⟨ Write md defining scrap numbers 103a ⟩ Referenced in 102b.
⟨ Write md defs references 107c ⟩ Referenced in 107b.
⟨ Write md file cross-refs 90b ⟩ Referenced in 88a.
⟨ Write md file defs 90c ⟩ Referenced in 90b.
⟨ Write md file's defining scrap numbers 100d ⟩ Referenced in 100c.
⟨ Write md in-text scrap 90a ⟩ Referenced in 86a.
⟨ Write md index of file names 100a ⟩ Referenced in 86a.
⟨ Write md index of macro names 101a ⟩ Referenced in 86a.
⟨ Write md index of user-specified names 104a ⟩ Referenced in 86a.
⟨ Write md macro cross-refs 90d ⟩ Referenced in 88b.
⟨ Write md macro definition 88b ⟩ Referenced in 86a.
⟨ Write md macro defs 91a ⟩ Referenced in 90d.
⟨ Write md macro refs 91b ⟩ Referenced in 90d.
⟨ Write md output file definition 88a ⟩ Referenced in 86a.
⟨ Write md referencing scrap numbers 103b ⟩ Referenced in 102b.
⟨ Write md uses references 106b ⟩ Referenced in 106a.
⟨ Write one def reference 188b ⟩ Referenced in 188a.
⟨ Write one md def reference 108 ⟩ Referenced in 107c.
⟨ Write one md referenced scrap 107a ⟩ Referenced in 106c, 108.
⟨ Write one md use reference 106c ⟩ Referenced in 106b.
⟨ Write one referenced scrap 187b ⟩ Referenced in 187a, 188b.
⟨ Write one use reference 187a ⟩ Referenced in 186c.
⟨ Write out `files->spelling` 115a ⟩ Referenced in 114b.
⟨ Write output file definition 59 ⟩ Referenced in 57a.
⟨ Write referencing scrap numbers 80b ⟩ Referenced in 79b.
⟨ Write the hyperlink usage macro 56b ⟩ Referenced in 56a.
⟨ Write the label to file 190c ⟩ Referenced in 190a.
⟨ Write the macro's name 60b ⟩ Referenced in 60a, 79b.
⟨ Write the md macro's name 88c ⟩ Referenced in 88b, 102b.
⟨ Write uses references 186c ⟩ Referenced in 186b.

5.3 Identifiers

Knuth prints his index of identifiers in a two-column format. I could force this automatically by emitting the `\twocolumn` command; but this has the side effect of forcing a new page. Therefore, it seems better to leave it this up to the user.

`add_to_use`: 52, 133c, 134a, 134b, 174a.
`add_uses`: 133ab, 183, 184c, 185b.
`already_warned`: 126e, 127a, 127b, 149a.
`arena`: 193a, 193bc, 194ab.
`arena_free`: 37, 192c, 194b.
`arena_getmem`: 31b, 36, 43b, 53, 124c, 127d, 128c, 133ab, 134a, 137c, 138a, 153b, 158, 170c, 173c, 180a, 181, 182, 183, 185b, 190f, 192c, 193b.
`Arglist`: 50d, 55a, 58b, 73, 75, 84a, 87b, 97, 99, 124b, 133c, 135, 136b, 137abc, 138ab, 139a, 140, 145b, 146c, 147b, 152c, 170b, 170c, 171, 173cd, 174b, 176b, 177a, 178a, 184b.
`arglist_count`: 140.
`arglist_lookup`: 135, 138b, 140, 184b.
`arglist_register`: 133d, 140.
`arglist_reset`: 140, 192c, 194b.
`arglist_table`: 140.
`ArgManager`: 176c, 177ab, 178c, 180a, 183, 189b.
`ArgMgr`: 176b, 176c, 177ab, 178c.
`argpop`: 177b, 183, 184ab.
`ARG_CHR`: 60b, 73, 75, 88c, 97, 99, 137c, 147a, 148a, 168b, 168c, 172, 173b.
`backup`: 136a, 143a.
`blockBuff`: 44b, 47c, 47d, 48b.
`buildArglist`: 170c, 172, 173c, 174b.

build_gotos: 180a, 180b.
Chunk: 192d, 193ac, 194a.
clear_marks: 113a, 113b.
collect_file_name: 42e, 59, 88a, 152c, 163.
collect_lang_def: 39b, 152c, 161.
collect_macro_name: 43a, 60a, 88b, 152c, 167.
collect_numbers: 37, 148b, 149a.
collect_scrap: 42e, 43a, 52, 124b, 128b, 173c.
collect_scrap_name: 58b, 73, 87b, 97, 133c, 152c, 171, 174a.
collect_weave_format: 39c, 152c, 162.
command_name: 29a, 29bc, 30, 33, 37, 41, 43c, 44a, 55b, 65a, 73, 75, 84b, 91b, 97, 99, 113b, 115b, 116c, 120, 121, 122a, 123a, 126e, 129, 130, 132, 138b, 140, 145b, 154b, 161, 162, 163, 164ab, 167, 168a, 169bc, 171, 172, 184b.
comment_ArglistElement: 147a, 147b, 148a.
comment_begin: 49a, 142cd, 146a, 166.
comment_end: 49a, 142cd, 146a, 166.
comment_mid: 49a, 166.
compare: 153a, 154a, 155a.
compare_flag: 27, 28a, 30, 116a.
copy_scrap: 54a, 55a, 62d, 63d, 68.
current_sector: 37, 42ab, 42c, 42d, 77b, 81c, 101a, 104a, 128c, 132, 167, 171.
dangling_flag: 27, 28a, 30, 82b, 104c.
defs: 43b, 50d, 58b, 64cd, 73, 74, 75, 77a, 80a, 82b, 87b, 90c, 91a, 97, 99, 100d, 103a, 104c, 106c, 107a, 108, 110, 112, 115a, 123c, 124b, 126a, 128c, 133b, 137c, 139a, 141, 142bcd, 145ab, 146ac, 147ac, 151b, 158, 173d, 174c, 183, 187abd, 188b.
delayed_indent: 50d, 139b, 141, 143c, 145b, 146ab.
depths: 175d, 180a, 181, 182.
double_at: 118a, 120, 123a.
dump_scrap_text: 112, 126b, 135.
Embed_Node: 50d, 137c, 147c, 173d.
emit_name_json: 110, 111a.
EQUAL: 153a, 154a, 155a.
exit: 21, 25a, 33, 37, 43c, 44a, 66b, 113b, 115b, 120, 121, 122a, 123a, 129, 130, 132, 138b, 140, 145b, 163, 167, 168a, 169bc, 171, 172, 177a, 184b.
EXTENSION: 153a, 154a, 155a.
extract_flag: 27, 28a, 30, 32b.
extract_fragment: 112, 113b.
extract_name: 27, 28a, 32b, 37.
extra_scraps: 62d, 63a, 63b, 63c, 75, 89bc, 99.
FALSE: 22a, 27, 28a, 30, 31abc, 32ab, 37, 56c, 57a, 58b, 62c, 63d, 65b, 75, 80a, 81b, 82b, 85ab, 86a, 87b, 90a, 92, 99, 103a, 104c, 106c, 108, 113a, 120, 123a, 145b, 149a, 155a, 158, 159, 161, 164a, 165a, 185a, 187a, 188b, 189ab.
fclose: 21, 55b, 84b, 115ab, 116b, 122b, 149a.
FILE: 21, 55ab, 61a, 65b, 67c, 68, 75, 76b, 78b, 82a, 84ab, 89a, 92, 93a, 98, 99, 100b, 102a, 104b, 106a, 107b, 109, 110, 112, 115a, 116b, 118ab, 124b, 125ab, 126b, 135, 139a, 147b, 149a, 186ab, 187cd, 190ab, 195.
file_line: 123c, 126d, 128c, 129, 141.
file_name: 38ab, 54b, 55b, 83, 84b, 123c, 126c, 128c, 129, 142bc.
file_names: 37, 38b, 50b, 76a, 100a, 113b, 114a, 152a, 152b, 163.
find_lang: 159, 165a.
find_name: 111b, 113b.
first: 106c, 107a, 108, 110, 111a, 113b, 124b, 125a, 187ab, 188b, 189b, 193a, 194b, 195.
fopen: 21, 55b, 84b, 115b, 116b, 121, 123a, 149a.
format_defs_refs: 59, 60a, 187c, 187d.
format_entry: 55a, 77b, 78b.
format_file_entry: 55a, 76a, 76b.
format_user_entry: 55a, 81c, 82a.
format_uses_refs: 59, 60a, 186a, 186b.
fprintf: 21, 30, 32c, 33, 37, 38b, 41, 43c, 44a, 48a, 54a, 55b, 56b, 59, 61b, 64c, 65a, 66b, 72ab, 73, 75, 76c, 82b, 84b, 88a, 89b, 90c, 91b, 96bc, 97, 99, 100c, 104c, 106c, 108, 109, 110, 112, 113b, 115ab, 116c, 120, 121, 122a, 123a, 125a, 126e, 129, 130, 132, 135, 138b, 140, 142bcd, 145ab, 147ab, 154b, 161, 162, 163, 164ab, 167, 168a, 169bc,

171, 172, 177a, 184b, 187a, 188b, 190cd, 191ab.
fputs: 21, 48a, 49a, 50d, 54a, 56a, 58a, 59, 60a, 61ab, 62abcd, 63a, 64abcd, 65ab, 67c, 68, 70a, 71a, 72ab, 73, 74, 75, 76ac, 77ab, 79b, 80ab, 81c, 82b, 87a, 88ab, 89ab, 90c, 91ab, 92, 93ab, 95a, 96ab, 97, 98, 99, 100acd, 101a, 102b, 103ab, 104ac, 106bc, 107ac, 108, 109, 110, 111a, 112, 125a, 141, 145a, 146a, 186c, 187ab, 188ab.
getc: 21, 116b, 118c, 120, 121, 122ab, 123a.
getenv: 21, 32c.
get_scrap_defs: 107b, 124b, 126a.
get_scrap_uses: 106a, 110, 112, 124b, 125c.
goto_lookup: 176a, 181, 182, 183.
Goto_Node: 175b, 175cd, 176a, 180a, 181, 182, 183.
GREATER: 153a, 154a, 155a.
has_sector: 77b, 81a, 81b, 81c, 101a, 104a.
hyperoptions: 27, 28a, 32a, 56ab.
hyperopt_flag: 27, 28a, 30, 32a.
hyperref_flag: 27, 28a, 30, 32a, 56a.
incl: 27, 28a, 31b, 36, 121.
includepath_flag: 27, 28a, 30, 31b.
include_depth: 118a, 121, 122b, 123a.
init_scraps: 38b, 124b, 124c.
instance: 58b, 87b, 137a, 137bc, 145b.
isgraph: 21, 122a, 163, 171.
islower: 21, 156a.
isspace: 21, 63a, 89c, 132, 163, 164a, 167, 169c.
json_puts: 109, 110.
label_node: 190f, 191cd, 192a, 192b.
label_tab: 191c, 191d, 192b.
LangEntry: 159, 165a.
lang_table: 159.
lang_to_flags: 159, 165a.
LESS: 153a, 154a, 155a.
letter: 123c, 125a, 128c, 150d.
load_entry: 78a, 78b.
lookup: 50d, 136b, 137c.
macro_names: 37, 38b, 50b, 77b, 101a, 113b, 114a, 152a, 152b, 169b.
main: 25a, 195.
malloc: 21, 78b, 102a, 177a, 194a.
Manager: 127c, 127d, 128ab, 134c, 135, 136a, 138b, 141, 176c, 178c, 183.
map_flag: 27, 28a, 30, 37.
markdown_flag: 27, 28a, 30, 37.
MAX_ARGLISTS: 140.
max_depth: 175d, 180a, 181, 182.
MAX_INDENT: 58b, 87b, 114c, 115a, 142a, 143b.
MAX_NAME_LEN: 22b, 132, 135, 138b, 163, 167, 171, 173a, 184b, 189c.
md_copy_scrap: 84a, 89b, 90a, 93a.
md_format_defs_refs: 84a, 90bd, 107b.
md_format_entry: 84a, 101a, 102a.
md_format_file_entry: 84a, 100a, 100b.
md_format_user_entry: 84a, 104a, 104b.
md_format_uses_refs: 84a, 90bd, 106a.
md_load_entry: 101b, 102a.
md_print_scrap_numbers: 84a, 90c, 91ab, 92, 100d, 103b.
md_scrap_type: 93a, 95a.
md_write_arg: 84a, 88c, 89a, 95a, 99.
md_write_ArglistElement: 84a, 97, 99.
md_write_literal: 84a, 97, 98, 99.
Move_Node: 175c, 176a, 181, 182.
Name: 42e, 43a, 50d, 55a, 58b, 59, 60a, 68, 73, 75, 76b, 78ab, 79a, 81ab, 82a, 84a, 87b, 88ab, 93a, 97, 99, 100b, 101b, 102a, 104b, 106c, 108, 110, 111ab, 112, 113abc, 114ab, 132, 133c, 134ab, 136b, 137ac, 145b, 147b, 151b, 152abc,

154a, 155a, 157, 158, 163, 167, 169d, 170abc, 172, 173c, 174c, 175a, 178a, 180ab, 183, 184c, 185bc, 187a, 188b, 189b.
name_add: 133ab, 152c, 157, 163.
Name_Node: 175a, 175b, 181, 182, 183.
next: 27, 30, 31b, 36, 43b, 50d, 64cd, 65ab, 73, 74, 75, 77a, 80ab, 82b, 90c, 91ab, 92, 97, 99, 100d, 103ab, 104c, 106bc, 107c, 108, 110, 112, 117a, 121, 123b, 127d, 133ab, 134ac, 136b, 137ac, 138a, 139a, 147a, 148a, 151a, 170bc, 173c, 174abd, 175abc, 176a, 178a, 181, 182, 183, 185abc, 186c, 187a, 188ab, 192d, 193c, 194a, 195.
number_flag: 27, 28a, 30, 37, 149a.
num_scraps: 78b, 102a, 124a, 124b.
nw_char: 28b, 28c, 37, 38c, 41, 43c, 44a, 45b, 47b, 48b, 49d, 52, 53, 54a, 56c, 57a, 59, 66a, 67c, 68, 69a, 70a, 71d, 72a, 76c, 82b, 85a, 86a, 93a, 95a, 96b, 118c, 120, 123a, 129, 130, 131a, 132, 133cd, 135, 138b, 141, 143c, 144a, 145ab, 147ab, 148a, 154b, 161, 162, 163, 167, 168ab, 169c, 171, 172, 173a, 184ab, 187a, 188b, 189ac.
op_char: 189a, 189b.
output_flag: 27, 28a, 30, 37.
page: 65b, 80a, 82b, 92, 103a, 104c, 106c, 107a, 108, 123c, 124b, 125ab, 128c, 149ab, 150bd, 187ab, 188b, 195.
Parameters: 50c, 53, 58b, 87b, 124b, 138b, 139a, 141.
pass1: 37, 38a, 38b, 70a, 95a.
pending_lang: 39a, 152a, 152b, 165ab.
pop: 49abd, 53, 134c, 135, 138bc, 141, 143a, 144ac, 177b.
pop_scrap_name: 138b, 145b.
PREFIX: 153a, 154a, 155a.
prefix_add: 152c, 154a, 169b.
prev_char: 178c, 189b.
prev_sector: 37, 42a, 42c, 42d.
print_scrap_numbers: 55a, 64cd, 65a, 65b, 77a, 80b.
push: 48b, 52, 127d, 128a, 129, 130, 131a, 133cd.
pushArglist: 177a, 178a, 184b.
pushs: 52, 128a, 131a, 133d.
putc: 21, 49ab, 56c, 57a, 68, 69c, 73, 75, 76c, 77a, 79b, 80ab, 82b, 85ab, 86a, 93ab, 94, 95a, 97, 99, 100c, 102b, 103a, 125a, 141, 143bd, 144a, 145b, 146abc.
reject_match: 180a, 183, 189b.
remove: 21, 116ab.
reverse: 174c, 174d.
reverse_lists: 50b, 152c, 174c.
robs_strcmp: 78c, 111b, 155b, 156a, 157, 158, 185b.
root: 175d, 180a, 181, 182, 183, 184c, 185b.
save_string: 31b, 36, 121, 128c, 152c, 153b, 154a, 158, 170a, 174b.
SCRAP: 124a, 124c, 128c.
ScrapEntry: 123c, 124ac, 128c.
scraps: 22e, 23a, 27, 54a, 55a, 59, 60a, 61b, 63a, 65b, 84a, 88ab, 89c, 90bd, 92, 110, 124a, 124c, 128c, 149ab, 150bc, 183, 195.
scrap_array: 124a, 125ac, 126acd, 128c, 129, 133ab, 135, 141, 142bc, 149ab, 150bd, 183, 186b, 187d.
SCRAP_BITS: 124a.
scrap_file_line: 110, 112, 126b, 126d.
scrap_file_name: 110, 112, 126b, 126c.
scrap_flag: 27, 28a, 30, 59, 90b.
scrap_is_in: 183, 184c, 185a.
SCRAP_MASK: 124a, 128c.
Scrap_Node: 43b, 50d, 55a, 65b, 74, 75, 77a, 80ab, 82b, 84a, 92, 97, 99, 100d, 103ab, 104c, 106c, 108, 110, 112, 124b, 133ab, 134a, 137c, 139a, 151a, 151b, 173cd, 174cd, 183, 184c, 185a, 187a, 188b.
SCRAP_SHIFT: 124a, 128c.
SCRAP_SIZE: 124a, 124c, 128c.
scrap_type: 48a, 67b, 68, 70a, 72ab, 73, 75.
search: 38b, 179c, 180a, 195.
sector: 54b, 55ab, 77b, 78b, 79b, 81bc, 82ab, 83, 84ab, 101a, 102ab, 104abc, 110, 123c, 128c, 132, 133abd, 135, 145b, 148a, 151b, 152c, 154ab, 155a, 157, 158, 167, 169b, 171, 183.
setlocale: 21, 32c.
size_t: 21, 192c, 193b, 194a.

Slab: 123b, 123c, 127cd, 128c, 134c.
slab: 123b, 123c, 128c, 135, 141, 183.
SLAB_SIZE: 123b, 127d, 134c, 136a, 179b.
source_file: 118a, 118c, 119b, 120, 121, 122ab, 123a.
source_get: 38c, 39a, 40, 41, 42ab, 43c, 44a, 45ab, 46a, 47ab, 52, 54a, 56c, 57a, 58ab, 63ad, 68, 70a, 71cde, 72a, 76a, 77b, 81c, 85ab, 86a, 87ab, 89c, 90a, 93a, 95a, 96b, 100a, 101a, 104a, 117a, 118c, 121, 122b, 129, 130, 131b, 132, 133c, 161, 162, 163, 164ab, 167, 168ab, 169c, 171, 172, 173a.
source_last: 54a, 68, 93a, 117a, 118c.
source_line: 41, 43c, 44a, 117b, 117c, 118c, 119b, 120, 121, 122ab, 123a, 128c, 130, 132, 154b, 161, 162, 163, 164ab, 167, 169b, 171, 172.
source_name: 34, 35, 36, 37, 41, 43c, 44a, 117b, 117c, 120, 121, 122ab, 123a, 128c, 130, 132, 154b, 161, 162, 163, 164ab, 167, 168a, 169bc, 171, 172.
source_open: 38b, 55b, 84b, 117a, 123a.
source_peek: 45a, 46a, 47a, 117a, 118c, 119b, 120, 121, 122b, 123a.
source_ungetc: 45b, 46a, 47b, 77b, 101a, 119a, 119b.
stack: 118b, 121, 122b.
stderr: 21, 30, 32c, 33, 37, 38b, 41, 43c, 44a, 55b, 64c, 65a, 66b, 73, 75, 84b, 90c, 91b, 97, 99, 113b, 115ab, 116c, 120, 121, 122a, 123a, 126e, 129, 130, 132, 138b, 140, 145b, 154b, 161, 162, 163, 164ab, 167, 168a, 169bc, 171, 172, 177a, 184b, 190d, 191ab.
strlen: 21, 49a, 149b, 153ab, 179a, 189b.
sym_char: 189a, 189b.
tex_flag: 27, 28a, 30, 37, 38b, 145b.
toupper: 21, 156a.
TRUE: 22a, 27, 28a, 30, 32a, 37, 54a, 57a, 58b, 62ad, 65b, 75, 80a, 81b, 82b, 85ab, 86a, 87b, 89bc, 92, 99, 103a, 104c, 106c, 108, 112, 120, 125b, 126e, 145b, 155a, 158, 159, 161, 164a, 185a, 187a, 188b, 189ab.
update_delimit_scrap: 41, 57a, 69a, 69b, 86a.
user_langs: 159, 161.
user_lang_count: 159, 161.
user_lang_names: 159, 161.
user_names: 38b, 50b, 81c, 104a, 133ab, 152a, 152b, 180a.
Uses: 56a, 106ab, 107b, 110, 112, 124b, 125c, 126a, 184c, 185b, 185c, 186b, 187d.
uses: 65a, 80b, 82b, 91b, 103b, 104c, 108, 110, 112, 123c, 125c, 128c, 133a, 134a, 151b, 158, 159, 174c, 183, 185c, 186b, 188b.
verbose_flag: 27, 28a, 30, 38b, 55b, 84b, 115a.
version_info_flag: 27, 28a, 30, 31c.
version_string: 27, 28a, 31c, 58a, 71a, 87a.
weave_flag: 27, 28a, 30, 37.
weave_format: 27, 28a, 30, 37, 162.
weave_format_from_cli: 27, 28a, 30, 37, 162.
write_arg: 55a, 60b, 61a, 70a, 75.
write_ArglistElement: 55a, 73, 75.
write_extract: 37, 113b, 114a.
write_files: 37, 113c, 114b.
write_label: 71b, 144b, 190a, 190b.
write_literal: 55a, 67c, 73, 75.
write_map: 37, 111a, 114a.
write_md: 37, 83, 84b.
write_scraps: 50d, 58b, 75, 87b, 99, 115a, 124b, 139a, 145b, 147c.
write_scrap_ref: 65b, 80a, 82b, 92, 103a, 104c, 107a, 124b, 125a, 125b, 187b.
write_single_scrap_ref: 54a, 59, 60a, 65a, 74, 77a, 80b, 82b, 88ab, 91b, 97, 99, 100d, 103b, 104c, 124b, 125b, 146ac, 190c.
write_tex: 37, 54b, 55b.
xref_flag: 27, 28a, 30, 146ac.

Bibliography

[1] A[lfred] V. Aho and M[argaret] J. Corasick, *Efficient string matching: An aid to bibliographic search.*, CACM **18** (1975), no. 6, 333–340. 176

[2] C[hris] Hanson, *Efficient stack allocation for tail-recursive languages.*, (1990). 193

[3] Donald E. Knuth, *Literate programming*, The Computer Journal **27** (1984), no. 2, 97–111. 1

[4] ———, *The TEXbook*, Computers and Typesetting, vol. 1986aA, Addison-Wesley, Reading, MA, USA, 1986. 1

[5] ———, *TEX: The program*, Computers and Typesetting, vol. B, Addison-Wesley, Reading, MA, USA, 1986b1986. 1

[6] ———, METAFONT: THE PROGRAM, Computers and Typesetting, vol. D, Addison-Wesley, Reading, MA, USA, 1986d1986. 1

[7] Leslie Lamport, *LATEXa document preparation systemuser's guide and reference manual*, Addison-Wesley, Reading, MA, USA, 1985. 1

[8] Silvio Levy, *WEB adapted to C, another approach*, TUB **8** (1987), no. 1, 12–13. 1

[9] Norman Ramsey, *Literate-programming tools need not be complex*, Report at `ftp.cs.princeton.edu` in `/reports/1991/351.ps.Z`. Software at `ftp.cs.princeton.edu` in `/pub/noweb.shar.Z` and at `bellcore.com` in `/pub/norman/noweb.shar.Z`. CS-TR-351-91, Department of Computer Science, Princeton University, August 1992, Submitted to *IEEE Software*. 1

[10] Ross Williams, *Funnelweb user's manual*, `ftp.adelaide.edu.au` in `/pub/compression` and `/pub/funnelweb`, University of Adelaide, Adelaide, South Australia, Australia, 1992. 1